ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO

KẾT QUẢ NGHIÊN CỨU KHOA HỌC

HIỆN THỰC CƠ CHẾ DYNAMIC PARTIAL RECONFIGURATION CHO THIẾT KẾ MÃ HÓA AES-256 TRÊN FPGA

Người hướng dẫn: PGS.TS. TRƯƠNG QUANG VINH

Sinh viên thực hiện:

STT	MSSV	Họ và tên
1	2451167	Lê Minh Thịnh
2	2451101	Nguyễn Bảo Long
3	2451155	Ông Bảo Thành

Thành phố Hồ Chí Minh, tháng 11 năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 6

1.1 Đặt Vấn Đề . . . 6

1.2 Mục Tiêu Nghiên Cứu . . . 7

1.3 Phạm vi và nội dung thực hiện . . . 8

1.4 Cấu trúc báo cáo . . . 9

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 11

2.1 Tổng quan về thuật toán mã hóa AES-256 . . . 11

2.2 Các phép biến đổi trong thuật toán AES . . . 15

2.3 Thuật toán Key Expansion của AES-256 . . . 20

2.4 Các phương pháp hiện thực S-box trên FPGA . . . 25

2.5 Nguyên lý hoạt động của Dummy Noise . . . 30

2.6 Nền tảng FPGA và kiến trúc SoC . . . 32

2.7 Cơ chế triển khai và lựa chọn biến thể phần cứng . . . 36

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG MÃ HÓA AES-256 40

3.1 Kiến trúc tổng quan hệ thống 40

3.1.1 Yêu cầu thiết kế . . . 40

3.1.2 Kiến trúc PS–PL . . . 40

3.1.3 Nguyên lý hoạt động tổng thể . . . 41

3.2 Thiết kế chi tiết lõi mã hóa AES-256 41

3.2.1 Cấu trúc tổng thể AES-256 Core . . . 41

3.2.2 Thiết kế Cipher Datapath . . . 42

3.2.3 Thiết kế Key Expansion . . . 44

3.2.4 Thiết kế logic điều khiển Round . . . 46

3.2.5 Trình tự xử lý và đồng bộ tín hiệu . . . 47

3.3 Thiết kế các biến thể phần cứng AES-256 48

3.3.1 Biến thể LUT S-box . . . 49

3.3.2 Biến thể BRAM S-box . . . 49

3.3.3 Biến thể Composite Field . . . 51

3.3.4 Biến thể Dummy Noise . . . 52

3.3.5 So sánh cấu trúc các biến thể . . . 53

3.4 Thiết kế giao tiếp PS–PL **54**
3.4.1 Kiến trúc kết nối PS–PL . . . 54
3.4.2 Custom AXI4-Lite Controller . . . 55
3.4.3 Register Map và tín hiệu điều khiển . . . 56
3.4.4 Handshake và trình tự trao đổi dữ liệu . . . 56
3.5 Thiết kế kiến trúc DPR và Replica **57**
3.5.1 Kiến trúc Top-level DPR . . . 57
3.5.2 Kiến trúc Top-level Replica . . . 59
3.5.3 Giao diện chung và cơ chế chuyển đổi biến thể . . . 60
CHƯƠNG 4: HIỆN THỰC HỆ THỐNG TRÊN FPGA VÀ ARM/LINUX **62**
4.1 Môi trường và quy trình hiện thực **62**
4.1.1 Nền tảng phần cứng và công cụ thiết kế . . . 62
4.1.2 Phân chia quy trình triển khai . . . 62
4.2 Tích hợp hệ thống SoC trong Vivado **63**
4.2.1 Xây dựng Block Design và các kết nối PS–PL . . . 63
4.2.2 Phân phối Clock, Reset và giao diện Top-level . . . 64
4.2.3 Gán địa chỉ cho AXI Controller . . . 64
4.3 Hiện thực kiến trúc DPR **65**
4.3.1 Xác định vùng tái cấu hình trong Top-level . . . 65
4.3.2 Xây dựng các DFX Configurations và Synthesis Runs . . . 66
4.3.3 Parent Run, Child Runs và kết quả Routing . . . 67
4.3.4 Phân vùng vật lý và Pblock . . . 67
4.4 Hiện thực kiến trúc Replica **68**
4.4.1 Tích hợp bốn lõi trong một thiết kế tĩnh . . . 68
4.4.2 Trạng thái Synthesis, Implementation và Bitstream . . . 69
4.4.3 Bố trí phần cứng trên FPGA . . . 70
4.5 Triển khai hệ thống trên Kria KV260 **71**
4.5.1 Môi trường ARM/Linux trên KV260 . . . 71
4.5.2 Chuẩn bị Bitstream và Device Tree Overlay . . . 72
4.5.3 Nạp cấu hình và thay đổi Reconfigurable Module của DPR . . . 72
4.5.4 Nạp Full Bitstream của kiến trúc Replica . . . 74

4.6 Chương trình điều khiển ARM/Linux 74
4.6.1 Truy cập các thanh ghi AXI4-Lite từ Linux . . . 75
4.6.2 Chương trình điều khiển kiến trúc DPR . . . 76
4.6.3 Chương trình điều khiển kiến trúc Replica . . . 77
4.6.4 Đối chiếu quy trình điều khiển . . . 78

CHƯƠNG 5: KIỂM CHỨNG VÀ ĐÁNH GIÁ THIẾT KẾ AES-256 79

5.1 Phương pháp và phạm vi kiểm chứng 79
5.1.1 Tiêu chí đánh giá chức năng . . . 79
5.1.2 Bộ dữ liệu chuẩn và phạm vi Test Vector . . . 79
5.1.3 Nguồn dữ liệu đánh giá phần cứng . . . 80

5.2 Kiểm chứng chức năng bằng mô phỏng RTL 80
5.2.1 Kiểm chứng lõi AES-256 LUT . . . 80
5.2.2 Kiểm chứng lõi AES-256 BRAM . . . 81
5.2.3 Kiểm chứng cơ chế chọn lõi trong Replica . . . 82

5.3 Kiểm chứng chức năng trên phần cứng Kria KV260 84
5.3.1 Phương pháp kiểm chứng trên hệ thống ARM/Linux . . . 84
5.3.2 Kết quả kiểm chứng kiến trúc DPR . . . 85
5.3.3 Kết quả kiểm chứng kiến trúc Replica . . . 86
5.3.4 Tổng hợp và đối chiếu kết quả kiểm chứng . . . 90

5.4 Đánh giá tài nguyên và Timing sau Implementation 91
5.4.1 Mức sử dụng tài nguyên của toàn thiết kế . . . 91
5.4.2 Phân tách tài nguyên lõi mã hóa và phần giao tiếp . . . 92
5.4.3 Kết quả Timing sau Routing . . . 92

5.5 Phân tích và đánh giá tổng hợp 93
5.5.1 Đánh giá tính đúng đắn chức năng . . . 93
5.5.2 Đánh giá sự đánh đổi về tài nguyên và Timing . . . 94
5.5.3 Giới hạn đánh giá và định hướng thực nghiệm tiếp theo . . . 94

CHƯƠNG 6: THỰC NGHIỆM VÀ KIỂM TRA HỆ THỐNG 95

6.1 Thiết lập thực nghiệm 95

6.2 Đánh giá quá trình chuyển đổi giữa các biến thể AES-256 96
6.2.1 Độ ổn định của quá trình tái cấu hình . . . 96
6.2.2 Phân bố lựa chọn RM và các hướng chuyển đổi . . . 97

6.2.3 Thời gian tái cấu hình DPR 98
6.2.4 Đối chiếu với cơ chế lựa chọn trong Replica 99
6.3 Đánh giá trên khối lượng công việc AES-256 **99**
6.3.1 Kết quả thực nghiệm trên kiến trúc DPR 100
6.3.2 Đối chiếu với kiến trúc Replica 100
6.3.3 Ảnh hưởng của tần suất tái cấu hình đến hiệu năng 102
6.4 Đánh giá công suất của các cấu hình **102**
6.4.1 Kết quả ước lượng công suất 102
6.4.2 Phân tích sự khác biệt giữa DPR và Replica 103
6.5 Thảo luận kết quả và giới hạn của thiết kế **104**
6.5.1 Ưu điểm của kiến trúc DPR 104
6.5.2 Hạn chế của cơ chế tái cấu hình 105
6.5.3 Đối chiếu với các nghiên cứu trước 105
6.5.4 Phạm vi đánh giá và hướng phát triển 106
Tài liệu Tham khảo **107**

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt Vấn Đề

Trong các hệ thống xử lý và truyền dữ liệu số, mã hóa là một trong những cơ chế quan trọng để bảo vệ tính bí mật của thông tin. Trong số các thuật toán mã hóa đối xứng hiện nay, Advanced Encryption Standard (AES) được chuẩn hóa bởi Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) theo tiêu chuẩn FIPS 197. AES là thuật toán mã hóa khối với kích thước bản rõ cố định 128 bit, hỗ trợ ba độ dài khóa 128, 192 và 256 bit. Đối với AES-256, quá trình mã hóa sử dụng khóa 256 bit và thực hiện 14 vòng biến đổi trên dữ liệu đầu vào.

AES có thể được hiện thực bằng phần mềm hoặc bằng các khối phần cứng chuyên dụng. Phương pháp phần mềm có ưu điểm về tính linh hoạt, thuận tiện khi phát triển và tích hợp vào hệ thống. Trong khi đó, việc hiện thực trên Field-Programmable Gate Array (FPGA) cho phép tổ chức các phép biến đổi của thuật toán thành những mạch logic hoạt động theo cấu trúc được thiết kế trước. Nhờ khả năng khai thác xử lý song song, pipeline và tài nguyên phần cứng chuyên dụng, FPGA là một nền tảng phù hợp để khảo sát các kiến trúc thực thi AES-256. Tuy nhiên, hiệu quả của một thiết kế không chỉ phụ thuộc vào thuật toán mà còn chịu ảnh hưởng của cách tổ chức đường dữ liệu, bộ điều khiển, tài nguyên sử dụng và tần số hoạt động.

Một trong những thành phần đáng quan tâm khi hiện thực AES trên FPGA là phép biến đổi SubBytes, được thực hiện thông qua bảng thay thế phi tuyến S-box. Chức năng này có thể được xây dựng bằng nhiều phương pháp, chẳng hạn sử dụng bảng tra được ánh xạ vào logic LUT, lưu dữ liệu trong bộ nhớ khối BRAM hoặc mô tả phép biến đổi bằng các phép toán trên trường hữu hạn. Mỗi phương pháp có đặc điểm riêng về cấu trúc logic, đường truyền dữ liệu và yêu cầu tài nguyên. Chẳng hạn, bảng tra tổ hợp có thể cung cấp kết quả mà không cần thêm một chu kỳ đọc bộ nhớ đồng bộ, trong khi việc sử dụng BRAM làm thay đổi cách tổ chức dữ liệu và lịch điều khiển của lõi mã hóa. Vì vậy, cùng một thuật toán AES-256 có thể được triển khai bằng những kiến trúc phần cứng khác nhau, dẫn đến sự khác biệt về tài nguyên và thời gian xử lý.

Đối với một hệ thống FPGA thông thường, kiến trúc phần cứng được xác định sau khi nạp cấu hình cho thiết bị. Khi cần thay đổi một khối chức năng đã được hiện thực, hệ thống có thể phải nạp lại cấu hình phần cứng tương ứng. Công nghệ Dynamic Partial Reconfiguration (DPR), được hỗ trợ trong quy trình Dynamic Function eXchange (DFX) của AMD, cho phép thay đổi một vùng được xác định trước trên FPGA trong khi các thành phần thuộc vùng tĩnh vẫn được giữ nguyên. Với phương pháp này, nhiều Reconfigurable Module (RM) có thể lần lượt sử dụng chung một Reconfigurable Partition (RP), thay vì phải tồn tại đồng thời trên phần cứng.

Khả năng tái sử dụng vùng tài nguyên là một ưu điểm cần được khảo sát của DPR. Tuy nhiên, mỗi lần thay đổi RM đều phát sinh thời gian nạp partial bitstream và các bước điều khiển liên quan. Do đó, việc áp dụng DPR cần được đánh giá đồng thời trên hai khía cạnh: tài nguyên phần cứng và chi phí chuyển đổi cấu hình. Một phương án đối chiếu là kiến trúc Replica, trong đó các biến thể phần cứng được tích hợp đồng thời và hệ thống lựa chọn biến thể cần sử dụng thông qua logic điều khiển. Phương pháp này không yêu cầu nạp lại partial bitstream cho mỗi lần chuyển đổi, nhưng phải dành tài nguyên để hiện thực nhiều biến thể trong cùng một cấu

hình FPGA.

Việc ứng dụng Partial Reconfiguration trong các hệ thống mã hóa đã được nghiên cứu trước đây. Hettwer và cộng sự [11] khai thác sự đa dạng trong hiện thực phần cứng kết hợp với Partial Reconfiguration nhằm tăng cường bảo vệ các mạch mã hóa trên FPGA. Bên cạnh đó, Bow và cộng sự [12] nghiên cứu khả năng chống phân tích công suất thông qua việc sử dụng các kiến trúc hiện thực mã hóa khác nhau. Các công trình này cung cấp cơ sở cho việc khảo sát nhiều biến thể phần cứng trong cùng một hệ thống. Trong đề tài này, nhóm tập trung vào quá trình triển khai trên Kria KV260 và đánh giá sự đánh đổi về tài nguyên, thời gian chuyển đổi và hiệu năng giữa DPR và Replica.

Trên cơ sở đó, nhóm nghiên cứu lựa chọn nền tảng AMD Kria KV260 để xây dựng hệ thống mã hóa AES-256 có khả năng thay đổi giữa nhiều kiến trúc phần cứng. KV260 sử dụng kiến trúc Zynq UltraScale+ MPSoC, kết hợp hệ thống xử lý (Processing System – PS) và vùng logic lập trình được (Programmable Logic – PL) trên cùng một thiết bị. Đây là cơ sở để triển khai phần mềm điều khiển trên bộ xử lý ARM, đồng thời thực hiện thuật toán mã hóa bằng phần cứng trong PL.

Hệ thống được phát triển với bốn biến thể AES-256 gồm LUT, BRAM, Composite và Noise. Các biến thể được xây dựng trên cùng chức năng mã hóa AES-256 nhằm phục vụ việc tích hợp, kiểm chứng và khảo sát hai phương pháp triển khai DPR và Replica. Phần mềm điều khiển được thực thi trên hệ điều hành Linux của KV260, sử dụng giao tiếp AXI4-Lite để trao đổi dữ liệu với phần cứng và quản lý quá trình vận hành. Bên cạnh việc kiểm tra tính đúng của phép mã hóa, đề tài còn thực hiện các thử nghiệm chuyển đổi biến thể, trong đó có cơ chế lựa chọn ngẫu nhiên nhằm đánh giá khả năng vận hành của hệ thống khi cấu hình phần cứng thay đổi liên tục.

Từ những vấn đề trên, đề tài tập trung vào việc thiết kế, hiện thực và kiểm chứng hệ thống AES-256 đa kiến trúc trên FPGA, đồng thời khảo sát sự khác biệt giữa hai phương pháp DPR và Replica trong điều kiện triển khai thực tế. Trọng tâm của nghiên cứu là tổ chức hệ thống phần cứng–phần mềm, khả năng chuyển đổi giữa các biến thể và các chỉ tiêu đánh giá có thể thu thập được trên nền tảng KV260.

1.2 Mục Tiêu Nghiên Cứu

Mục tiêu tổng quát của đề tài là xây dựng và kiểm chứng một hệ thống mã hóa AES-256 trên FPGA có khả năng sử dụng nhiều kiến trúc hiện thực phần cứng, đồng thời triển khai hai phương pháp lựa chọn biến thể là DPR và Replica. Thông qua quá trình thiết kế và thực nghiệm, đề tài hướng đến việc đánh giá tính đúng của hệ thống, khả năng thay đổi kiến trúc khi đang vận hành và các đặc điểm về tài nguyên cũng như thời gian xử lý của từng phương pháp.

Để thực hiện mục tiêu trên, nhóm nghiên cứu xác định các mục tiêu cụ thể như sau:

1. **Thiết kế các biến thể AES-256 trên FPGA.** Nghiên cứu và hiện thực các kiến trúc LUT, BRAM, Composite và Noise bằng ngôn ngữ mô tả phần cứng, bảo đảm các biến thể thực hiện cùng chức năng mã hóa và có giao diện phù hợp cho quá trình tích hợp.

2. **Xây dựng hệ thống tích hợp PS–PL.** Thiết kế giao tiếp AXI4-Lite để bộ xử lý ARM có thể cung cấp khóa, bản rõ, phát lệnh điều khiển và đọc kết quả mã hóa. Xây dựng phần mềm C trên Linux để thực hiện các thao tác trao đổi dữ liệu và quản lý hoạt động của hệ thống.

3. **Hiện thực cơ chế DPR và Replica.** Tổ chức vùng tĩnh và vùng tái cấu hình trong thiết kế DPR, xây dựng các cấu hình phần cứng tương ứng với từng biến thể AES-256. Đồng thời, triển khai phương án Replica để lựa chọn giữa các biến thể đã được tích hợp trong cùng hệ thống.

4. **Kiểm chứng chức năng và đánh giá thiết kế.** Kiểm tra hoạt động của các lõi AES-256 bằng mô phỏng RTL và thực nghiệm trên FPGA. Thu thập các thông tin phù hợp về kết quả mã hóa, tài nguyên phần cứng, timing và thời gian thực thi để phục vụ việc phân tích và so sánh.

5. **Xây dựng và kiểm tra cơ chế chuyển đổi biến thể ngẫu nhiên.** Phát triển chương trình điều khiển cho phép lựa chọn biến thể AES-256 theo thứ tự không cố định, thực hiện chuyển đổi cấu hình hoặc lựa chọn lõi tương ứng và kiểm tra kết quả mã hóa sau mỗi lần chuyển đổi. Các thử nghiệm được sử dụng để đánh giá tính đúng và độ ổn định của quy trình vận hành.

Các mục tiêu trên được triển khai theo hướng kết hợp thiết kế phần cứng, phát triển phần mềm và kiểm chứng thực nghiệm. Trong phạm vi báo cáo này, những nội dung đã hoàn thành được trình bày cùng kết quả tương ứng; các chỉ tiêu chưa có đủ dữ liệu đánh giá được phân biệt với những kết quả đã xác nhận.

1.3 Phạm vi và nội dung thực hiện

Đề tài tập trung vào quá trình mã hóa AES-256 với khóa đầu vào 256 bit và khối dữ liệu 128 bit. Nội dung chính là thiết kế các lõi mã hóa, xây dựng hệ thống điều khiển và tích hợp trên FPGA. Các phép biến đổi của thuật toán được thực hiện ở mức phần cứng, trong khi việc cung cấp dữ liệu, phát lệnh và kiểm tra kết quả được thực hiện từ phần mềm chạy trên bộ xử lý ARM.

Về nền tảng triển khai, hệ thống sử dụng AMD Kria KV260 với môi trường phát triển phần cứng Vivado và hệ điều hành Ubuntu 22.04. Thiết kế giao tiếp AXI4-Lite cung cấp các thanh ghi để trao đổi khóa mã hóa, bản rõ, bản mã và tín hiệu điều khiển giữa PS và PL. Phần mềm C trên Linux sử dụng cơ chế truy cập thanh ghi ánh xạ bộ nhớ để giao tiếp với lõi AES-256. Đối với DPR, phần mềm còn phối hợp với cơ chế quản lý cấu hình FPGA của Linux để nạp các partial bitstream tương ứng với từng RM.

Về kiến trúc mã hóa, bốn biến thể LUT, BRAM, Composite và Noise được sử dụng làm đối tượng nghiên cứu. Các biến thể có cùng chức năng AES-256 nhưng khác nhau về cách hiện thực phần cứng. Việc duy trì giao diện chức năng thống nhất giúp các biến thể có thể được tích hợp vào cùng một hệ thống điều khiển và tạo điều kiện đối chiếu kết quả. Những đặc điểm cụ thể

của từng kiến trúc được trình bày ở Chương 2.

Về phương pháp triển khai, đề tài khảo sát hai hướng. Với DPR, các biến thể được đưa vào cùng một vùng tái cấu hình thông qua các partial bitstream tương ứng. Với Replica, các biến thể tồn tại đồng thời và được lựa chọn bằng cơ chế điều khiển. Việc so sánh hai phương pháp tập trung vào tổ chức phần cứng, mức sử dụng tài nguyên, thời gian chuyển đổi và kết quả xử lý trong những điều kiện thí nghiệm có thể đối chiếu.

Về kiểm chứng, nhóm đã triển khai các bước kiểm tra từ mức thiết kế RTL đến thực nghiệm trên KV260. Các kết quả thực nghiệm đã ghi nhận hoạt động của giao tiếp AXI4-Lite, khả năng mã hóa AES-256 và quá trình chuyển đổi giữa các RM. Bên cạnh đó, chương trình lựa chọn ngẫu nhiên đã được sử dụng trong các phép thử chuyển đổi liên tục giữa bốn biến thể. Những kết quả này là cơ sở để phân tích tính đúng và khả năng vận hành của hệ thống; các số liệu đo thời gian và tài nguyên sẽ được trình bày trong những chương đánh giá tương ứng.

Phạm vi nghiên cứu hiện tại chưa mở rộng sang thiết kế một hệ thống mã hóa và giải mã hoàn chỉnh với nhiều chế độ hoạt động như CBC hoặc CTR, cũng chưa tập trung vào truyền dữ liệu tốc độ cao bằng DMA. Ngoài ra, cơ chế lựa chọn biến thể ngẫu nhiên trong đề tài được sử dụng để khảo sát khả năng chuyển đổi và độ ổn định của hệ thống. Báo cáo không xem đây là bằng chứng về khả năng chống tấn công kênh kề nếu chưa có các phép đo và phân tích chuyên biệt.

Bảng 1 tổng hợp các thành phần chính và phạm vi khảo sát của đề tài.

Bảng 1: Các thành phần chính và phạm vi nghiên cứu

Thành phần	**Nội dung nghiên cứu**
Thuật toán	Mã hóa AES-256; khóa 256 bit, khối dữ liệu 128 bit, 14 vòng mã hóa.
Nền tảng	AMD Kria KV260, kết hợp PS và PL.
Biến thể phần cứng	LUT, BRAM, Composite và Noise.
Giao tiếp hệ thống	AXI4-Lite; trao đổi dữ liệu và tín hiệu điều khiển giữa ARM và lõi AES-256.
Phương pháp triển khai	Dynamic Partial Reconfiguration và Replica.
Phần mềm	Chương trình C trên Ubuntu 22.04, truy cập thanh ghi và điều khiển chuyển đổi biến thể.
Kiểm chứng	Mô phỏng RTL, kiểm tra chức năng trên FPGA, thử nghiệm chuyển đổi biến thể và đánh giá các chỉ tiêu phần cứng phù hợp.

Kết quả kiểm chứng và thực nghiệm được trình bày ở Chương 5 và Chương 6. Những nội dung chưa được đánh giá trực tiếp, đặc biệt là khả năng chống tấn công kênh kề, không thuộc phạm vi kết luận của báo cáo.

1.4 Cấu trúc báo cáo

Báo cáo gồm sáu chương. Chương 1 giới thiệu vấn đề, mục tiêu và phạm vi nghiên cứu. Chương 2 trình bày cơ sở lý thuyết về AES-256, các phương pháp hiện thực S-box và cơ chế DPR.

Chương 3 mô tả thiết kế lõi AES-256, bốn biến thể phần cứng và giao tiếp PS–PL. Chương 4 trình bày quá trình hiện thực trong Vivado, triển khai trên Kria KV260 và chương trình điều khiển ARM/Linux.

Chương 5 tập trung vào kiểm chứng bằng mô phỏng RTL, bộ dữ liệu NIST và đánh giá tài nguyên, Timing sau Implementation. Chương 6 trình bày thực nghiệm chuyển đổi biến thể, xử lý workload AES-256, công suất ước lượng và so sánh DPR–Replica.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Tổng quan về thuật toán mã hóa AES-256

2.1.1 Nguyên lý mã hóa khối và cấu trúc State

Advanced Encryption Standard (AES) là thuật toán mã hóa khối sử dụng khóa đối xứng, được Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) chuẩn hóa trong tài liệu FIPS 197. Thuật toán được phát triển từ Rijndael, do Joan Daemen và Vincent Rijmen đề xuất. Theo đặc tả FIPS 197, AES hỗ trợ ba độ dài khóa là 128, 192 và 256 bit, trong khi kích thước khối dữ liệu được giữ cố định ở 128 bit [1].

Trong mô hình mã hóa đối xứng, cùng một khóa bí mật được sử dụng để xác định phép biến đổi mã hóa và phép biến đổi giải mã tương ứng. Với AES, quá trình mã hóa biến đổi một khối bản rõ (*plaintext*) thành một khối bản mã (*ciphertext*) có cùng độ dài. Có thể biểu diễn phép mã hóa dưới dạng:

$$C = E_K(P), \tag{1}$$

trong đó P là bản rõ 128 bit, C là bản mã 128 bit, K là khóa mã hóa và E_K là phép biến đổi AES ứng với khóa K. Đối với mỗi khóa xác định, AES thực hiện một phép hoán vị trên tập hợp các khối 128 bit, do đó có thể khôi phục bản rõ bằng phép biến đổi ngược tương ứng.

AES thuộc nhóm thuật toán mã hóa dựa trên mạng thay thế–hoán vị (*Substitution–Permutation Network*, SPN). Thay vì xử lý trực tiếp toàn bộ khối dữ liệu bằng một phép biến đổi duy nhất, AES thực hiện nhiều vòng biến đổi liên tiếp. Trong mỗi vòng, dữ liệu trung gian được thay thế, hoán vị và kết hợp với khóa vòng. Sự phối hợp giữa các phép biến đổi phi tuyến và tuyến tính giúp tạo ra mối quan hệ phức tạp giữa bản rõ, khóa và bản mã.

Dữ liệu được xử lý bên trong AES dưới dạng một ma trận hai chiều gọi là *State*. Mỗi State gồm 16 byte, được sắp xếp thành bốn hàng và bốn cột. Mỗi phần tử của ma trận có độ rộng 8 bit, vì vậy tổng kích thước State là 128 bit.

Gọi $p_0, p_1, \ldots, p_{15}$ là các byte của khối bản rõ theo thứ tự đầu vào. Theo quy ước của FIPS 197, các byte được đưa vào State theo từng cột, từ trên xuống dưới và từ trái sang phải:

$$S = \begin{bmatrix} p_0 & p_4 & p_8 & p_{12} \\ p_1 & p_5 & p_9 & p_{13} \\ p_2 & p_6 & p_{10} & p_{14} \\ p_3 & p_7 & p_{11} & p_{15} \end{bmatrix}. \tag{2}$$

Với chỉ số hàng r và cột c bắt đầu từ 0, quan hệ giữa mảng byte đầu vào và State được xác định bởi:

$$s_{r,c} = p_{4c+r}, \qquad 0 \le r, c < 4. \tag{3}$$

Cách sắp xếp này cần được phân biệt với việc điền dữ liệu lần lượt theo từng hàng. Chẳng hạn, với khối bản rõ:

$$P = \texttt{00112233445566778899AABBCCDDEEFF},$$

State ban đầu có dạng:

$$S = \begin{bmatrix} 00 & 44 & 88 & CC \\ 11 & 55 & 99 & DD \\ 22 & 66 & AA & EE \\ 33 & 77 & BB & FF \end{bmatrix}. \tag{4}$$

Sau khi hoàn thành các vòng mã hóa, State cuối cùng được chuyển trở lại thành một dãy 16 byte theo cùng quy ước cột để tạo thành bản mã đầu ra.

Việc tổ chức dữ liệu dưới dạng State có ý nghĩa đối với hiện thực phần cứng. Phép SubBytes tác động độc lập lên từng byte, ShiftRows thay đổi vị trí các byte trong cùng một hàng, còn MixColumns thực hiện phép biến đổi trên từng cột. Do đó, cấu trúc State không chỉ là một quy ước biểu diễn mà còn làm rõ cách tổ chức đường dữ liệu khi thiết kế lõi AES trên FPGA.

Hệ thống được thiết kế để mã hóa từng khối dữ liệu 128 bit bằng AES-256. Các chế độ xử lý thông điệp nhiều khối như CBC hoặc CTR nằm ngoài phạm vi hiện thực của đề tài.

2.1.2 Thông số của các phiên bản AES

Theo FIPS 197, ba phiên bản được chuẩn hóa là AES-128, AES-192 và AES-256. Các phiên bản này cùng sử dụng khối dữ liệu 128 bit và cùng hệ thống phép biến đổi cơ bản. Sự khác biệt chủ yếu nằm ở độ dài khóa, số vòng mã hóa và quy tắc mở rộng khóa.

Ba tham số thường được sử dụng để mô tả cấu hình của thuật toán gồm:

- N_b: số word 32 bit trong một khối dữ liệu, luôn bằng 4 đối với AES.
- N_k: số word 32 bit trong khóa mã hóa ban đầu.
- N_r: số vòng mã hóa được thực hiện trên mỗi khối dữ liệu.

Mối quan hệ giữa các tham số của ba phiên bản được trình bày trong Bảng 2.

Bảng 2: Thông số của các phiên bản AES theo FIPS 197.

Phiên bản	**Khóa (bit)**	N_k	N_b	N_r
AES-128	128	4	4	10
AES-192	192	6	4	12
AES-256	256	8	4	14

Đối với AES-256, khóa đầu vào có độ dài 256 bit, tương ứng với tám word 32 bit ($N_k = 8$). Mỗi khối dữ liệu vẫn chứa bốn word 32 bit ($N_b = 4$), nhưng thuật toán cần thực hiện 14 vòng mã hóa ($N_r = 14$).

Ngoài khóa ban đầu, quá trình mã hóa còn sử dụng các khóa vòng (*round key*) được sinh ra bằng thuật toán mở rộng khóa (*Key Expansion*). Mỗi khóa vòng có độ dài 128 bit và được sử dụng trong phép AddRoundKey tương ứng.

Với N_r vòng mã hóa, thuật toán cần N_r+1 khóa vòng, bao gồm một khóa cho bước AddRoundKey

ban đầu và một khóa cho mỗi vòng tiếp theo. Vì vậy, AES-256 sử dụng 15 khóa vòng, tương ứng với:

$$N_{\text{word}} = N_b(N_r + 1) = 4(14 + 1) = 60.$$

Như vậy, từ khóa gốc gồm 8 word, Key Expansion tạo ra lịch khóa gồm tổng cộng 60 word 32 bit. Toàn bộ lịch khóa tương đương 1920 bit dữ liệu khóa vòng. Cần phân biệt độ dài của lịch khóa với độ dài khóa bí mật ban đầu: việc mở rộng khóa không làm tăng số bit độc lập của khóa gốc.

Số vòng mã hóa cũng không đồng nghĩa trực tiếp với số chu kỳ xung nhịp của một thiết kế phần cứng. Một lõi AES có thể tái sử dụng cùng một datapath qua nhiều chu kỳ, thực hiện một vòng trên mỗi chu kỳ hoặc tổ chức nhiều tầng xử lý song song. Vì vậy, độ trễ và thông lượng của lõi AES phụ thuộc vào kiến trúc hiện thực, bên cạnh các tham số cố định của thuật toán.

Trong đề tài này, AES-256 được lựa chọn làm thuật toán cơ sở chung cho bốn biến thể phần cứng LUT, BRAM, Composite và Noise. Các biến thể phải tạo ra cùng một kết quả mã hóa đối với cùng khóa và bản rõ. Sự khác biệt giữa chúng nằm ở tổ chức logic và tài nguyên phần cứng, không nằm ở việc thay đổi số vòng hoặc các phép biến đổi được chuẩn hóa.

2.1.3 Quy trình mã hóa AES-256

Quá trình mã hóa AES-256 được thực hiện qua một bước biến đổi ban đầu, 13 vòng mã hóa thông thường và một vòng mã hóa cuối cùng. Các phép biến đổi sử dụng dữ liệu trung gian trong State và các khóa vòng sinh ra từ Key Expansion.

Bốn phép biến đổi cơ bản của AES gồm:

1. **SubBytes:** Thay thế từng byte của State bằng một giá trị tương ứng trong bảng thay thế phi tuyến S-box. Phép biến đổi này được thực hiện độc lập trên 16 byte của State.
2. **ShiftRows:** Dịch vòng các byte theo từng hàng của State. Hàng thứ nhất được giữ nguyên, trong khi ba hàng còn lại lần lượt được dịch vòng sang trái một, hai và ba vị trí.
3. **MixColumns:** Biến đổi các byte trong từng cột của State thông qua phép nhân ma trận trên trường hữu hạn $GF(2^8)$. Phép biến đổi này kết hợp thông tin giữa bốn byte thuộc cùng một cột.
4. **AddRoundKey:** Thực hiện phép XOR từng bit giữa State và khóa vòng 128 bit tương ứng. Đây là bước đưa thông tin của khóa vào quá trình biến đổi dữ liệu.

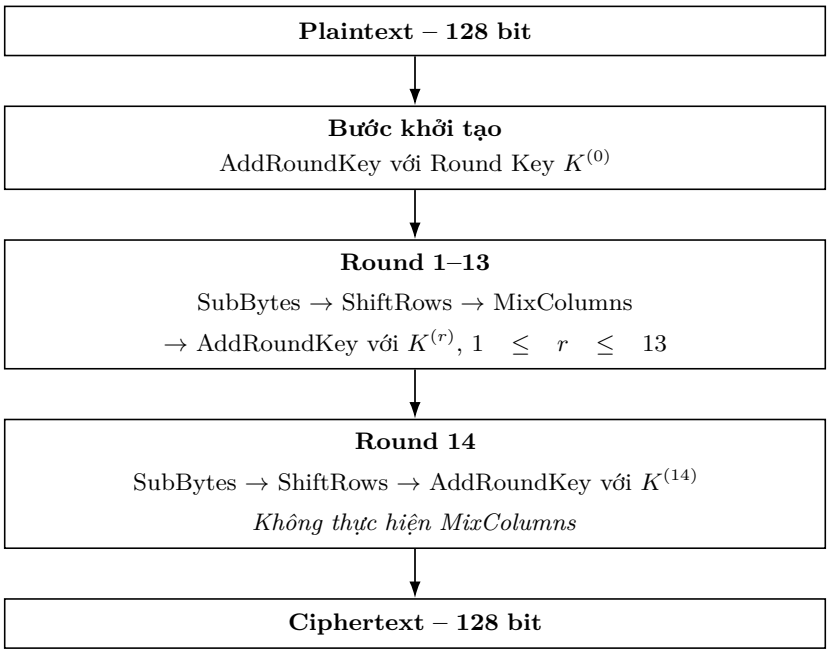
Các phép biến đổi trên được thực hiện theo thứ tự xác định. Tuy nhiên, không phải mọi vòng mã hóa đều bao gồm đầy đủ cả bốn phép. Quy trình AES-256 có thể chia thành ba giai đoạn chính như sau.

Giai đoạn khởi tạo. Khối bản rõ 128 bit được ánh xạ vào State. Trước khi thực hiện vòng mã hóa đầu tiên, State được XOR với khóa vòng $K^{(0)}$ thông qua phép AddRoundKey. Đây là bước cộng khóa ban đầu, không phải một vòng mã hóa độc lập trong số 14 vòng AES-256.

Các vòng mã hóa thông thường. Từ vòng thứ 1 đến vòng thứ 13, State lần lượt trải qua

bốn phép biến đổi: SubBytes, ShiftRows, MixColumns và AddRoundKey. Kết quả của vòng trước trở thành dữ liệu đầu vào cho vòng tiếp theo. Mỗi vòng sử dụng một khóa vòng khác nhau được xác định từ lịch khóa AES-256.

Vòng mã hóa cuối cùng. Ở vòng thứ 14, thuật toán chỉ thực hiện SubBytes, ShiftRows và AddRoundKey. Phép MixColumns được loại bỏ theo đặc tả AES. State sau AddRoundKey của vòng cuối được chuyển thành bản mã 128 bit. Quy trình tổng thể được minh họa trong Hình 1.



Hình 1: Quy trình mã hóa một khối dữ liệu bằng AES-256.

Để biểu diễn các giai đoạn trên dưới dạng toán học, gọi P_s là State được tạo từ bản rõ, $S^{(r)}$ là State sau vòng thứ r và $K^{(r)}$ là khóa vòng tương ứng. Khi đó, bước khởi tạo được xác định bởi:

$$S^{(0)} = P_s \oplus K^{(0)}. \tag{5}$$

Với các vòng từ 1 đến 13, quá trình biến đổi có dạng:

$$S^{(r)} = \mathrm{ARK}_{K^{(r)}}\Big(\mathrm{MC}\big(\mathrm{SR}(\mathrm{SB}(S^{(r-1)}))\big)\Big), 1 \leq r \leq 13. \tag{6}$$

Trong đó, SB, SR và MC lần lượt biểu diễn SubBytes, ShiftRows và MixColumns; toán tử $\mathrm{ARK}_{K^{(r)}}$ thực hiện XOR State với khóa vòng $K^{(r)}$.

Đối với vòng cuối cùng, biểu thức được rút gọn thành:

$$S^{(14)} = \mathrm{ARK}_{K^{(14)}}\Big(\mathrm{SR}\big(\mathrm{SB}(S^{(13)})\big)\Big). \tag{7}$$

Ciphertext C được tạo thành bằng cách chuyển các byte của $S^{(14)}$ về thứ tự đầu ra theo FIPS 197.

Các biểu thức trên mô tả quan hệ giữa những bước biến đổi của thuật toán, không quy định cách tổ chức thời gian thực hiện trên phần cứng. Chẳng hạn, một phép SubBytes có thể được hiện thực bằng bảng tra tổ hợp hoặc bộ nhớ đồng bộ; hai phương án có thể tạo ra cùng giá trị đầu ra nhưng khác nhau về độ trễ, số chu kỳ xử lý và mức sử dụng tài nguyên FPGA. Tương tự, thuật toán Key Expansion xác định giá trị của các khóa vòng nhưng không bắt buộc phần cứng phải lưu sẵn toàn bộ lịch khóa. Cách tạo và cung cấp khóa vòng phụ thuộc vào kiến trúc được lựa chọn. Những đặc điểm này tạo cơ sở để nghiên cứu các phương án hiện thực AES-256 có cùng chức năng mã hóa nhưng khác nhau về tổ chức phần cứng. Trong các mục tiếp theo, các phép biến đổi của AES sẽ được phân tích chi tiết hơn, đặc biệt là cấu trúc S-box và Key Expansion, trước khi xem xét các phương pháp hiện thực tương ứng trên FPGA.

2.2 Các phép biến đổi trong thuật toán AES

Mục này trình bày cơ sở toán học và đặc điểm hiện thực phần cứng của bốn phép biến đổi AES: SubBytes, ShiftRows, MixColumns và AddRoundKey. Các biểu thức được xây dựng theo FIPS 197 và làm cơ sở cho thiết kế Datapath ở Chương 3.

2.2.1 Phép biến đổi SubBytes

SubBytes là phép biến đổi phi tuyến được thực hiện độc lập trên từng byte của State. Với mỗi byte đầu vào, thuật toán sử dụng một bảng thay thế cố định gọi là S-box (*Substitution Box*) để xác định giá trị đầu ra tương ứng.

Nếu $s_{r,c}$ là một byte của State trước SubBytes thì giá trị sau biến đổi được xác định bởi:

$$s'_{r,c} = \mathrm{SBox}(s_{r,c}), \qquad 0 \le r, c < 4. \tag{8}$$

Do State gồm 16 byte, mỗi lần thực hiện SubBytes trên toàn bộ State cần xử lý 16 phép thay thế byte. Các phép thay thế không phụ thuộc trực tiếp vào nhau nên có thể được tổ chức xử lý song song trong phần cứng.

Theo FIPS 197, S-box của AES là một ánh xạ song ánh từ tập hợp 256 giá trị byte sang chính tập hợp đó. Bảng S-box được xây dựng từ hai bước toán học: tìm Multiplicative Inverse trong trường hữu hạn $GF(2^8)$ và thực hiện Affine Transformation.

Cơ sở toán học của S-box

Mỗi byte trong AES có thể được xem là một phần tử của trường hữu hạn $GF(2^8)$. Với byte gồm tám bit $b_7, b_6, \ldots, b_0$, biểu diễn đa thức tương ứng là:

$$b(x) = b_7x^7 + b_6x^6 + b_5x^5 + b_4x^4 + b_3x^3 + b_2x^2 + b_1x + b_0, \tag{9}$$

trong đó mỗi hệ số b_i thuộc tập $\{0, 1\}$. Các phép cộng hệ số được thực hiện theo modulo 2, tương đương với phép XOR.

Phép nhân trong $GF(2^8)$ được xác định bằng cách nhân hai đa thức và lấy phần dư theo đa thức bất khả quy:

$$m(x) = x^8 + x^4 + x^3 + x + 1. \tag{10}$$

Đa thức này có biểu diễn Hex là `0x11B`. Việc lấy phần dư theo $m(x)$ bảo đảm kết quả của phép nhân vẫn biểu diễn được bằng 8 bit.

Đối với một phần tử khác không $b \in GF(2^8)$, Multiplicative Inverse là phần tử b^{-1} thỏa mãn:

$$b \cdot b^{-1} = 1 \qquad \text{trong } GF(2^8). \tag{11}$$

Trong bước đầu tiên của S-box, byte đầu vào b được chuyển thành giá trị trung gian $\widetilde{b}$ theo quy tắc:

$$\widetilde{b} = \begin{cases} 0, & b = 0, \\ b^{-1}, & b \neq 0. \end{cases} \tag{12}$$

Trường hợp $b = 0$ được xử lý riêng vì phần tử 0 không có Multiplicative Inverse. Theo quy định của AES, giá trị trung gian tương ứng được chọn bằng 0.

Affine Transformation

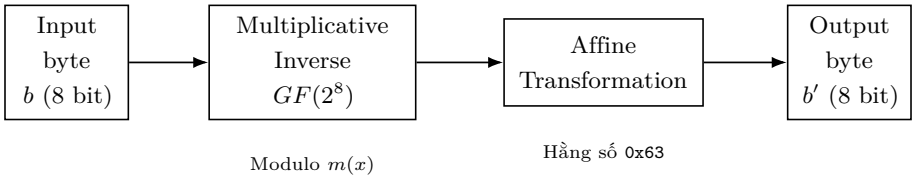
Sau khi xác định $\widetilde{b}$, thuật toán tiếp tục thực hiện Affine Transformation để tạo byte đầu ra b'. Phép biến đổi này kết hợp các bit của $\widetilde{b}$ bằng XOR với một hằng số cố định.

Gọi $\widetilde{b}_i$ là bit thứ i của giá trị trung gian, với bit thấp nhất mang chỉ số 0. Bit đầu ra b'_i được xác định bởi:

$$b'_i = \widetilde{b}_i \oplus \widetilde{b}_{(i+4) \bmod 8} \oplus \widetilde{b}_{(i+5) \bmod 8} \oplus \widetilde{b}_{(i+6) \bmod 8} \oplus \widetilde{b}_{(i+7) \bmod 8} \oplus c_i, \tag{13}$$

với $i = 0, 1, \ldots, 7$ và c_i là bit thứ i của hằng số $c =$ `0x63`.

Như vậy, đầu ra của S-box không đơn thuần là Multiplicative Inverse mà còn phải trải qua Affine Transformation. Hai bước này kết hợp tạo thành phép thay thế phi tuyến được quy định trong AES. Cấu trúc toán học của S-box được minh họa trong Hình 2.



Hình 2: Nguyên lý xây dựng AES S-box từ Multiplicative Inverse và Affine Transformation.

Trong trường hợp đầu vào bằng `0x00`, giá trị trung gian sau bước nghịch đảo bằng 0 và kết quả Affine Transformation là `0x63`. Đây cũng là phần tử đầu tiên của bảng S-box.

Hai ví dụ cho thấy mỗi byte đầu vào được ánh xạ đến một giá trị xác định, không phụ thuộc vào vị trí của byte trong State hay giá trị của các byte khác.

Ý nghĩa đối với thiết kế phần cứng

Về nguyên lý, S-box có thể được hiện thực bằng nhiều cách khác nhau. Một phương án là lưu sẵn 256 giá trị đầu ra và sử dụng byte đầu vào làm địa chỉ tra cứu. Phương án khác là xây dựng mạch logic thực hiện trực tiếp Multiplicative Inverse và Affine Transformation.

Đối với FPGA, bảng thay thế có thể được hiện thực bằng logic tổ hợp hoặc lưu trong tài nguyên bộ nhớ. Trong khi đó, hướng tính toán trực tiếp có thể khai thác phép biến đổi trường hữu hạn để xây dựng mạng logic tương ứng.

Các phương án này có thể khác nhau về tài nguyên sử dụng, độ trễ tổ hợp và cách tổ chức dữ liệu theo chu kỳ xung nhịp, nhưng phải bảo đảm cùng ánh xạ S-box theo FIPS 197.

2.2.2 Phép biến đổi ShiftRows

ShiftRows thực hiện phép hoán vị vị trí các byte theo từng hàng của State. Khác với SubBytes, phép biến đổi này không thay đổi giá trị của từng byte mà chỉ thay đổi vị trí của chúng trong ma trận.

Theo FIPS 197, mỗi hàng của State được dịch vòng sang trái một số vị trí xác định:

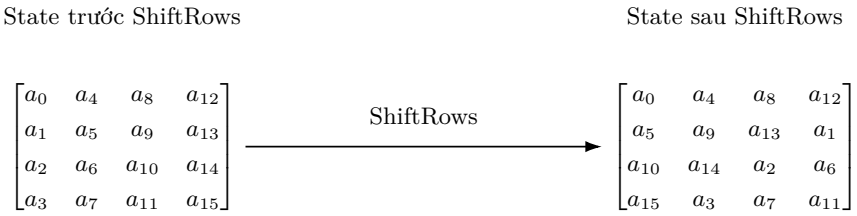
- Hàng $r = 0$ được giữ nguyên vị trí.
- Hàng $r = 1$ được dịch vòng sang trái 1 byte.
- Hàng $r = 2$ được dịch vòng sang trái 2 byte.
- Hàng $r = 3$ được dịch vòng sang trái 3 byte.

Với $s_{r,c}$ là phần tử ở hàng r, cột c trước biến đổi, giá trị đầu ra được xác định bởi:

$$s'_{r,c} = s_{r,(c+r) \bmod 4}, \qquad 0 \le r, c < 4. \tag{14}$$

Phép lấy modulo 4 bảo đảm các byte dịch vượt quá cột cuối sẽ quay trở lại đầu hàng. Do đó, ShiftRows là một phép dịch vòng và không làm mất bất kỳ byte dữ liệu nào.

Để minh họa, gọi $a_0, a_1, \ldots, a_{15}$ là 16 byte của State tại thời điểm trước ShiftRows, được đánh chỉ số theo thứ tự cột. Kết quả biến đổi được trình bày trong Hình 3.



Hình 3: Nguyên lý hoán vị byte của phép biến đổi ShiftRows.

Có thể thấy sau ShiftRows, các byte vốn nằm trên cùng một cột được phân bố lại giữa các cột khác nhau. Sự thay đổi này đặc biệt quan trọng khi kết hợp với MixColumns ở bước kế tiếp.

Nếu chỉ thực hiện MixColumns mà không có ShiftRows, quá trình kết hợp dữ liệu sẽ chủ yếu diễn ra bên trong từng cột. ShiftRows giúp đưa các byte từ những cột khác nhau vào cùng một

cột trước khi MixColumns được thực hiện ở các Round tiếp theo.

Về mặt phần cứng, ShiftRows có thể được hiện thực bằng cách sắp xếp lại kết nối giữa các byte của State. Nếu vị trí dữ liệu được quy định cố định, phép biến đổi này không nhất thiết cần một mạng logic tính toán phức tạp. Tuy nhiên, cách tổ chức thanh ghi và Multiplexer cụ thể vẫn phụ thuộc vào kiến trúc Datapath được sử dụng.

2.2.3 Phép biến đổi MixColumns

MixColumns là phép biến đổi tuyến tính được thực hiện độc lập trên bốn cột của State. Đối với mỗi cột, bốn byte đầu vào được kết hợp thông qua phép nhân ma trận trên trường hữu hạn $GF(2^8)$.

Mục đích của MixColumns là tạo ra sự phụ thuộc giữa các byte trong cùng một cột, sao cho một byte đầu ra được xác định từ cả bốn byte đầu vào.

Theo FIPS 197, phép biến đổi được xác định bởi ma trận hệ số cố định:

$$\begin{bmatrix} s'_{0,c} \\ s'_{1,c} \\ s'_{2,c} \\ s'_{3,c} \end{bmatrix} = \begin{bmatrix} \texttt{02} & \texttt{03} & \texttt{01} & \texttt{01} \\ \texttt{01} & \texttt{02} & \texttt{03} & \texttt{01} \\ \texttt{01} & \texttt{01} & \texttt{02} & \texttt{03} \\ \texttt{03} & \texttt{01} & \texttt{01} & \texttt{02} \end{bmatrix} \begin{bmatrix} s_{0,c} \\ s_{1,c} \\ s_{2,c} \\ s_{3,c} \end{bmatrix}, \tag{15}$$

với $c = 0, 1, 2, 3$. Trong biểu thức trên, các hệ số được viết ở dạng Hex và phép nhân ma trận được hiểu là phép toán trên $GF(2^8)$, không phải phép nhân số nguyên thông thường.

Đối với byte đầu ra đầu tiên, biểu thức được khai triển thành:

$$s'_{0,c} = (\texttt{02} \cdot s_{0,c}) \oplus (\texttt{03} \cdot s_{1,c}) \oplus s_{2,c} \oplus s_{3,c}. \tag{16}$$

Các byte đầu ra còn lại được tính tương tự theo các hàng tương ứng của ma trận hệ số.

Điểm cần chú ý là MixColumns chỉ sử dụng các hệ số `01`, `02` và `03`. Nhờ đó, các phép nhân trong $GF(2^8)$ có thể được rút gọn thành những phép dịch bit và XOR, phù hợp với việc hiện thực bằng mạch số.

Phép nhân trong $GF(2^8)$

Như đã giới thiệu ở phần SubBytes, các byte của AES được biểu diễn bằng đa thức trên $GF(2)$ và phép nhân được thực hiện modulo đa thức $m(x)$.

Đối với các hệ số của MixColumns, phép nhân với `01` không làm thay đổi giá trị byte

Phép nhân với `02` tương đương với nhân đa thức đại diện của byte a với x, sau đó rút gọn modulo $m(x)$. Phép toán này thường được ký hiệu là xtime(a).

Với a_7 là bit cao nhất của byte a, ta có:

$$\text{xtime}(a) = \begin{cases} (a \ll 1)_8, & a_7 = 0, \\ (a \ll 1)_8 \oplus \texttt{0x1B}, & a_7 = 1, \end{cases} \tag{17}$$

trong đó $(a \ll 1)_8$ là kết quả dịch trái một bit và chỉ giữ lại 8 bit thấp.

Hằng số `0x1B` xuất hiện do quá trình rút gọn theo đa thức bất khả quy $m(x)$. Cụ thể, trong $GF(2^8)$ ta có:

$$x^8 \equiv x^4 + x^3 + x + 1 \pmod{m(x)}. \tag{18}$$

Vì vậy, khi phép dịch trái tạo ra thành phần x^8, thành phần đó được thay thế bằng đa thức tương ứng với `0x1B`.

Phép nhân với `03` được xác định bởi:

$$\texttt{03} \cdot a = \text{xtime}(a) \oplus a. \tag{19}$$

Từ các quan hệ trên, MixColumns có thể được thực hiện bằng mạng XOR và logic xử lý bit, không cần sử dụng bộ nhân số học thông thường.

Ví dụ tính toán MixColumns

Xét một cột State có giá trị đầu vào $[\texttt{D4}, \texttt{BF}, \texttt{5D}, \texttt{30}]^T$. Áp dụng phép biến đổi MixColumns, byte đầu ra đầu tiên được tính như sau:

$$s'_0 = (\texttt{02} \cdot \texttt{D4}) \oplus (\texttt{03} \cdot \texttt{BF}) \oplus \texttt{5D} \oplus \texttt{30} = \texttt{B3} \oplus \texttt{DA} \oplus \texttt{5D} \oplus \texttt{30} = \texttt{04}.$$

Thực hiện tương tự đối với ba byte còn lại, kết quả MixColumns của cột State là:

$$\begin{bmatrix} \texttt{D4} \\ \texttt{BF} \\ \texttt{5D} \\ \texttt{30} \end{bmatrix} \xrightarrow{\text{MixColumns}} \begin{bmatrix} \texttt{04} \\ \texttt{66} \\ \texttt{81} \\ \texttt{E5} \end{bmatrix}.$$

Ý nghĩa đối với thiết kế phần cứng

MixColumns có tính chất tuyến tính trên $GF(2)$ và sử dụng các hệ số cố định. Do đó, logic của phép biến đổi có thể được xây dựng từ các mạng XOR kết hợp với phép nhân xtime.

Bốn cột State được xử lý độc lập trong MixColumns. Vì vậy, tùy kiến trúc, phần cứng có thể triển khai nhiều khối xử lý song song hoặc tái sử dụng một khối tính toán qua nhiều chu kỳ.

Trong AES, ShiftRows và MixColumns bổ sung cho nhau. ShiftRows thay đổi vị trí byte giữa các cột, còn MixColumns kết hợp dữ liệu bên trong từng cột. Qua nhiều Round, sự phối hợp này giúp lan truyền ảnh hưởng của các byte đầu vào trên State.

Theo đặc tả AES-256, MixColumns được thực hiện trong các Round từ 1 đến 13, nhưng không xuất hiện ở Round cuối cùng. Đây là điều kiện cần được bảo đảm trong logic điều khiển của lõi mã hóa.

2.2.4 Phép biến đổi AddRoundKey

AddRoundKey kết hợp State với Round Key bằng phép XOR theo từng bit. Đây là phép biến đổi trực tiếp đưa thông tin của Encryption Key vào quá trình mã hóa.

Gọi S là State đầu vào và $K^{(r)}$ là Round Key được sử dụng tại Round thứ r. Khi đó:

$$S' = S \oplus K^{(r)}. \tag{20}$$

Vì State và Round Key đều có độ dài 128 bit, phép XOR được thực hiện trên 128 cặp bit tương ứng. Nếu biểu diễn State và Round Key dưới dạng ma trận 4×4 byte, phép biến đổi được viết:

$$s'_{i,j} = s_{i,j} \oplus k^{(r)}_{i,j}, \qquad 0 \leq i, j < 4. \tag{21}$$

Trong đó $k^{(r)}_{i,j}$ là byte tương ứng của Round Key thứ r. Cách sắp xếp các byte trong Round Key tuân theo cùng quy ước cột của State.

Đối với AES-256, AddRoundKey xuất hiện 15 lần: một lần trước Round đầu tiên và một lần trong mỗi Round từ 1 đến 14.

Mỗi lần thực hiện sử dụng Round Key 128 bit tương ứng được xác định từ thuật toán Key Expansion. Do đó, việc cung cấp Round Key đúng thứ tự là điều kiện cần thiết để bảo đảm kết quả mã hóa.

Về mặt phần cứng, AddRoundKey có cấu trúc tương đối đơn giản vì có thể được hiện thực bằng các cổng XOR. Khi xử lý toàn bộ State song song, phép biến đổi tương ứng với 128 phép XOR một bit.

Mặc dù phép XOR không đòi hỏi một khối tính toán phức tạp, thời điểm cung cấp Round Key phải phù hợp với quá trình cập nhật State. Vì vậy, trong một thiết kế AES sử dụng nhiều chu kỳ, AddRoundKey cần được phối hợp với Datapath và logic điều khiển của từng Round.

Tóm lại, bốn phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey có chức năng khác nhau nhưng liên kết chặt chẽ trong thuật toán AES-256. SubBytes tạo phép thay thế phi tuyến; ShiftRows hoán vị vị trí byte; MixColumns thực hiện biến đổi tuyến tính trên các cột; còn AddRoundKey kết hợp State với Round Key.

Trong quá trình hiện thực trên FPGA, các phép biến đổi này có thể được tổ chức theo những kiến trúc Datapath khác nhau, nhưng phải giữ nguyên quan hệ toán học và thứ tự xử lý được quy định trong FIPS 197.

Đặc biệt, SubBytes và AddRoundKey có quan hệ trực tiếp với Key Expansion do quá trình sinh Round Key cũng sử dụng phép SubWord dựa trên AES S-box. Vì vậy, thuật toán Key Expansion sẽ được trình bày chi tiết trong mục tiếp theo.

2.3 Thuật toán Key Expansion của AES-256

Trong thuật toán AES-256, phép AddRoundKey được thực hiện nhiều lần trong quá trình mã hóa, với mỗi lần sử dụng một Round Key 128 bit tương ứng. Các Round Key này không phải những giá trị độc lập mà được xác định từ Encryption Key ban đầu thông qua thuật toán Key Expansion.

Theo FIPS 197, Key Expansion sử dụng các phép biến đổi RotWord, SubWord, Round Constant (Rcon) và XOR để tạo thành một dãy word 32 bit gọi là Key Schedule [1]. Đối với AES-256, Encryption Key có độ dài 256 bit và thuật toán thực hiện 14 Round, do đó Key Schedule phải cung cấp đủ dữ liệu cho 15 lần AddRoundKey, bao gồm bước khởi tạo và 14 Round tiếp theo.

Việc nghiên cứu Key Expansion có ý nghĩa đối với thiết kế phần cứng AES-256, bởi quá trình sinh Round Key cần được phối hợp với hoạt động của Cipher Core. Đặc biệt, SubWord sử dụng cùng phép ánh xạ S-box với SubBytes, khiến cách hiện thực S-box có liên hệ trực tiếp đến cấu trúc của cả hai khối chức năng.

2.3.1 Cấu trúc Key Schedule và Round Key

Trong AES-256, ba tham số của thuật toán có giá trị $N_b = 4$, $N_k = 8$ và $N_r = 14$. Encryption Key ban đầu được chia thành tám word, mỗi word gồm bốn byte. Key Expansion tiếp tục tạo các word mới để thu được một Key Schedule gồm:

$$N_w = N_b(N_r + 1) = 4(14 + 1) = 60.$$

Gọi K là Encryption Key 256 bit và $k_0, k_1, \ldots, k_{31}$ là các byte của khóa theo thứ tự quy định. Tám word đầu tiên được xác định trực tiếp từ Encryption Key:

$$w[i] = [k_{4i}, k_{4i+1}, k_{4i+2}, k_{4i+3}], \qquad 0 \leq i < 8.$$

Các word còn lại, từ $w[8]$ đến $w[59]$, được sinh lần lượt theo quan hệ đệ quy của Key Expansion. Toàn bộ Key Schedule được biểu diễn dưới dạng:

$$W = \{w[0], w[1], \ldots, w[59]\}.$$

Mỗi Round Key được tạo thành bằng cách ghép bốn word liên tiếp. Với $K^{(r)}$ là Round Key được sử dụng tại Round thứ r, ta có:

$$K^{(r)} = w[4r] \parallel w[4r + 1] \parallel w[4r + 2] \parallel w[4r + 3], \quad 0 \leq r \leq 14,$$

trong đó $\parallel$ biểu diễn phép ghép word. Cách phân chia các word trong Key Schedule được minh họa ở Bảng 3.

Bảng 3: Quan hệ giữa Key Schedule và các Round Key trong AES-256.

Round Key	Các word	Vị trí sử dụng
$K^{(0)}$	$w[0]$–$w[3]$	Initial AddRoundKey
$K^{(1)}$	$w[4]$–$w[7]$	Round 1
$K^{(2)}$	$w[8]$–$w[11]$	Round 2
$K^{(3)}$	$w[12]$–$w[15]$	Round 3
...	...	...
$K^{(14)}$	$w[56]$–$w[59]$	Round 14

Một đặc điểm của AES-256 là tám word trong Encryption Key ban đầu tương ứng với hai Round Key đầu tiên. Cụ thể, $K^{(0)}$ được sử dụng cho Initial AddRoundKey, trong khi $K^{(1)}$ được sử dụng tại Round 1. Vì vậy, Key Expansion bắt đầu sinh thêm word từ $w[8]$, tương ứng với Round Key $K^{(2)}$.

Cách tổ chức này khác với AES-128, trong đó Encryption Key ban đầu chỉ gồm bốn word và tương ứng với một Round Key. Sự khác biệt về độ dài khóa cũng làm thay đổi quy tắc sinh các word tiếp theo.

2.3.2 Các phép biến đổi sử dụng trong Key Expansion

Key Expansion của AES-256 sử dụng ba thành phần chính: RotWord, SubWord và Rcon. Các phép biến đổi này được áp dụng có điều kiện, tùy theo vị trí của word cần sinh. **RotWord**

RotWord thực hiện phép dịch vòng sang trái một byte đối với word 32 bit. Với word đầu vào gồm bốn byte $[a_0, a_1, a_2, a_3]$, ta có:

$$\text{RotWord}([a_0, a_1, a_2, a_3]) = [a_1, a_2, a_3, a_0].$$

RotWord chỉ thay đổi thứ tự các byte mà không thực hiện phép toán trên giá trị của từng byte. Vì vậy, đối với một word có cấu trúc cố định, phép biến đổi có thể được biểu diễn bằng sự sắp xếp lại các đường dữ liệu.

SubWord

SubWord thực hiện phép thay thế S-box độc lập trên từng byte của một word 32 bit:

$$\text{SubWord}([a_0, a_1, a_2, a_3]) = [\text{SBox}(a_0), \text{SBox}(a_1), \text{SBox}(a_2), \text{SBox}(a_3)]. \tag{22}$$

S-box được sử dụng trong SubWord là cùng một phép ánh xạ phi tuyến đã trình bày ở phần SubBytes. Do đó, SubWord không yêu cầu một bảng S-box khác, mà chỉ áp dụng phép thay thế trên bốn byte của word thay vì toàn bộ State 128 bit.

Về mặt thuật toán, các phép thay thế byte trong SubWord có thể được thực hiện độc lập. Tuy nhiên, việc sử dụng bao nhiêu phần tử S-box vật lý và chúng được tổ chức song song hay tuần tự sẽ phụ thuộc vào kiến trúc phần cứng cụ thể.

Round Constant (Rcon)

Rcon là dãy word hằng số được sử dụng tại những vị trí xác định trong Key Expansion. Mỗi phần tử Rcon gồm bốn byte, trong đó chỉ byte đầu tiên có thể khác 0:

$$\text{Rcon}[j] = [rc_j, \texttt{00}, \texttt{00}, \texttt{00}].$$

Giá trị rc_j được xác định trên trường hữu hạn $GF(2^8)$ theo quan hệ:

$$rc_1 = \texttt{01}, \qquad rc_{j+1} = \text{xtime}(rc_j).$$

Ở đây, xtime là phép nhân với 02 trong $GF(2^8)$, đã được giới thiệu ở Mục 2.2.

Đối với AES-256, thuật toán sử dụng bảy phần tử Rcon đầu tiên, được trình bày trong Bảng 4.

Bảng 4: Các giá trị Rcon sử dụng trong Key Expansion của AES-256

j	Rcon[j]	j	Rcon[j]
1	01000000	5	10000000
2	02000000	6	20000000
3	04000000	7	40000000
4	08000000	–	–

Rcon chỉ được sử dụng trong nhánh biến đổi gồm RotWord và SubWord. Trong AES-256, nhánh này xuất hiện tại các chỉ số word là bội số của 8, bắt đầu từ $i = 8$.

2.3.3 Quy tắc sinh word của AES-256

Sau khi khởi tạo tám word đầu tiên, thuật toán tiếp tục sinh các word từ $w[8]$ đến $w[59]$. Mỗi word mới được tính dựa trên word liền trước và word cách đó tám vị trí.

Với $8 \leq i < 60$, quan hệ tổng quát được viết:

$$w[i] = w[i-8] \oplus t_i,$$

trong đó t_i là giá trị trung gian được xác định từ $w[i-1]$.

Để biểu diễn ngắn gọn quy tắc biến đổi, đặt:

$$\mathcal{G}(x, j) = \text{SubWord}(\text{RotWord}(x)) \oplus \text{Rcon}[j].$$

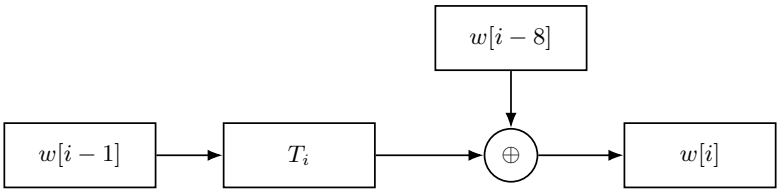
Khi đó, giá trị t_i được tính theo ba trường hợp:

$$t_i = \begin{cases} \mathcal{G}(w[i-1], i/8), & i \bmod 8 = 0, \\ \text{SubWord}(w[i-1]), & i \bmod 8 = 4, \\ w[i-1], & \text{các trường hợp còn lại.} \end{cases} \tag{23}$$

Ở trường hợp thứ nhất, khi i là bội số của 8, word trước đó được xử lý qua RotWord, SubWord và XOR với Rcon. Đối với trường hợp thứ hai, khi $i \bmod 8 = 4$, thuật toán chỉ áp dụng SubWord mà không sử dụng RotWord hoặc Rcon. Ở các vị trí còn lại, giá trị trung gian chính là word liền trước.

Trường hợp $i \bmod 8 = 4$ là một đặc điểm riêng của AES-256 so với AES-128 và AES-192. Nhánh này bổ sung phép biến đổi SubWord trong quá trình sinh Key Schedule, nhưng không làm thay đổi nguyên lý sử dụng Round Key của Cipher Core.

Hình 4 biểu diễn quan hệ tổng quát giữa các word trong thuật toán.



Hình 4: Quan hệ sinh word trong Key Expansion của AES-256.

Trong sơ đồ, khối T_i biểu diễn phép biến đổi tạo ra giá trị trung gian t_i. Đây là một ký hiệu chức năng nhằm mô tả thuật toán, không phải một module phần cứng cố định hoặc một tầng xử lý bắt buộc trong RTL.

Đối với Key Schedule gồm 60 word, thuật toán phải sinh thêm 52 word từ tám word của Encryption Key ban đầu. Trong quá trình này, nhánh RotWord–SubWord–Rcon được áp dụng 7 lần, còn nhánh SubWord riêng được áp dụng 6 lần.

Những số lần trên mô tả số phép biến đổi của thuật toán Key Expansion. Chúng không trực tiếp quyết định số S-box hoặc số chu kỳ xung nhịp của một thiết kế phần cứng cụ thể.

2.3.4 Ý nghĩa của Key Expansion đối với thiết kế phần cứng

Trong đặc tả AES-256, Key Expansion xác định giá trị và thứ tự sử dụng các Round Key, nhưng không quy định cách tổ chức tài nguyên phần cứng để tạo ra chúng. Vì vậy, cùng một thuật toán có thể được hiện thực theo nhiều kiến trúc khác nhau.

Một phương án là tính trước toàn bộ Key Schedule và lưu lại để cung cấp cho Cipher Core. Phương án khác là sinh Round Key theo từng giai đoạn, đồng bộ với quá trình thực hiện các Round.

Hai phương án có những yêu cầu khác nhau về lưu trữ và điều khiển. Việc tính trước Key Schedule đòi hỏi cơ chế lưu hoặc truy xuất các Round Key đã được tạo ra, trong khi phương án sinh theo từng giai đoạn cần phối hợp thời điểm cập nhật Key Expansion với Cipher Datapath.

Bên cạnh đó, SubWord sử dụng cùng S-box với SubBytes. Do đó, việc lựa chọn phương pháp hiện thực S-box có thể ảnh hưởng đến cả đường dữ liệu mã hóa và logic sinh Round Key. Chẳng hạn, một S-box tổ hợp và một S-box sử dụng bộ nhớ đồng bộ có thể tạo ra cùng giá trị thay thế nhưng có đặc điểm thời gian truy xuất khác nhau.

Tuy nhiên, không thể suy ra số chu kỳ xử lý hoặc mức sử dụng tài nguyên của một thiết kế chỉ từ thuật toán Key Expansion. Các thông số này còn phụ thuộc vào số lượng phần tử S-box, cách chia sẻ tài nguyên, tổ chức thanh ghi và FSM.

Trong đề tài này, Key Expansion là một thành phần cần được xem xét khi phân tích các kiến trúc AES-256 trên FPGA. Những quyết định về cách hiện thực SubWord, cách cung cấp Round Key và phối hợp điều khiển sẽ được đối chiếu với RTL thực tế ở chương thiết kế hệ thống.

2.4 Các phương pháp hiện thực S-box trên FPGA

S-box là thành phần thực hiện phép biến đổi phi tuyến SubBytes trong thuật toán AES. Như đã trình bày ở Mục 2.2, S-box nhận một byte đầu vào và tạo ra một byte đầu ra thông qua phép nghịch đảo trong $GF(2^8)$ kết hợp với Affine Transformation. Toàn bộ phép biến đổi được xác định bởi một ánh xạ cố định gồm 256 giá trị đầu vào và 256 giá trị đầu ra tương ứng.

Mặc dù chức năng của S-box đã được quy định trong chuẩn AES, cách hiện thực phần cứng không bị giới hạn ở một cấu trúc duy nhất. Trên FPGA, có thể sử dụng bảng tra được ánh xạ vào logic LUT, bộ nhớ Block RAM (BRAM), hoặc mạch tính toán dựa trên Composite Field.

Ba phương pháp này thực hiện cùng một phép ánh xạ S-box nhưng có sự khác biệt về cách tổ chức phần cứng, tài nguyên sử dụng và đặc điểm thời gian xử lý. Vì vậy, việc lựa chọn kiến trúc S-box có thể ảnh hưởng đến hiệu năng của Cipher Core cũng như Key Expansion trong thiết kế AES-256.

2.4.1 Hiện thực S-box bằng Look-Up Table (LUT)

Phương pháp Look-Up Table (LUT) sử dụng các giá trị đã được xác định trước để thực hiện phép thay thế byte. Thay vì tính toán Multiplicative Inverse và Affine Transformation trong thời gian hoạt động, toàn bộ kết quả S-box được lưu dưới dạng một bảng ánh xạ cố định.

Với đầu vào a có độ rộng 8 bit, giá trị S-box được xác định bởi:

$$b = \mathrm{SBox}[a], \qquad a, b \in \{0,1\}^8.$$

Do có $2^8 = 256$ giá trị đầu vào, bảng S-box có thể được biểu diễn dưới dạng một ROM logic gồm 256 phần tử, mỗi phần tử có độ rộng 8 bit. Dung lượng thông tin của bảng là:

$$M_{\mathrm{SBox}} = 256 \times 8 = 2048 \text{ bit}.$$

Giá trị này mô tả số bit cần để biểu diễn bảng tra, không đồng nghĩa với số bit lưu trữ vật lý hoặc số LUT FPGA thực tế sau quá trình Synthesis. Với cách hiện thực tổ hợp, tám bit đầu vào được sử dụng để lựa chọn một trong 256 giá trị của bảng. Kết quả đầu ra được xác định bởi hàm logic của dữ liệu đầu vào mà không cần một xung nhịp đọc bộ nhớ riêng.

Về mặt RTL, S-box có thể được mô tả bằng câu lệnh `case`, một hàm ánh xạ byte, hoặc một bảng hằng số. Công cụ Synthesis sẽ phân tích mô tả này và ánh xạ thành các tài nguyên logic phù hợp với thiết bị FPGA được lựa chọn.

Cần phân biệt bảng tra S-box ở mức thuật toán với các phần tử LUT vật lý của FPGA. Một bảng S-box 256 phần tử không nhất thiết tương ứng với một LUT phần cứng duy nhất. Tùy cấu trúc RTL và khả năng tối ưu hóa, bảng ánh xạ có thể được triển khai bằng mạng LUT tổ hợp hoặc những tài nguyên khác được công cụ Synthesis lựa chọn.

Đặc điểm đối với kiến trúc AES-256

Phương pháp LUT có ưu điểm là cấu trúc dễ mô tả và thuận tiện cho việc kiểm chứng chức

năng. Dữ liệu đầu ra được xác định trực tiếp từ byte đầu vào, giúp việc tích hợp vào Datapath tương đối đơn giản.

Do SubBytes xử lý 16 byte của State, một kiến trúc thực hiện toàn bộ phép biến đổi song song cần cung cấp khả năng tra S-box cho 16 byte. Ngoài ra, Key Expansion cũng sử dụng S-box trong phép SubWord để xử lý các word 32 bit.

Việc bố trí nhiều đường tra S-box song song có thể làm tăng nhu cầu sử dụng tài nguyên logic. Ngược lại, chia sẻ một số đường tra giữa nhiều byte có thể giảm tài nguyên nhưng cần thêm cơ chế điều khiển và phân chia thời gian xử lý.

Đặc điểm này tạo ra sự đánh đổi giữa mức độ song song và chi phí phần cứng. Vì vậy, độ trễ của kiến trúc LUT không chỉ phụ thuộc vào bản thân S-box mà còn phụ thuộc vào cách tổ chức Datapath, thanh ghi và số đường tra được triển khai.

Trong nghiên cứu này, phương pháp LUT được xem là một hướng hiện thực S-box dựa trên logic tra bảng. Các thông số thực tế về LUT, Flip-Flop và Timing cần được xác định từ kết quả Synthesis và Implementation, thay vì suy ra trực tiếp từ kích thước bảng S-box.

2.4.2 Hiện thực S-box bằng Block RAM (BRAM)

Bên cạnh các phần tử LUT, FPGA còn tích hợp những khối bộ nhớ chuyên dụng gọi là Block RAM (BRAM). Các khối này có thể được sử dụng để lưu trữ dữ liệu, hệ số tính toán hoặc các bảng hằng số được truy xuất trong quá trình hoạt động.

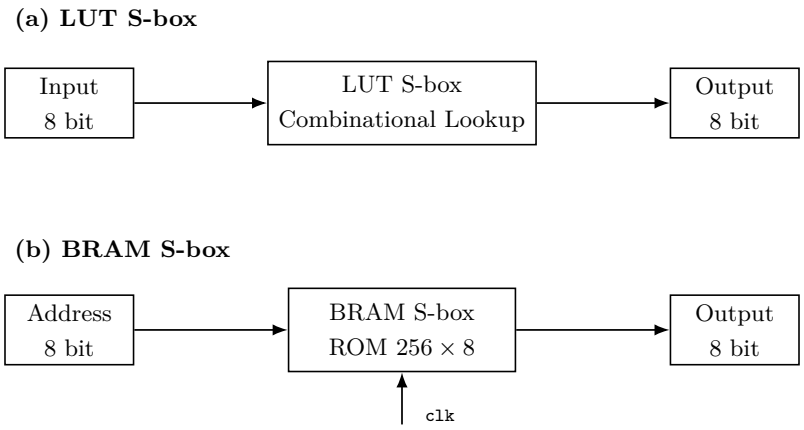
Đối với AES S-box, bảng ánh xạ gồm 256 phần tử có thể được tổ chức thành một ROM với độ sâu 256 và độ rộng dữ liệu 8 bit. Byte đầu vào đóng vai trò địa chỉ truy xuất, còn dữ liệu đọc từ ROM là kết quả của phép thay thế.

Về mặt chức năng, phương pháp BRAM vẫn thực hiện ánh xạ: $b = \text{SBox}[a]$ Tuy nhiên, điểm khác biệt quan trọng nằm ở đặc điểm truy xuất bộ nhớ. Trên các kiến trúc FPGA AMD/Xilinx, BRAM sử dụng cơ chế đọc đồng bộ với xung nhịp, trong khi Distributed RAM có thể hỗ trợ đọc bất đồng bộ.

Đối với cấu hình đọc đồng bộ thông thường, địa chỉ được lấy mẫu tại cạnh xung nhịp và dữ liệu tương ứng được cập nhật ở đầu ra theo thời điểm quy định của bộ nhớ.

Quan hệ hoạt động có thể được biểu diễn ở mức khái niệm như sau: $q_{n+1} = \text{SBox}[a_n]$

trong đó a_n là địa chỉ được lấy mẫu tại cạnh xung nhịp thứ n và q_{n+1} biểu diễn dữ liệu được quan sát ở giai đoạn kế tiếp. Biểu thức trên mô tả một mô hình đọc đồng bộ đơn giản; độ trễ cụ thể còn phụ thuộc vào cấu hình đầu ra và các thanh ghi bổ sung.



Hình 5: Nguyên lý truy xuất S-box bằng logic LUT tổ hợp và BRAM đồng bộ.

Ảnh hưởng đến tổ chức Datapath

Khi thay thế một S-box tổ hợp bằng một S-box sử dụng BRAM đồng bộ, Datapath phải bảo đảm dữ liệu đọc từ bộ nhớ được sử dụng đúng thời điểm.

Nếu các phép biến đổi tiếp theo phụ thuộc vào kết quả SubBytes, logic điều khiển có thể cần bổ sung giai đoạn đọc S-box hoặc điều chỉnh thời điểm cập nhật State Register.

Tương tự, nếu Key Expansion sử dụng BRAM để thực hiện SubWord, Round Key cũng phải được cung cấp phù hợp với lịch xử lý của Cipher Core.

Một yếu tố khác là số lượng cổng đọc bộ nhớ. Các khối BRAM có số cổng truy xuất giới hạn, vì vậy một ROM đơn lẻ không thể mặc nhiên đáp ứng mọi yêu cầu tra cứu song song.

Để xử lý nhiều byte trong cùng một giai đoạn, thiết kế có thể sử dụng nhiều bản sao bảng S-box, nhiều khối bộ nhớ độc lập hoặc chia sẻ tài nguyên qua nhiều chu kỳ xung nhịp.

Việc lựa chọn phương án nào phụ thuộc vào yêu cầu về Latency, Throughput và số tài nguyên BRAM sẵn có.

Phương pháp BRAM có khả năng chuyển một phần chức năng tra bảng từ mạng LUT sang bộ nhớ chuyên dụng. Tuy nhiên, một bảng S-box chỉ chứa 2048 bit, trong khi mỗi khối BRAM vật lý thường có dung lượng lớn hơn. Do đó, việc sử dụng BRAM cần được xem xét cả về hiệu suất khai thác dung lượng bộ nhớ và số lượng khối bộ nhớ cần triển khai.

2.4.3 Hiện thực S-box bằng Composite Field

Khác với phương pháp LUT và BRAM dựa trên việc lưu trữ các giá trị thay thế, Composite Field khai thác trực tiếp cấu trúc đại số của S-box để xây dựng mạch tính toán. Trong AES, S-box được xác định bởi Multiplicative Inverse trên $GF(2^8)$, sau đó là Affine Transformation. Phép nghịch đảo là thành phần có cấu trúc toán học phức tạp hơn so với các phép XOR thông thường.

Phương pháp Composite Field biến đổi cách biểu diễn phần tử của $GF(2^8)$ sang một trường mở rộng được xây dựng từ các trường con có bậc thấp hơn. Một hướng được nghiên cứu phổ

biến là sử dụng:

$$GF(2^8) \cong GF\big((2^4)^2\big),$$

trong đó ký hiệu $\cong$ biểu diễn quan hệ đẳng cấu giữa hai trường hữu hạn.

Hai trường có cùng số phần tử và có cấu trúc đại số tương đương, nhưng sử dụng các cách biểu diễn khác nhau.

Nhờ phép biến đổi cơ sở, một byte có thể được biểu diễn bằng hai phần tử 4 bit thuộc $GF(2^4)$. Phép nghịch đảo trong $GF(2^8)$ sau đó được thực hiện thông qua các phép toán trên trường con.
Isomorphic Mapping

Giả sử Φ là phép Isomorphic Mapping từ biểu diễn AES ban đầu sang Composite Field:

$$\Phi : GF(2^8) \longrightarrow GF\big((2^4)^2\big).$$

Với byte đầu vào b, giá trị sau ánh xạ là:$u = \Phi(b)$. Phần tử u sau đó được xử lý bằng các phép toán trên Composite Field để xác định Multiplicative Inverse.

Kết quả được chuyển ngược về biểu diễn AES thông qua Inverse Isomorphic Mapping Φ^{-1}.

Với quy ước giá trị nghịch đảo của 0 được đặt bằng 0 trong S-box, phép biến đổi tổng thể có thể biểu diễn:

$$\mathrm{SBox}(b) = \mathrm{Affine}\left(\Phi^{-1}\left(\mathrm{Inv}_{CF}(\Phi(b))\right)\right),$$

trong đó Inv_{CF} là phép nghịch đảo thực hiện trong Composite Field.

Biểu thức trên cho thấy Composite Field không thay đổi chức năng S-box. Phương pháp này chỉ thay đổi cách thực hiện phép nghịch đảo bằng phần cứng.

Isomorphic Mapping có thể được biểu diễn dưới dạng một phép biến đổi tuyến tính trên các bit. Do đó, các khối Mapping và Inverse Mapping có thể được hiện thực bằng mạng XOR với các kết nối được xác định trước.

Tuy nhiên, ma trận ánh xạ không thể lựa chọn tùy ý. Nó phụ thuộc vào đa thức xác định trường và cơ sở biểu diễn được chọn cho Composite Field. Hai khối Mapping phải tương thích với phép toán nghịch đảo được triển khai ở giữa. **Phép nghịch đảo trên Composite Field**

Để minh họa nguyên lý, xét một cách biểu diễn Composite Field bằng phần tử z thỏa mãn:

$$z^2 + z + \lambda = 0, \qquad \lambda \in GF(2^4),$$

trong đó λ được lựa chọn để đa thức trên là bất khả quy trong $GF(2^4)$.

Một phần tử u của Composite Field có thể viết dưới dạng:

$$u = az + b, \qquad a, b \in GF(2^4).$$

Trong biểu diễn này, a và b là hai phần tử 4 bit.

Do $z^2 + z + \lambda = 0$, phần tử liên hợp của u được xác định bởi:

$$\overline{u} = az + (a + b).$$

Tích của u với phần tử liên hợp thuộc trường con $GF(2^4)$:

$$N(u) = u\overline{u} = b^2 + ab + \lambda a^2.$$

Với $u \neq 0$, phép nghịch đảo được tính theo:

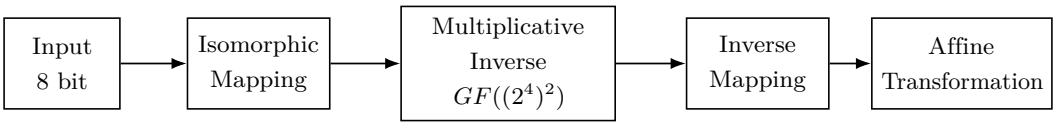
$$u^{-1} = \overline{u}\,[N(u)]^{-1}.$$

Như vậy, thay vì trực tiếp tìm nghịch đảo trên $GF(2^8)$, phép toán có thể được phân tách thành các phép nhân, bình phương và nghịch đảo trên $GF(2^4)$.

Các phép toán trên $GF(2^4)$ tiếp tục có thể được hiện thực bằng những mạng logic nhỏ hơn, hoặc được phân rã thành các trường con bậc thấp, chẳng hạn $GF(2^2)$.

Đây là nguyên lý cơ bản tạo nên khả năng tối ưu phần cứng của phương pháp Composite Field. Tuy nhiên, hiệu quả cụ thể phụ thuộc vào cách lựa chọn cơ sở trường, ma trận ánh xạ và phương pháp tối ưu các biểu thức logic.

Sơ đồ khối nguyên lý được trình bày trong Hình 6.



Hình 6: Nguyên lý hiện thực AES S-box bằng Composite Field

Sơ đồ trên mô tả các khối chức năng ở mức lý thuyết. Cấu trúc cổng logic và các phép toán cụ thể có thể thay đổi giữa những kiến trúc Composite Field khác nhau. **Đặc điểm đối với hiện thực FPGA**

Phương pháp Composite Field có khả năng loại bỏ nhu cầu lưu trực tiếp bảng S-box 256 phần tử bằng cách thay thế phép tra bảng bằng mạch tính toán. Các phép toán trong trường con có thể được triển khai thông qua XOR, AND và các mạng logic tổ hợp khác. Tuy nhiên, Isomorphic Mapping, phép nghịch đảo và Affine Transformation cùng đóng góp vào độ phức tạp của mạch. Do đó, Composite Field không mặc nhiên sử dụng ít LUT hơn hoặc có đường truyền nhanh hơn phương pháp tra bảng trên mọi nền tảng FPGA.

Một kiến trúc được tối ưu cho diện tích cổng logic có thể không phải phương án đạt Timing tốt nhất trên một FPGA cụ thể. Kết quả còn phụ thuộc vào khả năng ánh xạ logic, fanout, độ sâu đường tổ hợp và đặc điểm của công cụ Synthesis.

Đối với nghiên cứu AES-256, Composite Field là một phương án đáng xem xét vì sử dụng cơ sở toán học khác với LUT và BRAM, trong khi vẫn bảo đảm cùng chức năng S-box. Hiệu quả thực tế phải được đánh giá bằng các kết quả Synthesis, Implementation và kiểm chứng chức năng.

2.4.4 So sánh nguyên lý của các phương pháp hiện thực

Ba phương pháp LUT, BRAM và Composite Field đều hướng đến việc thực hiện cùng phép ánh xạ AES S-box. Sự khác biệt chính nằm ở tài nguyên thực hiện và cách tổ chức quá trình xử lý.

Bảng 5 tổng hợp các đặc điểm nguyên lý của từng phương pháp.

Bảng 5: So sánh nguyên lý các phương pháp hiện thực AES S-box trên FPGA.

Tiêu chí	LUT	BRAM	Composite Field
Nguyên lý	Tra bảng ánh xạ S-box.	Đọc bảng S-box từ ROM đồng bộ.	Tính toán S-box bằng phép toán trường hữu hạn.
Tài nguyên chính	Logic LUT hoặc Distributed ROM.	Block RAM và logic giao tiếp.	Mạng logic tổ hợp, chủ yếu XOR và các phép toán bit.
Đặc điểm thời gian	Đầu ra theo đường tổ hợp.	Phụ thuộc cạnh clock và cấu hình bộ nhớ.	Phụ thuộc độ sâu và cấu trúc mạng logic.
Khả năng song song	Có thể nhân bản các đường tra.	Phụ thuộc số cổng đọc và số khối bộ nhớ.	Có thể nhân bản các khối tính toán.
Vấn đề cần đánh giá	Số LUT và Timing.	Số BRAM, Latency và Timing.	Số LUT, độ sâu logic và Timing.

Bảng so sánh cho thấy không thể xác định một phương pháp hiện thực S-box là tối ưu trong mọi trường hợp chỉ dựa trên nguyên lý hoạt động.

Phương pháp LUT thuận tiện cho việc tổ chức tra bảng tổ hợp. BRAM khai thác tài nguyên bộ nhớ chuyên dụng nhưng cần quan tâm đến cơ chế đọc đồng bộ. Composite Field sử dụng phép tính toán đại số, do đó hiệu quả phụ thuộc nhiều vào cấu trúc mạng logic được xây dựng.

Để đánh giá công bằng các phương án hiện thực, cần đặt chúng trong những điều kiện triển khai phù hợp, chẳng hạn cùng thiết bị FPGA, cùng yêu cầu chức năng và các ràng buộc Timing được xác định rõ.

Bên cạnh số tài nguyên LUT, Flip-Flop và BRAM, việc đánh giá cần xem xét Latency, Throughput và khả năng đáp ứng Timing của toàn bộ lõi AES-256, không chỉ riêng phần tử S-box.

2.5 Nguyên lý hoạt động của Dummy Noise

Bên cạnh việc thay đổi phương pháp hiện thực S-box, một hướng tiếp cận khác trong thiết kế phần cứng AES là bổ sung hoạt động chuyển mạch trong quá trình mã hóa. Phương pháp này không thay đổi các phép biến đổi của thuật toán AES mà sử dụng thêm logic để tạo ra Switching Activity bên trong mạch.

2.5.1 Switching Activity và Dynamic Power

Trong mạch số CMOS, Dynamic Power chủ yếu phát sinh khi các nút logic chuyển đổi trạng thái trong quá trình hoạt động. Mức công suất này phụ thuộc vào tần số xung nhịp, điện áp hoạt động, điện dung chuyển mạch và mức độ thay đổi của các tín hiệu bên trong mạch. Một mô hình gần đúng thường được sử dụng để mô tả Dynamic Power là:

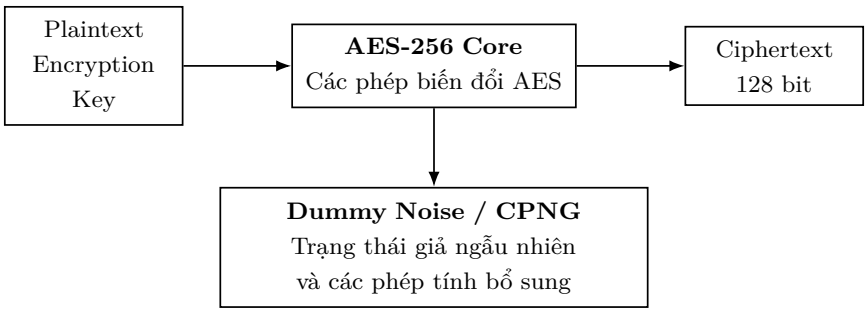
$$P_{\text{dyn}} \approx \alpha C V^2 f,$$

trong đó α là Switching Activity Factor, C là điện dung chuyển mạch tương đương, V là điện áp hoạt động và f là tần số hoạt động. Đối với một thiết kế gồm nhiều nút logic, công suất tổng thể phụ thuộc vào hoạt động chuyển mạch của từng nút.

Trong AES Core, các phép biến đổi SubBytes, MixColumns và quá trình cập nhật các thanh ghi tạo ra Switching Activity phụ thuộc vào dữ liệu đang được xử lý. Vì vậy, hoạt động nội bộ của mạch có thể ảnh hưởng đến đặc điểm tiêu thụ công suất của thiết bị. Đây cũng là một trong những cơ sở của Power Side-Channel Analysis, trong đó thông tin thu được từ đặc điểm tiêu thụ công suất được sử dụng để phân tích hoạt động của mạch mã hóa.

2.5.2 Nguyên lý tạo Dummy Noise

Dummy Noise sử dụng một khối logic bổ sung hoạt động song song với AES Core. Trong quá trình mã hóa, khối này thực hiện các phép biến đổi nội bộ nhằm tạo thêm các chuyển trạng thái logic. Đối với hướng hiện thực CPNG (Correlated-Power-Noise Generator), dữ liệu trung gian của AES có thể được kết hợp với một trạng thái giả ngẫu nhiên để tạo đầu vào cho các phép tính bổ sung. Các kết quả này chỉ phục vụ hoạt động của khối CPNG và không được sử dụng để xác định Ciphertext. Nguyên lý tổ chức được minh họa trong Hình 7.



Hình 7: Nguyên lý bổ sung hoạt động chuyển mạch bằng Dummy Noise trong AES-256.

Trong cấu trúc này, AES Core vẫn thực hiện đầy đủ các phép biến đổi theo FIPS 197. Khối Dummy Noise có thể sử dụng thông tin trung gian của AES để tạo hoạt động logic bổ sung, nhưng kết quả của các phép tính này không được đưa trở lại Datapath mã hóa. Cần lưu ý rằng việc bổ sung Dummy Noise có thể làm tăng số phần tử logic và mức tiêu thụ Dynamic Power. Mức độ ảnh hưởng thực tế phụ thuộc vào kiến trúc của khối tạo nhiễu và kết quả ánh xạ phần cứng.

2.5.3 Giới hạn và phương pháp đánh giá

Các nghiên cứu về Side-Channel Countermeasures đã xem xét việc bổ sung hoạt động tính toán hoặc tạo nhiễu nhằm thay đổi đặc điểm công suất của hệ thống mã hóa. Tuy nhiên, Switching Activity bổ sung không đồng nghĩa với việc hệ thống đã có khả năng chống Power Side-Channel Attack. Hiệu quả bảo vệ cần được đánh giá bằng các phép đo và phương pháp phân tích chuyên biệt, chẳng hạn Correlation Power Analysis (CPA). Đối với biến thể Dummy Noise, hai yêu cầu kiểm chứng chức năng cơ bản là:

- Ciphertext phải khớp với kết quả AES-256 tham chiếu đối với cùng Plaintext và Encryption

Key.

- Logic Dummy Noise phải có hoạt động chuyển mạch trong thời gian AES Core thực hiện mã hóa.

Bên cạnh kiểm chứng chức năng, việc đánh giá biến thể còn cần xem xét tài nguyên FPGA, Timing và công suất tiêu thụ nếu có đủ dữ liệu đo hoặc ước lượng phù hợp. Trong phạm vi báo cáo này, Dummy Noise được xem là một phương án bổ sung Switching Activity để khảo sát đặc điểm hiện thực phần cứng. Các kết quả kiểm chứng hoạt động của CPNG không được sử dụng như bằng chứng về khả năng chống Side-Channel Attack khi chưa thực hiện đánh giá bảo mật tương ứng. Kiến trúc CPNG và cơ chế điều khiển cụ thể của biến thể Noise sẽ được trình bày trong chương thiết kế hệ thống.

2.6 Nền tảng FPGA và kiến trúc SoC

FPGA (Field-Programmable Gate Array) là một loại mạch tích hợp có cấu trúc phần cứng có thể cấu hình sau quá trình chế tạo. Khác với bộ xử lý thực thi tuần tự các lệnh phần mềm, FPGA cho phép xây dựng các khối logic hoạt động song song và tổ chức đường dữ liệu theo yêu cầu của ứng dụng. Đối với AES-256, FPGA cho phép hiện thực trực tiếp các phép biến đổi của thuật toán bằng phần cứng. Tuy nhiên, hiệu quả của thiết kế phụ thuộc không chỉ vào số lượng tài nguyên sử dụng mà còn vào cấu trúc Datapath, phương pháp điều khiển, Timing và cách giao tiếp với hệ thống xử lý.

2.6.1 Kiến trúc và tài nguyên FPGA

Các FPGA hiện đại được tổ chức từ nhiều loại tài nguyên có chức năng khác nhau. Trong đó, những tài nguyên thường được quan tâm khi thiết kế mạch số gồm Look-Up Table (LUT), Flip-Flop (FF), Block RAM (BRAM) và Digital Signal Processing (DSP) Block.

- **Look-Up Table (LUT):** Là phần tử logic có thể lập trình để thực hiện các hàm Boolean. LUT thường được sử dụng để hiện thực logic tổ hợp như Multiplexer, XOR và các mạch điều khiển.
- **Flip-Flop (FF):** Là phần tử lưu trữ trạng thái theo xung nhịp. Trong một thiết kế tuần tự, FF được sử dụng để xây dựng các thanh ghi dữ liệu, bộ đếm và State Register của FSM.
- **Block RAM (BRAM):** Là tài nguyên bộ nhớ chuyên dụng được tích hợp bên trong FPGA. BRAM có thể được cấu hình thành các bộ nhớ với độ rộng và độ sâu phù hợp, chẳng hạn ROM hoặc RAM đồng bộ.
- **DSP Block:** Là khối phần cứng chuyên dụng hỗ trợ các phép toán số học như nhân và cộng tích lũy. DSP đặc biệt hữu ích đối với các ứng dụng xử lý tín hiệu, nhưng không nhất thiết phải được sử dụng trong mọi thiết kế AES.

Trong thuật toán AES-256, nhiều phép toán được thực hiện bằng XOR, hoán vị bit và các phép biến đổi phi tuyến. Do đó, LUT và FF là những tài nguyên quan trọng đối với Datapath và

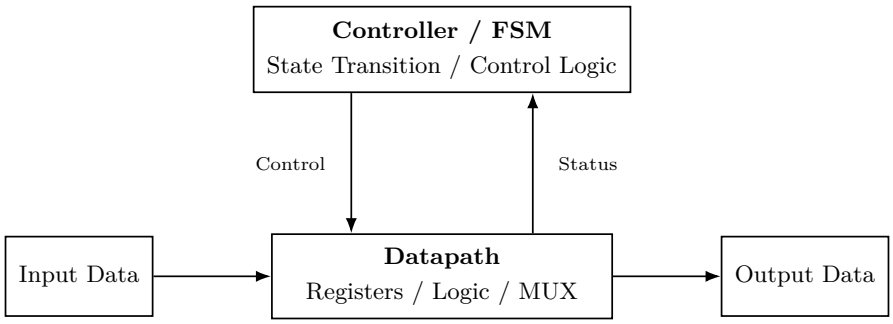
Controller. BRAM có thể được sử dụng để lưu bảng S-box hoặc dữ liệu trung gian, tùy theo kiến trúc được lựa chọn.

2.6.2 Tổ chức Datapath, Controller và các chỉ tiêu Timing

Một hệ thống xử lý dữ liệu số thường có thể được phân chia thành hai thành phần chức năng chính: Datapath và Controller.

Datapath thực hiện các phép toán và lưu trữ dữ liệu trung gian. Thành phần này thường bao gồm các thanh ghi, Multiplexer và các khối logic tổ hợp. Trong khi đó, Controller tạo ra các tín hiệu điều khiển để xác định phép toán cần thực hiện, thời điểm cập nhật thanh ghi và trình tự chuyển giữa các giai đoạn xử lý.

Controller có thể được tổ chức dưới dạng Finite State Machine (FSM). Tại mỗi trạng thái, FSM xác định các tín hiệu điều khiển tương ứng và chuyển sang trạng thái kế tiếp dựa trên điều kiện đầu vào hoặc tín hiệu phản hồi từ Datapath. Quan hệ chức năng giữa hai thành phần được minh họa trong Hình 8.



Hình 8: Mối quan hệ chức năng giữa Datapath và Controller trong hệ thống xử lý số.

Đối với một lõi AES-256 sử dụng kiến trúc lặp, Datapath có thể được tái sử dụng qua nhiều Round, trong khi Controller quản lý trình tự thực hiện và thời điểm cung cấp Round Key. Khi thay đổi cấu trúc Datapath, chẳng hạn sử dụng S-box đồng bộ thay cho S-box tổ hợp, trình tự điều khiển cũng có thể cần được điều chỉnh. Bên cạnh chức năng, việc đánh giá một thiết kế FPGA cần xem xét những chỉ tiêu về thời gian hoạt động.

Clock Frequency và Latency

Clock Frequency (f_{clk}) là tần số xung nhịp được cấp cho thiết kế. Chu kỳ xung nhịp tương ứng được xác định bởi:

$$T_{\text{clk}} = \frac{1}{f_{\text{clk}}}. \tag{24}$$

Latency là khoảng thời gian từ khi một tác vụ được chấp nhận đến khi kết quả của tác vụ đó sẵn sàng. Nếu lõi xử lý cần N_{cycles} chu kỳ xung nhịp, Latency ở mức lõi được xác định bởi:

$$L_{\text{core}} = N_{\text{cycles}} T_{\text{clk}}. \tag{25}$$

Thời gian xử lý theo chu kỳ Clock của AES Core khác với thời gian quan sát từ chương trình ARM/Linux. Phép đo phía phần mềm còn bao gồm chi phí truy cập thanh ghi, giao tiếp AXI

và kiểm tra trạng thái.

Maximum Clock Frequency

Maximum Clock Frequency ($F_{\max}$) biểu thị giới hạn tần số xung nhịp mà thiết kế có thể đáp ứng với các điều kiện Timing xác định. Giới hạn này chịu ảnh hưởng của độ trễ logic tổ hợp, đường định tuyến, đặc điểm thanh ghi và các ràng buộc Timing.

Trong Static Timing Analysis (STA), các đường truyền tín hiệu được kiểm tra theo những yêu cầu về Setup Time và Hold Time. Một đường truyền có độ trễ quá lớn có thể khiến dữ liệu không đến kịp trước cạnh clock yêu cầu, dẫn đến Setup Timing Violation.

Vivado cung cấp các chỉ tiêu như Worst Negative Slack (WNS) và Total Negative Slack (TNS) để đánh giá mức độ đáp ứng Timing của thiết kế [8]. Khi WNS của phân tích Setup không âm, các đường đã được kiểm tra không vi phạm yêu cầu Setup tại các ràng buộc đang áp dụng. Tuy nhiên, kết luận Timing Closure còn cần kiểm tra các yêu cầu khác, bao gồm Hold và tính đầy đủ của các Timing Constraints.

Giá trị $F_{\max}$ không đồng nhất với tần số clock được cấu hình trong Block Design. Để đánh giá $F_{\max}$, cần dựa trên kết quả Timing Analysis của thiết kế đã triển khai với các ràng buộc phù hợp.

Throughput

Throughput mô tả lượng dữ liệu mà hệ thống có thể xử lý trong một đơn vị thời gian. Với lõi AES có Block Size bằng 128 bit, Throughput ở mức lõi có thể được biểu diễn:

$$\text{Throughput}_{\text{core}} = \frac{128 f_{\text{clk}}}{II}, \tag{26}$$

trong đó II (Initiation Interval) là số chu kỳ trung bình giữa hai lần bắt đầu xử lý các Block liên tiếp khi lõi hoạt động liên tục. Đối với kiến trúc xử lý tuần tự từng Block và không chồng lấp các tác vụ, II thường tương ứng với số chu kỳ cần thiết cho mỗi Block, bao gồm cả những khoảng điều khiển bắt buộc giữa hai lần mã hóa. Vì vậy, một thiết kế có $F_{\max}$ cao hơn chưa chắc đạt Throughput cao hơn nếu số chu kỳ xử lý cũng lớn hơn. Khi so sánh các biến thể AES-256, cần xem xét đồng thời Clock Frequency, Latency và Throughput.

2.6.3 Kiến trúc PS–PL trong Zynq UltraScale+ MPSoC

System-on-Chip (SoC) là kiến trúc tích hợp nhiều thành phần xử lý và giao tiếp trên cùng một thiết bị bán dẫn. Đối với Zynq UltraScale+ MPSoC, hệ thống kết hợp Processing System (PS) và Programmable Logic (PL).

PS bao gồm các bộ xử lý ARM cùng những thành phần phục vụ quản lý bộ nhớ, giao tiếp ngoại vi và vận hành hệ thống. Trong khi đó, PL cung cấp các tài nguyên FPGA để hiện thực những khối phần cứng theo yêu cầu ứng dụng.

Sự kết hợp giữa PS và PL cho phép phân chia nhiệm vụ xử lý. PS có thể thực hiện các công việc điều khiển, quản lý dữ liệu và vận hành phần mềm; PL đảm nhiệm những phép toán cần

cấu trúc phần cứng chuyên dụng hoặc khả năng xử lý song song. Hai thành phần giao tiếp với nhau thông qua các giao diện AXI và những tín hiệu hệ thống khác, bao gồm clock, reset và interrupt.

Trong Zynq UltraScale+ MPSoC, các giao diện AXI High-Performance Master (HPM) cho phép PS khởi tạo các giao dịch truy cập tới những IP được kết nối trong PL. Ngược lại, thiết bị cũng hỗ trợ các giao diện khác phục vụ giao dịch từ PL về PS hoặc hệ thống bộ nhớ [7].

Đối với hệ thống AES-256 trên Kria KV260, mô hình PS–PL cho phép phần mềm trên PS điều khiển lõi mã hóa được triển khai trong PL. Việc trao đổi dữ liệu và trạng thái có thể được thực hiện qua những thanh ghi Memory-Mapped, không đòi hỏi phần mềm trực tiếp xử lý các phép biến đổi nội bộ của AES.

2.6.4 Giao tiếp AXI4-Lite và Memory-Mapped I/O

AXI (Advanced eXtensible Interface) là một giao thức thuộc kiến trúc AMBA do Arm phát triển. Trong đó, AXI4-Lite là phiên bản được thiết kế cho các giao dịch truy cập đơn, thường phù hợp với việc điều khiển và truy xuất thanh ghi của các IP phần cứng.

AXI4-Lite không hỗ trợ Burst Transaction gồm nhiều Data Beat như AXI4 thông thường. Mỗi giao dịch đọc hoặc ghi chỉ truyền một Data Beat. Theo đặc tả AMBA AXI, AXI4-Lite hỗ trợ Data Bus có độ rộng 32 hoặc 64 bit [9]. Giao thức được tổ chức thành năm kênh độc lập, gồm ba kênh phục vụ giao dịch ghi và hai kênh phục vụ giao dịch đọc. Các kênh được tổng hợp trong Bảng 6.

Bảng 6: Các kênh giao tiếp trong AXI4-Lite.

Kênh	Handshake	Chức năng
Write Address	`AWVALID / AWREADY`	Truyền địa chỉ ghi từ Master đến Slave.
Write Data	`WVALID / WREADY`	Truyền dữ liệu cần ghi.
Write Response	`BVALID / BREADY`	Phản hồi kết quả giao dịch ghi.
Read Address	`ARVALID / ARREADY`	Truyền địa chỉ đọc từ Master đến Slave.
Read Data	`RVALID / RREADY`	Truyền dữ liệu đọc và thông tin phản hồi.

Nguyên lý VALID/READY Handshake

Mỗi kênh AXI sử dụng hai tín hiệu VALID và READY để điều khiển việc truyền dữ liệu. VALID được phát bởi phía cung cấp thông tin, cho biết địa chỉ, dữ liệu hoặc phản hồi đang hợp lệ. READY được phát bởi phía tiếp nhận, cho biết khả năng chấp nhận thông tin. Một lần truyền trên kênh được hoàn tất tại cạnh lên của clock khi cả VALID và READY đều bằng 1:

$$\text{Transfer} = \text{VALID} \wedge \text{READY}.$$

Nếu VALID bằng 1 nhưng READY bằng 0, phía phát phải giữ thông tin hợp lệ và ổn định cho đến khi giao dịch được chấp nhận theo quy định của giao thức. Do các kênh AXI hoạt động độc lập, Write Address và Write Data không bắt buộc phải được chấp nhận trong cùng một chu kỳ. Sau khi cả địa chỉ và dữ liệu ghi đã được tiếp nhận, Slave trả về Write Response. Đối với giao

dịch đọc, Master gửi địa chỉ qua Read Address Channel, sau đó Slave trả về dữ liệu thông qua Read Data Channel.

Memory-Mapped I/O

Trong mô hình Memory-Mapped I/O (MMIO), các thanh ghi điều khiển và trạng thái của IP được gán những địa chỉ trong không gian địa chỉ của hệ thống. Phần mềm có thể truy cập các thanh ghi này thông qua thao tác đọc và ghi tại địa chỉ tương ứng. Nhờ đó, những dữ liệu như Encryption Key, Plaintext, Ciphertext và Command/Status có thể được trao đổi giữa PS và PL.

Trên hệ thống Linux, chương trình điều khiển có thể sử dụng cơ chế ánh xạ bộ nhớ như `mmap()` thông qua giao diện phù hợp để truy cập vùng thanh ghi phần cứng. Cách giao tiếp này phù hợp với những IP có tập thanh ghi điều khiển tương đối nhỏ. Tuy nhiên, thời gian truy cập qua phần mềm và AXI cần được phân biệt với thời gian tính toán bên trong lõi phần cứng. Trong hệ thống AES-256, AXI4-Lite đóng vai trò giao tiếp điều khiển giữa phần mềm và phần cứng. Các địa chỉ thanh ghi, cấu trúc AXI Controller và trình tự thực hiện giao dịch cụ thể sẽ được phân tích ở chương thiết kế hệ thống.

2.7 Cơ chế triển khai và lựa chọn biến thể phần cứng

Bên cạnh kiến trúc của từng lõi AES-256, cách tổ chức các biến thể trên FPGA cũng ảnh hưởng đến mức sử dụng tài nguyên và khả năng chuyển đổi chức năng của hệ thống. Hai hướng tiếp cận được xem xét trong nghiên cứu này là Dynamic Partial Reconfiguration (DPR) và kiến trúc Replica. DPR cho phép thay đổi một phần cấu hình FPGA trong quá trình thiết bị hoạt động, trong khi Replica tích hợp đồng thời nhiều khối phần cứng và lựa chọn khối cần sử dụng bằng logic điều khiển. Ngoài ra, việc lựa chọn biến thể có thể được thực hiện theo một trình tự xác định hoặc dựa trên cơ chế giả ngẫu nhiên.

2.7.1 Nguyên lý Dynamic Partial Reconfiguration

Dynamic Partial Reconfiguration (DPR) là kỹ thuật cho phép thay đổi cấu hình của một vùng xác định trên FPGA mà không cần cấu hình lại toàn bộ thiết bị. Trong hệ sinh thái AMD/Xilinx, phương pháp thiết kế và triển khai này được gọi là Dynamic Function eXchange.

Trong một thiết kế DFX, hệ thống được phân chia thành phần logic cố định và phần logic có thể tái cấu hình. Khi cần thay đổi chức năng của vùng tái cấu hình, hệ thống nạp một Partial Bitstream tương ứng với chức năng mới. Các thành phần thuộc Static Design được giữ nguyên cấu hình trong quá trình này. Khác với Full Bitstream dùng để cấu hình toàn bộ thiết bị, Partial Bitstream chỉ chứa thông tin cấu hình cần thiết cho vùng được xác định trước. Vì vậy, hệ thống có thể chuyển đổi chức năng phần cứng mà không phải khởi tạo lại toàn bộ thiết kế FPGA. Một quy trình DPR ở mức nguyên lý thường gồm:

1. Hoàn tất hoặc tạm dừng các tác vụ đang sử dụng Reconfigurable Module.
2. Cô lập hoặc kiểm soát các tín hiệu giao tiếp với vùng tái cấu hình.
3. Nạp Partial Bitstream của Reconfigurable Module được lựa chọn.

4. Khởi tạo trạng thái của phần cứng mới và khôi phục giao tiếp.
5. Tiếp tục thực hiện các tác vụ với chức năng vừa được cấu hình.

Trong thời gian tái cấu hình, dữ liệu đầu ra của Reconfigurable Module không được xem là hợp lệ. Do đó, thiết kế cần có cơ chế Decoupling hoặc Isolation phù hợp để tránh ảnh hưởng đến phần logic cố định.

Ưu điểm của DPR là nhiều chức năng phần cứng có thể sử dụng chung một vùng tài nguyên FPGA tại những thời điểm khác nhau. Tuy nhiên, việc chuyển đổi chức năng phát sinh thời gian nạp cấu hình và khởi tạo lại phần cứng. Trong khoảng thời gian này, chức năng thuộc vùng đang tái cấu hình không sẵn sàng xử lý dữ liệu. Do đó, DPR phù hợp với những hệ thống có nhu cầu thay đổi chức năng theo từng giai đoạn, đặc biệt khi việc sử dụng đồng thời tất cả các chức năng không phải là yêu cầu bắt buộc.

2.7.2 Reconfigurable Partition, Reconfigurable Module và Floorplanning

Một thiết kế DFX được tổ chức dựa trên các thành phần chính sau đây:

- **Static Design:** Phần thiết kế được giữ nguyên cấu hình trong quá trình Partial Reconfiguration. Thành phần này có thể bao gồm các khối giao tiếp, điều khiển và những chức năng cần duy trì hoạt động.
- **Reconfigurable Partition (RP):** Một phân vùng logic được xác định là có khả năng tái cấu hình. Đây là vị trí trong hệ thống mà các chức năng phần cứng khác nhau có thể thay thế nhau.
- **Reconfigurable Module (RM):** Một module hoặc netlist được triển khai trong RP. Một RP có thể có nhiều RM tương ứng với các cấu hình chức năng khác nhau.
- **Partial Bitstream:** Dữ liệu cấu hình được sử dụng để nạp một RM vào vùng tái cấu hình tương ứng.

Để nhiều RM có thể sử dụng cùng một RP, các module cần tuân theo giao diện kết nối tương thích với Static Design. Điều này bao gồm những tín hiệu đầu vào, đầu ra và các yêu cầu giao tiếp tại ranh giới của phân vùng.

Một thành phần quan trọng khác của DFX là Floorplanning, tức quá trình xác định vị trí và phạm vi tài nguyên vật lý dành cho các khối thiết kế trên FPGA. Trong Vivado, vùng tài nguyên dành cho RP được xác định bằng ràng buộc Pblock. Theo AMD UG909, mỗi RP cần có một Pblock để xác định các tài nguyên vật lý có thể được sử dụng bởi RM.

Pblock có thể bao gồm các tài nguyên như SLICE, BRAM hoặc DSP, tùy theo yêu cầu của thiết kế và khả năng tái cấu hình của thiết bị.

Khi nhiều RM sử dụng cùng một RP, Pblock cần được lựa chọn sao cho đáp ứng được yêu cầu tài nguyên và khả năng Placement/Routing của tất cả các module. Vì vậy, việc xác định Pblock không chỉ phụ thuộc vào tổng số LUT hoặc FF mà còn liên quan đến số lượng, loại và vị trí phân bố của các tài nguyên cần sử dụng.

Một Pblock quá nhỏ có thể khiến một hoặc nhiều RM không thể Implementation thành công. Ngược lại, việc dành một vùng quá lớn có thể làm giảm lượng tài nguyên còn lại cho Static Design và ảnh hưởng đến hiệu quả Floorplanning của hệ thống.

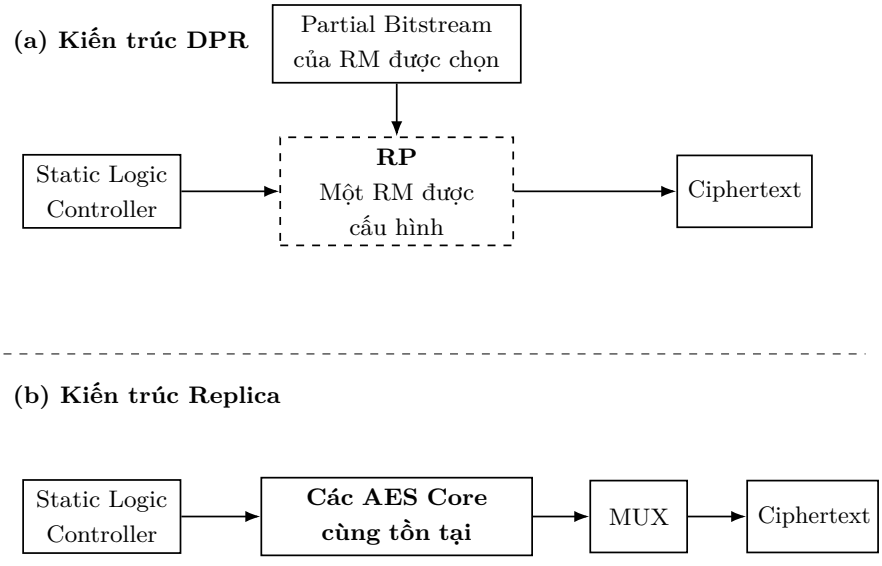
Ngoài khả năng đáp ứng tài nguyên, DFX còn phải duy trì sự tương thích của các kết nối giữa Static Design và RP qua những cấu hình khác nhau. Các yêu cầu này được kiểm tra trong quá trình Implementation và DFX Verification. Đối với hệ thống có nhiều biến thể AES-256, RP có thể được sử dụng để triển khai lần lượt những RM có cùng giao diện chức năng. Khi đó, mỗi thời điểm chỉ có một biến thể AES-256 được cấu hình hoạt động trong RP tương ứng.

Phương pháp xác định Pblock và kết quả Floorplanning thực tế sẽ được trình bày tại chương hiện thực hệ thống trên FPGA.

2.7.3 Nguyên lý kiến trúc Replica

Kiến trúc Replica được triển khai đồng thời nhiều biến thể phần cứng trong cùng một thiết kế FPGA. Các biến thể được tổng hợp và cấu hình từ trước, do đó không cần nạp Partial Bitstream khi chuyển đổi chức năng. Việc lựa chọn biến thể có thể được thực hiện thông qua Multiplexer hoặc logic điều khiển tương ứng. Tại một thời điểm, hệ thống sử dụng kết quả từ biến thể được lựa chọn để cung cấp cho các khối giao tiếp bên ngoài. Trong mô hình Replica, các lõi phần cứng cùng tồn tại trong cấu hình FPGA, nhưng điều đó không nhất thiết có nghĩa tất cả các lõi đều được kích hoạt để mã hóa đồng thời.

So với DPR, ưu điểm của Replica là quá trình chuyển đổi lựa chọn không yêu cầu nạp lại cấu hình FPGA. Tuy nhiên, việc triển khai đồng thời nhiều lõi có thể làm tăng mức sử dụng tài nguyên logic và bộ nhớ. Hình 9 minh họa sự khác biệt ở mức kiến trúc giữa DPR và Replica.



Hình 9: So sánh nguyên lý tổ chức phần cứng giữa DPR và Replica.

Trong sơ đồ DPR, các biến thể được triển khai luân phiên trong cùng một RP. Đối với Replica, các biến thể cùng tồn tại trên FPGA, còn việc chọn kết quả đầu ra được thực hiện bằng logic điều khiển. Những khác biệt cơ bản được tổng hợp trong Bảng 7.

Bảng 7: So sánh nguyên lý DPR và Replica.

Tiêu chí	DPR	Replica
Tổ chức phần cứng	Các RM sử dụng chung một RP theo thời gian.	Nhiều lõi được triển khai đồng thời trên FPGA.
Chuyển đổi biến thể	Nạp Partial Bitstream.	Thay đổi tín hiệu lựa chọn.
Tài nguyên FPGA	Chia sẻ vùng tài nguyên giữa các RM.	Cần tài nguyên để chứa các lõi đồng thời.
Chi phí chuyển đổi	Bao gồm quá trình tái cấu hình và khởi tạo.	Phụ thuộc logic lựa chọn và điều khiển.
Yêu cầu thiết kế	RP, Pblock và giao diện tương thích giữa các RM.	Logic lựa chọn, phân phối dữ liệu và điều khiển các lõi.

Bảng trên chỉ thể hiện những khác biệt về nguyên lý. Hiệu quả của hai phương pháp phụ thuộc vào kiến trúc các lõi, số lượng biến thể và điều kiện triển khai thực tế.

Đặc biệt, để so sánh DPR và Replica cần phân biệt thời gian chuyển đổi biến thể với thời gian mã hóa AES. Ngoài ra, số tài nguyên sử dụng, Timing và công suất cũng là những chỉ tiêu cần được xem xét.

2.7.4 Nguyên lý lựa chọn biến thể giả ngẫu nhiên

Đối với hệ thống có nhiều biến thể phần cứng, trình tự lựa chọn có thể được xác định trước hoặc sinh ra bằng một thuật toán giả ngẫu nhiên (Pseudorandom Selection).

Trong mô hình giả ngẫu nhiên, một Pseudorandom Number Generator (PRNG) tạo ra chuỗi giá trị dựa trên trạng thái nội bộ và giá trị khởi tạo (Seed). Chuỗi đầu ra sau đó được ánh xạ thành chỉ số của biến thể cần lựa chọn. Giả sử tập hợp các biến thể được ký hiệu:

$$\mathcal{V} = \{V_{\text{LUT}}, V_{\text{BRAM}}, V_{\text{Composite}}, V_{\text{Noise}}\}.$$

Tại lần lựa chọn thứ t, thuật toán xác định một phần tử $v_t \in \mathcal{V}$. Nếu hệ thống yêu cầu không lựa chọn lại biến thể đang hoạt động, tập hợp ứng viên được xác định bởi:

$$\mathcal{V}_t = \mathcal{V} \setminus \{v_{t-1}\},$$

trong đó v_{t-1} là biến thể được sử dụng ở lần lựa chọn trước đó.

Cơ chế này giúp tạo ra chuỗi chuyển đổi giữa các biến thể khác nhau, phù hợp để kiểm tra khả năng hoạt động của hệ thống khi thứ tự chuyển đổi không được cố định. Việc sử dụng một Seed xác định cho phép tái tạo cùng chuỗi giả ngẫu nhiên khi thuật toán và môi trường thực thi được giữ nguyên. Điều này hữu ích cho quá trình Debug và kiểm chứng lặp lại.

Ngoài ra, việc thay đổi biến thể theo một chuỗi giả ngẫu nhiên không tự động chứng minh khả năng tăng cường bảo mật của hệ thống mã hóa. Đối với DPR, mỗi lần lựa chọn một biến thể khác có thể dẫn đến quá trình nạp Partial Bitstream tương ứng. Trong kiến trúc Replica, việc thay đổi lựa chọn có thể được thực hiện thông qua logic điều khiển mà không cần cấu hình lại FPGA.

CHƯƠNG 3: THIẾT KẾ HỆ THỐNG MÃ HÓA AES-256

3.1 Kiến trúc tổng quan hệ thống

Hệ thống AES-256 được thiết kế trên nền tảng AMD Kria KV260, kết hợp khả năng điều khiển của Processing System (PS) với tài nguyên phần cứng trong Programmable Logic (PL). Mục tiêu của thiết kế là thực hiện mã hóa AES-256, đồng thời hỗ trợ triển khai và chuyển đổi giữa bốn biến thể phần cứng gồm LUT, BRAM, Composite Field và Dummy Noise.

Kiến trúc hệ thống được tổ chức theo hướng tách biệt phần mềm điều khiển, giao tiếp dữ liệu và lõi mã hóa. Chương trình C chạy trên ARM/Linux đảm nhiệm việc cung cấp dữ liệu, phát lệnh và đọc kết quả. Trong khi đó, các phép biến đổi của thuật toán AES-256 được thực hiện trực tiếp bằng phần cứng FPGA.

3.1.1 Yêu cầu thiết kế

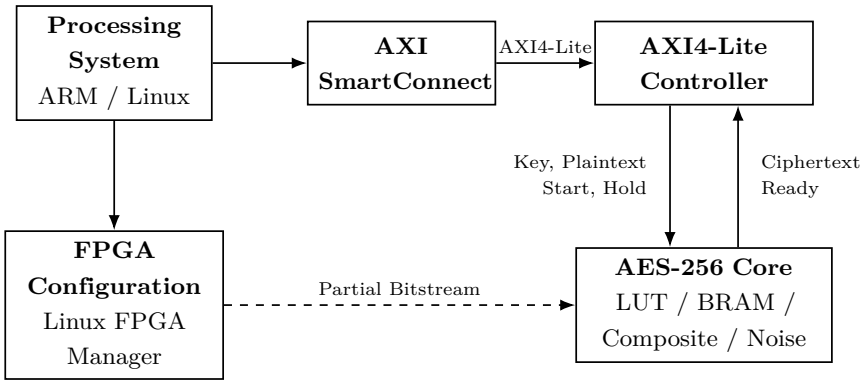
Hệ thống được xây dựng dựa trên ba yêu cầu chính: bảo đảm chức năng mã hóa AES-256, hỗ trợ giao tiếp PS–PL và duy trì khả năng thay đổi biến thể phần cứng.

Trước hết, AES Core phải tiếp nhận Encryption Key 256 bit và Plaintext 128 bit, thực hiện 14 Round theo chuẩn FIPS 197 và tạo ra Ciphertext 128 bit. Các biến thể phần cứng có thể khác nhau về cấu trúc nội bộ nhưng phải tạo ra cùng kết quả mã hóa đối với một bộ dữ liệu đầu vào.

Bên cạnh đó, hệ thống cần có giao diện cho phép phần mềm trên PS ghi dữ liệu, khởi tạo quá trình mã hóa và nhận kết quả từ PL. Đồng thời, các biến thể phải sử dụng giao diện chức năng tương thích để có thể triển khai trong kiến trúc DPR hoặc Replica mà không cần thay đổi toàn bộ hệ thống điều khiển.

3.1.2 Kiến trúc PS–PL

Kiến trúc tổng quan của hệ thống được trình bày trong Hình 10. Trong đó, PS thực thi chương trình C trên Linux, còn PL chứa các khối AXI SmartConnect, AXI4-Lite Controller và AES-256 Core.



Hình 10: Kiến trúc tổng quan hệ thống AES-256 trên nền tảng Kria KV260.

Trong đường giao tiếp dữ liệu, PS truy cập AXI4-Lite Controller thông qua AXI SmartConnect. Controller đóng vai trò trung gian, chuyển các giao dịch đọc/ghi thanh ghi thành những tín

hiệu điều khiển và dữ liệu phù hợp với AES Core. Nhờ đó, các biến thể AES-256 không cần trực tiếp xử lý giao thức AXI4-Lite.

Đối với cơ chế DPR, AES Core được tổ chức thành Reconfigurable Module (RM), trong khi AXI Controller và các khối giao tiếp chung được duy trì trong Static Design. Việc nạp Partial Bitstream được quản lý từ phía Linux thông qua cơ chế cấu hình FPGA, tách biệt với đường giao tiếp dữ liệu AXI4-Lite. Hình trên biểu diễn các đường chức năng ở mức hệ thống, không mô tả chi tiết phần cứng cấu hình bên trong SoC.

3.1.3 Nguyên lý hoạt động tổng thể

Quá trình mã hóa bắt đầu khi chương trình C trên PS ghi Encryption Key và Plaintext vào các thanh ghi AXI4-Lite Controller. Do giao diện dữ liệu sử dụng bus 32 bit, Encryption Key được chia thành tám word và Plaintext được chia thành bốn word trước khi truyền đến phần cứng.

Sau khi dữ liệu được thiết lập, phần mềm ghi lệnh Start để yêu cầu AES Core bắt đầu mã hóa. Controller phát tín hiệu điều khiển tương ứng, trong khi AES Core thực hiện các phép biến đổi và Key Expansion theo kiến trúc của biến thể đang hoạt động. Khi có kết quả hợp lệ, Controller lưu Ciphertext và cập nhật trạng thái để chương trình C có thể đọc lại thông qua AXI4-Lite.

Đối với DPR, quá trình thay đổi biến thể được thực hiện bằng cách nạp Partial Bitstream tương ứng vào vùng tái cấu hình. Trong kiến trúc Replica, các biến thể cùng tồn tại trên FPGA và được lựa chọn bằng logic điều khiển. Hai phương án này sử dụng chung nguyên tắc trao đổi dữ liệu với phần mềm nhưng khác nhau về tổ chức phần cứng và chi phí chuyển đổi.

3.2 Thiết kế chi tiết lõi mã hóa AES-256

Lõi mã hóa AES-256 được xây dựng bằng SystemVerilog theo kiến trúc xử lý lặp (Iterative Architecture), trong đó cùng một Datapath được sử dụng qua nhiều Round thay vì triển khai toàn bộ 14 Round thành các tầng phần cứng riêng biệt. Phương án này cho phép tái sử dụng các khối biến đổi dữ liệu và phối hợp trực tiếp với quá trình sinh Round Key.

Trong thiết kế của nhóm, kiến trúc LUT được sử dụng làm cơ sở để phân tích tổ chức Cipher Datapath, Key Expansion và logic điều khiển. Các biến thể Composite Field và Dummy Noise kế thừa cách xử lý lặp, trong khi biến thể BRAM có sự điều chỉnh về trình tự xử lý để đáp ứng cơ chế đọc S-box đồng bộ. Những khác biệt này sẽ được phân tích riêng trong Mục 3.3.

3.2.1 Cấu trúc tổng thể AES-256 Core

Trong mã nguồn `aes256_lut.sv`, module `aes256_lut` đóng vai trò Top Wrapper của lõi mã hóa. Module này tiếp nhận Encryption Key 256 bit, Plaintext 128 bit, Clock, Reset và tín hiệu yêu cầu mã hóa `cipher_en`. Hai tín hiệu đầu ra gồm `cipher_text` 128 bit và `cipher_ready`, dùng để cung cấp kết quả và trạng thái hoạt động cho khối điều khiển bên ngoài. Bên trong Top Wrapper, thiết kế được chia thành hai module chức năng chính:

- **AES-256 Cipher Core:** Thực hiện các phép biến đổi trên State 128 bit, quản lý State Register, theo dõi số Round và cung cấp trạng thái hoàn tất quá trình mã hóa.

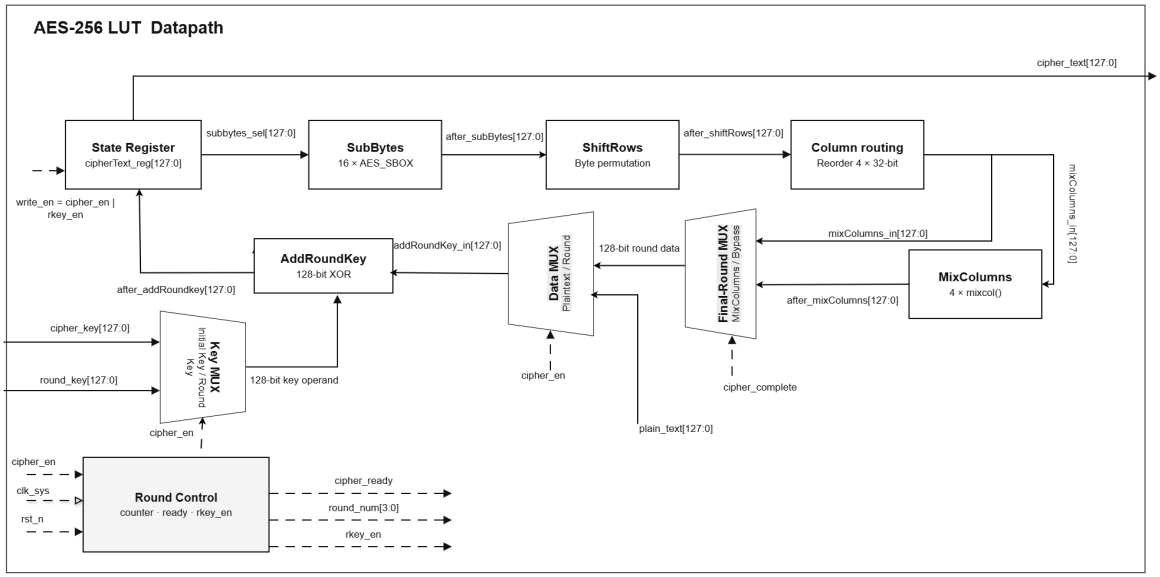
- **AES-256 Key Expansion:** Tiếp nhận Encryption Key 256 bit, lưu trữ các word khóa cần thiết và sinh Round Key 128 bit theo trình tự xử lý của Cipher Core.

Hai module được đồng bộ thông qua các tín hiệu `round_num` và `rkey_en`. Cipher Core cung cấp chỉ số Round hiện tại và tín hiệu cho phép cập nhật Key Schedule, còn Key Expansion trả về `round_key_out` có độ rộng 128 bit.

Ở bước khởi tạo, Cipher Core sử dụng 128 bit đầu tiên của Encryption Key để thực hiện Initial AddRoundKey. Sau đó, các Round Key tiếp theo được cung cấp bởi Key Expansion, bảo đảm mỗi Round sử dụng đúng phần khóa tương ứng. Việc tách Cipher Core và Key Expansion thành hai module giúp phân chia rõ chức năng xử lý dữ liệu và sinh khóa. Đồng thời, Top Wrapper duy trì giao diện đầu vào và đầu ra thống nhất, thuận tiện cho việc tích hợp các biến thể AES-256 vào hệ thống DPR hoặc Replica.

3.2.2 Thiết kế Cipher Datapath

Cipher Datapath của biến thể LUT được tổ chức theo kiến trúc Rolling, sử dụng một State Register 128 bit để lưu trữ dữ liệu trung gian. Sau mỗi Round, kết quả được ghi trở lại thanh ghi này và tiếp tục đi qua các khối biến đổi ở Round kế tiếp. Cấu trúc Datapath được trình bày trong Hình 11.



Hình 11: Kiến trúc Cipher Datapath của AES-256 LUT

Datapath gồm các khối SubBytes, ShiftRows, MixColumns, AddRoundKey và logic lựa chọn dữ liệu. Khác với kiến trúc Fully Unrolled, thiết kế không nhân bản toàn bộ chuỗi biến đổi cho 14 Round. Thay vào đó, State được cập nhật sau mỗi cạnh Clock và quay lại Datapath để thực hiện Round tiếp theo.

State Register và đường dữ liệu hồi tiếp

Trong RTL, tín hiệu `cipherText_reg` được sử dụng làm State Register 128 bit. Đây là thanh ghi lưu kết quả của Initial AddRoundKey và các Round tiếp theo, đồng thời được nối trực tiếp đến đầu ra `cipher_text`. Tại thời điểm nhận lệnh bắt đầu, dữ liệu ghi vào State Register được xác định bằng phép XOR giữa Plaintext và Round Key khởi tạo:

$$S_0 = P \oplus K^{(0)},$$

trong đó P là Plaintext 128 bit và $K^{(0)}$ là 128 bit đầu tiên của Encryption Key.

Sau bước khởi tạo, dữ liệu được lấy từ State Register, đưa qua các phép biến đổi và ghi kết quả trở lại cùng thanh ghi. Cơ chế này tạo thành đường hồi tiếp cho toàn bộ quá trình mã hóa.

Trong RTL, việc lựa chọn giữa Plaintext ban đầu và dữ liệu của các Round được điều khiển bằng tín hiệu `cipher_en`. Khi `cipher_en` được kích hoạt, Datapath sử dụng Plaintext và Initial Round Key. Trong các chu kỳ xử lý tiếp theo, đầu vào phép AddRoundKey được lấy từ kết quả của các phép biến đổi trên State hiện tại.

Khối SubBytes

Khối SubBytes của biến thể LUT sử dụng module `aes_sbox_lut_rom`, trong đó bảng S-box được mô tả dưới dạng ROM gồm 256 phần tử, mỗi phần tử có độ rộng 8 bit. Thiết kế tạo 16 instance S-box thông qua cấu trúc `generate`, tương ứng với 16 byte của State 128 bit. Nhờ đó, toàn bộ các phép thay thế byte có thể được thực hiện song song trong Datapath tổ hợp, không cần chia State thành những nhóm byte để tra cứu tuần tự.

Trong mã nguồn, tín hiệu `subbytes_sel` thực hiện sắp xếp lại các byte đầu vào cho các instance S-box. Sau khi tra bảng, các byte đầu ra được tập hợp trong tín hiệu `after_subBytes`. Thuộc tính `rom_style = "distributed"` được sử dụng để định hướng Vivado triển khai các bảng tra bằng tài nguyên Distributed ROM. Tuy nhiên, số LUT vật lý cuối cùng vẫn cần được xác định từ báo cáo Synthesis.

Khối ShiftRows

ShiftRows được hiện thực bằng các phép gán tổ hợp để thay đổi vị trí của các byte. Khối này không cần thực hiện phép toán số học và không sử dụng một State Register riêng. Do cách bố trí byte trong RTL, các phép gán tại `subbytes_sel`, `after_shiftRows` và `mixColumns_in` phối hợp với nhau để đưa dữ liệu về đúng thứ tự xử lý theo cột của AES. Vì vậy, khi phân tích sơ đồ ở mức RTL cần xét toàn bộ đường định tuyến byte thay vì chỉ một nhóm phép gán riêng lẻ. Cách tổ chức bằng kết nối tổ hợp giúp thực hiện thao tác hoán vị mà không cần bổ sung một chu kỳ xử lý dành riêng cho ShiftRows.

Khối MixColumns

Trong thiết kế, State sau ShiftRows được chia thành bốn nhóm 32 bit tương ứng với bốn cột của ma trận State. Mỗi cột được xử lý bằng hàm `mixcol`, sử dụng các hàm `mul2` và `mul3` để thực hiện phép nhân trong $GF(2^8)$. Bốn phép tính MixColumns được mô tả độc lập và có thể được tổng hợp thành các mạng logic hoạt động song song. Kết quả được ghép thành

bus `after_mixColumns` 128 bit để đưa đến phép AddRoundKey. Đối với Round cuối cùng, AES-256 không thực hiện MixColumns [1]. Vì vậy, Datapath có logic lựa chọn dựa trên tín hiệu `cipher_complete`. Khi bộ đếm đạt giá trị 14, dữ liệu được lấy từ `mixColumns_in`, tức State đã được sắp xếp đúng thứ tự sau ShiftRows, và bỏ qua kết quả MixColumns.

Khối AddRoundKey

AddRoundKey được thực hiện bằng phép XOR giữa dữ liệu State và Round Key 128 bit. Trong RTL, tín hiệu `addRoundKey_in` lựa chọn dữ liệu đầu vào, sau đó kết quả được xác định bởi:

$$\texttt{after_addRoundkey} = \texttt{addRoundKey_in} \oplus K,$$

với K là Initial Round Key khi bắt đầu mã hóa hoặc Round Key do Key Expansion cung cấp trong các Round tiếp theo. Sau khi tính toán, kết quả được ghi vào `cipherText_reg` tại cạnh lên của Clock nếu tín hiệu `cipher_en` hoặc `rkey_en` được kích hoạt. Như vậy, đối với biến thể LUT, các phép biến đổi của một Round được tổ chức thành đường logic tổ hợp nằm giữa hai lần cập nhật State Register. Thiết kế ưu tiên tái sử dụng Datapath theo thời gian, thay vì nhân bản toàn bộ phần cứng cho từng Round. Có thể biểu diễn hoạt động cập nhật State như sau:

$$S_r = \begin{cases} \text{MixColumns}\left(\text{ShiftRows}\left(\text{SubBytes}(S_{r-1})\right)\right) \oplus K^{(r)}, & 1 \le r \le 13, \\ \text{ShiftRows}\left(\text{SubBytes}(S_{13})\right) \oplus K^{(14)}, & r = 14. \end{cases}$$

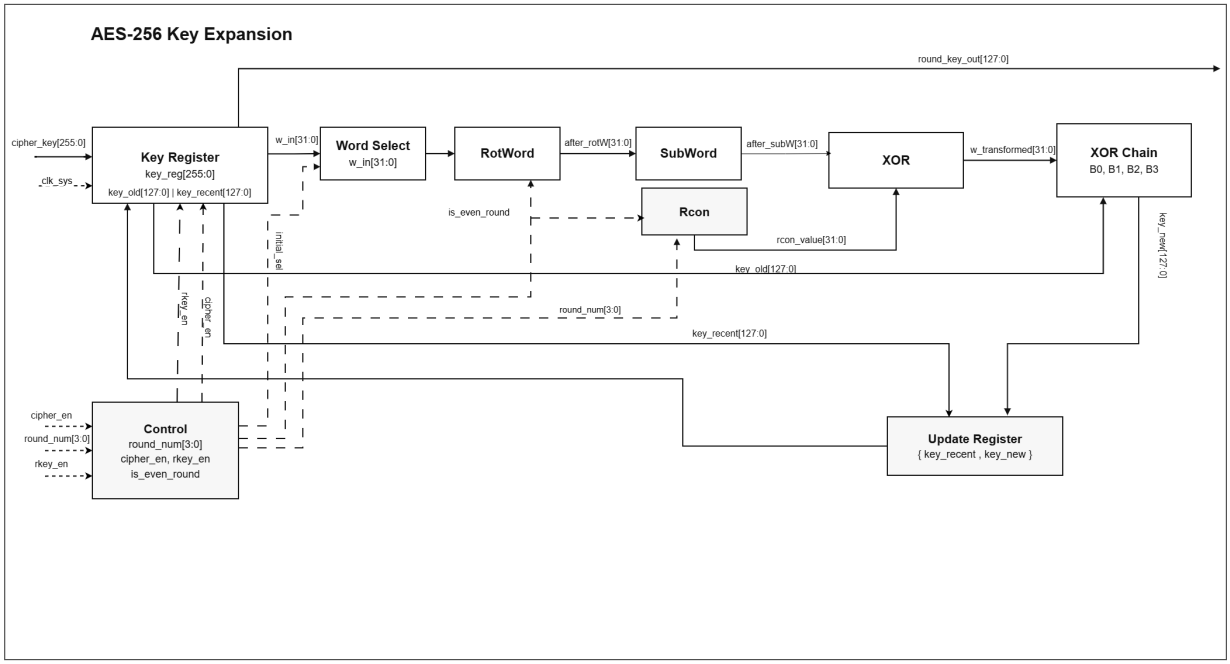
Kết quả cuối cùng S_{14} chính là Ciphertext 128 bit của phép mã hóa.

3.2.3 Thiết kế Key Expansion

Key Expansion có nhiệm vụ cung cấp Round Key 128 bit tương ứng với từng giai đoạn xử lý của Cipher Core. Đối với AES-256, quá trình sinh khóa phụ cần sử dụng tám word 32 bit gần nhất, thay vì chỉ bốn word như AES-128. Trong biến thể LUT, module `aes256_key_expansion` sử dụng thanh ghi `key_reg` 256 bit để lưu trữ một cửa sổ gồm tám word khóa. Thanh ghi được chia thành hai nhóm:

$$K_{\text{old}} = \texttt{key_reg[255:128]},$$
$$K_{\text{recent}} = \texttt{key_reg[127:0]}.$$

Trong đó, K_{old} chứa bốn word cũ hơn, còn K_{recent} chứa bốn word gần nhất. Tại thời điểm khởi tạo, hai nhóm này lần lượt tương ứng với $K^{(0)}$ và $K^{(1)}$ của Encryption Key ban đầu. Hình 12 trình bày cấu trúc sinh khóa được xây dựng từ RTL.



Hình 12: Cấu trúc Key Expansion sử dụng cửa sổ khóa 256 bit

Tổ chức cửa sổ khóa 256 bit

Khi `cipher_en` được kích hoạt, toàn bộ Encryption Key được nạp vào `key_reg`. Sau đó, tại mỗi lần cập nhật khóa, nhóm 128 bit gần nhất được dịch sang nửa trên của thanh ghi, còn nhóm 128 bit mới sinh được đưa vào nửa dưới:

$$K_{\text{window}}^{\text{next}} = \{K_{\text{recent}}, K_{\text{new}}\}.$$

Trong RTL, quá trình cập nhật được điều khiển bởi tín hiệu `rkey_en`. Cách tổ chức này giúp Key Expansion chỉ cần lưu tám word gần nhất để tạo khóa cho những Round tiếp theo, thay vì lưu toàn bộ 60 word của Key Schedule trong các thanh ghi riêng biệt.

RotWord, SubWord và Rcon

Tín hiệu `w_in` được lấy từ 32 bit cuối cùng của K_{recent}. Word này được đưa qua khối RotWord và SubWord để tạo giá trị trung gian phục vụ sinh bốn word mới. Đối với AES-256, cách biến đổi phụ thuộc vào vị trí của các word cần sinh. Trong RTL, tín hiệu `is_even_round` được xác định từ bit thấp nhất của `round_num`. Với quy ước Round của thiết kế, các Round lẻ sử dụng RotWord, SubWord và Rcon để chuẩn bị nhóm word mới, còn các Round chẵn chỉ sử dụng SubWord. Giá trị trung gian được mô tả:

$$T_r = \begin{cases} \text{SubWord}\left(\text{RotWord}(W_r)\right) \oplus \text{Rcon}\left[\dfrac{r+1}{2}\right], & r \text{ lẻ}, \\ \text{SubWord}(W_r), & r \text{ chẵn}, \end{cases}$$

trong đó W_r là word 32 bit cuối cùng của K_{recent} tại Round r.

Khối SubWord của biến thể LUT sử dụng bốn instance `aes_sbox_lut_rom`, cho phép thay thế đồng thời bốn byte của word đầu vào. Kết quả được lưu trên bus `after_subW` 32 bit. Rcon được cung cấp thông qua hàm `aes256_rcon`. Trong AES-256, nhánh sử dụng Rcon xuất hiện tại những lần sinh word tương ứng với chỉ số bội số của tám trong Key Schedule.

Mạch Cascade XOR

Sau khi tạo được T_r, bốn word mới được tính bằng một chuỗi XOR liên tiếp. Gọi A_0, A_1, A_2, A_3 là bốn word thuộc K_{old}, còn B_0, B_1, B_2, B_3 là những word mới, ta có:

$$\begin{aligned} B_0 &= A_0 \oplus T_r, \\ B_1 &= A_1 \oplus B_0, \\ B_2 &= A_2 \oplus B_1, \\ B_3 &= A_3 \oplus B_2. \end{aligned}$$

Bốn word này được ghép thành một Round Key mới: $K_{\text{new}} = B_0 \parallel B_1 \parallel B_2 \parallel B_3$. Đặc điểm của Cascade XOR là các word phía sau phụ thuộc vào kết quả của word phía trước. Do đó, đường logic sinh B_3 có thể đi qua nhiều tầng XOR hơn so với đường sinh B_0. Đây là một trong những đường tổ hợp cần được xem xét khi phân tích Timing, mặc dù chưa thể kết luận đó là Critical Path nếu chưa có báo cáo STA.

Đồng bộ Key Expansion với Cipher Core

Trong quá trình mã hóa, `round_key_out` được lấy từ K_{recent}. Sau khi Cipher Core hoàn tất cập nhật State cho một Round, cửa sổ khóa đồng thời được cập nhật để cung cấp Round Key cho Round kế tiếp. Ví dụ, sau Initial AddRoundKey, thanh ghi chứa hai nhóm Round Key $K^{(0)}$ và $K^{(1)}$. Tại Round 1, Cipher Core sử dụng $K^{(1)}$, trong khi Key Expansion tạo $K^{(2)}$ để chuẩn bị cho Round 2. Quá trình này tiếp tục cho đến Round cuối cùng. Việc phối hợp cập nhật State Register và Key Register trên cùng cạnh Clock giúp tránh phải lưu riêng toàn bộ Key Schedule. Tuy nhiên, hai module phải sử dụng nhất quán chỉ số Round và tín hiệu điều khiển; nếu Key Expansion cập nhật sai thời điểm, Cipher Core có thể sử dụng Round Key không tương ứng với State đang xử lý.

3.2.4 Thiết kế logic điều khiển Round

Logic điều khiển có nhiệm vụ quản lý quá trình bắt đầu mã hóa, cập nhật State, sinh Round Key và xác định thời điểm hoàn tất. Trong biến thể LUT, phần điều khiển được hiện thực chủ yếu bằng bộ đếm Round và các điều kiện logic, thay vì một FSM khai báo tường minh bằng nhiều trạng thái.

Cụ thể, thiết kế sử dụng tín hiệu `cipher_counter` có độ rộng 4 bit để theo dõi tiến trình mã hóa. Giá trị bộ đếm được đưa ra qua `round_num`, đồng thời được sử dụng để lựa chọn phép biến đổi và Rcon trong Key Expansion.

Điều kiện kết thúc được xác định bằng:

$$\texttt{cipher_complete} = (\texttt{cipher_counter} = 14)\,.$$

Khi nhận `cipher_en`, trạng thái `cipher_ready` được đưa về 0, cho biết lõi đang thực hiện mã hóa. Trong quá trình hoạt động, bộ đếm tiếp tục tăng và State Register được cập nhật qua các Round. Khi bộ đếm đạt 14, logic điều khiển lựa chọn đường dữ liệu bỏ qua MixColumns và cập nhật trạng thái hoàn tất. Tín hiệu cho phép Key Expansion được xác định bởi:

$$\texttt{rkey_en} = \neg\texttt{cipher_ready}.$$

Do đó, trong giai đoạn lõi đang xử lý, tín hiệu `rkey_en` được kích hoạt để Key Expansion cập nhật cửa sổ khóa cùng với tiến trình của Cipher Core. Có thể phân chia hoạt động điều khiển thành các giai đoạn chức năng trong Bảng 8. Các giai đoạn này dùng để diễn giải hành vi của bộ đếm và tín hiệu điều khiển, không phải tên các State được khai báo trực tiếp trong RTL LUT.

Bảng 8: Các giai đoạn điều khiển của lõi AES-256 LUT.

Giai đoạn	**Điều kiện**	**Hoạt động**
Chờ lệnh	`cipher_ready=1`	Giữ trạng thái chờ, tiếp nhận yêu cầu Start.
Khởi tạo	`cipher_en=1`	Thực hiện Initial AddRoundKey, nạp Encryption Key và đưa bộ đếm sang Round 1.
Xử lý Round	`round_num=1..13`	Thực hiện SubBytes, ShiftRows, MixColumns, AddRoundKey và cập nhật Key Expansion.
Round cuối	`round_num=14`	Bỏ qua MixColumns, cập nhật Ciphertext và xác lập trạng thái hoàn tất.
Hoàn tất	`cipher_ready=1`	Giữ kết quả trong State Register cho đến khi nhận tác vụ mới.

`cipher_ready` là tín hiệu trạng thái mức logic, không phải xung Done chỉ xuất hiện trong một chu kỳ. Theo RTL, tín hiệu này ở mức 1 khi lõi không xử lý tác vụ và được đưa về 1 khi quá trình mã hóa kết thúc. Vì vậy, khối điều khiển bên ngoài phải phân biệt trạng thái sẵn sàng ban đầu với việc hoàn tất một lệnh mã hóa mới. Ngoài ra, lõi LUT được thiết kế để nhận một xung Start khi đang ở trạng thái chờ. RTL không có một cơ chế hàng đợi lệnh độc lập; vì vậy, việc không phát yêu cầu mã hóa mới trong lúc Core đang xử lý là một điều kiện cần được bảo đảm ở cấp điều khiển hệ thống.

Đối với biến thể BRAM, logic điều khiển có cấu trúc khác do kết quả đọc S-box chỉ sẵn sàng theo xung Clock. RTL BRAM khai báo ba trạng thái `ST_IDLE`, `ST_SB_READ` và `ST_ROUND_UPDATE`. Phân tích chi tiết FSM này sẽ được trình bày trong mục thiết kế biến thể BRAM để làm rõ sự khác biệt về Latency.

3.2.5 Trình tự xử lý và đồng bộ tín hiệu

Trong kiến trúc LUT, Cipher Datapath và Key Expansion hoạt động đồng bộ theo `clk_sys`. Các phép biến đổi SubBytes, ShiftRows, MixColumns và AddRoundKey được mô tả dưới dạng logic tổ hợp; State Register và Key Register nhận dữ liệu mới tại cạnh lên của Clock theo điều kiện cho phép. Khi tín hiệu Start được chấp nhận, Initial AddRoundKey được thực hiện và kết quả được ghi vào State Register. Tại cùng cạnh Clock, Encryption Key 256 bit được nạp vào `key_reg`, còn bộ đếm được thiết lập để bắt đầu Round 1. Ở các cạnh Clock tiếp theo, State được cập nhật theo kết quả của Round hiện tại. Song song với đó, Key Expansion tính toán và

cập nhật cửa sổ khóa, chuẩn bị Round Key cho lần xử lý kế tiếp. Bảng 9 tóm tắt trình tự cập nhật của lõi LUT, với quy ước E_0 là cạnh Clock chấp nhận lệnh Start.

Bảng 9: Trình tự cập nhật State và Round Key của AES-256 LUT.

Cạnh Clock	Giai đoạn	Cập nhật State	Ready sau cạnh
E_0	Khởi tạo	$S_0 = P \oplus K^{(0)}$	0
E_1	Round 1	S_1 sử dụng $K^{(1)}$	0
E_2	Round 2	S_2 sử dụng $K^{(2)}$	0
⋮	⋮	⋮	⋮
E_{13}	Round 13	S_{13} sử dụng $K^{(13)}$	0
E_{14}	Round 14	$S_{14} = C$, sử dụng $K^{(14)}$	1

Theo trình tự trên, lõi LUT thực hiện một lần cập nhật State cho Initial AddRoundKey và 14 lần cập nhật cho các Round mã hóa, tổng cộng 15 lần cập nhật tại các cạnh Clock. Tính từ cạnh nhận Start đến cạnh xác lập Ciphertext cuối cùng, có 14 khoảng chu kỳ Clock giữa hai mốc này. Khi đo Latency, cần nêu rõ quy ước tính chu kỳ và thời điểm lấy mẫu tín hiệu để tránh nhầm lẫn giữa số lần cập nhật State và thời gian hoàn tất tác vụ. Đặc biệt, trong RTL LUT, State Register không có nhánh Reset riêng; dữ liệu thanh ghi được ghi đè khi một lệnh mã hóa được chấp nhận. Vì vậy, giá trị `cipher_text` trước khi thực hiện tác vụ đầu tiên không được xem là Ciphertext hợp lệ. Tương tự, Key Register được khởi tạo bằng Encryption Key tại thời điểm Start. Khi tích hợp với AXI4-Lite Controller, phần mềm không trực tiếp lấy mẫu các giá trị State trung gian. Thay vào đó, Controller sử dụng tín hiệu trạng thái để xác định thời điểm lưu Ciphertext cuối cùng và cung cấp kết quả qua các thanh ghi Memory-Mapped. Cách tổ chức này tách thời gian xử lý bên trong AES Core khỏi thời gian giao tiếp giữa PS và PL.

Những vấn đề cần xem xét về Timing

Do kiến trúc LUT sử dụng Datapath tổ hợp giữa các lần cập nhật State Register, độ trễ của các khối SubBytes, MixColumns, AddRoundKey và mạng định tuyến có thể ảnh hưởng đến chu kỳ Clock nhỏ nhất mà thiết kế đáp ứng được. Đối với Key Expansion, đường tính toán từ S-box qua các tầng Cascade XOR cũng cần được xem xét. Tuy nhiên, chỉ dựa trên cấu trúc RTL chưa thể xác định chắc chắn đường nào là Critical Path, bởi kết quả còn phụ thuộc vào Synthesis, Placement, Routing và các Timing Constraints. Vì vậy, phần thiết kế này chỉ xác định những đường xử lý cần quan tâm. Việc đánh giá Worst Negative Slack (WNS), Maximum Clock Frequency ($F_{\max}$), Latency và Throughput sẽ dựa trên kết quả thực nghiệm trong các chương tiếp theo.

3.3 Thiết kế các biến thể phần cứng AES-256

Từ kiến trúc AES-256 Core trình bày ở Mục 3.2, nhóm xây dựng bốn biến thể phần cứng gồm LUT S-box, BRAM S-box, Composite Field và Dummy Noise. Các biến thể cùng nhận khóa mã hóa 256 bit, Plaintext 128 bit và tín hiệu bắt đầu `cipher_en`; kết quả được trả qua `cipher_text[127:0]` và tín hiệu trạng thái `cipher_ready`. Sự khác biệt chủ yếu nằm ở phương pháp

hiện thực S-box, cách tổ chức chu kỳ xử lý hoặc sự bổ sung logic hoạt động song song với lõi mã hóa. Việc duy trì giao diện chung tạo điều kiện sử dụng cùng một lớp điều khiển ở các kiến trúc DPR và Replica, sẽ được trình bày trong Mục 3.5.

Nội dung sau tập trung vào những thay đổi được thể hiện trong RTL của từng biến thể. Các phép biến đổi AES cơ bản và cơ chế Rolling Datapath đã được giải thích ở Chương 2 và Mục 3.2 nên không trình bày lại. Tương tự, thông số tài nguyên, thời gian thực thi đo trên KV260 và kết quả kiểm chứng được dành cho Chương 5 và Chương 6.

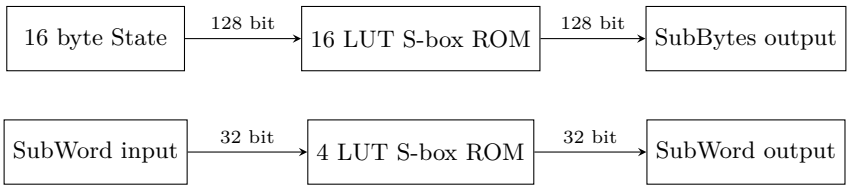
3.3.1 Biến thể LUT S-box

Biến thể LUT được hiện thực trong tệp `aes256_lut.sv`, gồm hai khối chính `aes256_cipher_core` và `aes256_key_expansion`.

Đây là biến thể tham chiếu khi so sánh với các phương án còn lại. Điểm đặc trưng của thiết kế là S-box được mô tả bằng bảng tra 256 phần tử, mỗi phần tử rộng 8 bit, với đầu ra tổ hợp. Module `aes_sbox_lut_rom` sử dụng mảng `sbox_rom[0:255]` và phép gán `dout = sbox_rom[addr]`.

Thuộc tính `rom_style = "distributed"` được đặt trên mảng ROM để hướng công cụ Vivado suy diễn bảng tra trên tài nguyên LUT.

Trong Cipher Datapath, 128 bit State được chia thành 16 byte và nối tới 16 thể hiện S-box hoạt động song song. Mỗi byte địa chỉ tạo ra một byte sau SubBytes; các đầu ra được ghép thành bus `after_subBytes[127:0]`. Key Expansion sử dụng thêm bốn S-box cùng loại để tính `after_subW[31:0]`. Như vậy, trong mô tả RTL tồn tại 20 vị trí tra cứu S-box gồm 16 vị trí cho State và bốn vị trí cho khóa. Con số này phản ánh số thể hiện logic ở mức RTL, không mặc nhiên bằng số lượng khối nhớ vật lý sau tổng hợp.



Hình 13: Tổ chức các S-box dạng LUT trong Cipher Datapath và Key Expansion.

Việc tra cứu theo logic tổ hợp cho phép đầu ra S-box phụ thuộc trực tiếp vào byte địa chỉ trong cùng chu kỳ, không cần trạng thái chờ đọc bộ nhớ. Điều này phù hợp với cơ chế cập nhật State theo từng Round của biến thể LUT ở Mục 3.2. Tuy nhiên, đường truyền dữ liệu của một Round vẫn phải đi qua S-box và các phép biến đổi phía sau trước khi được chốt tại cạnh Clock; do đó, thời gian đường tổ hợp và số LUT thực tế cần được xác định bằng kết quả tổng hợp và phân tích Timing, thay vì suy ra chỉ từ cách mô tả ROM.

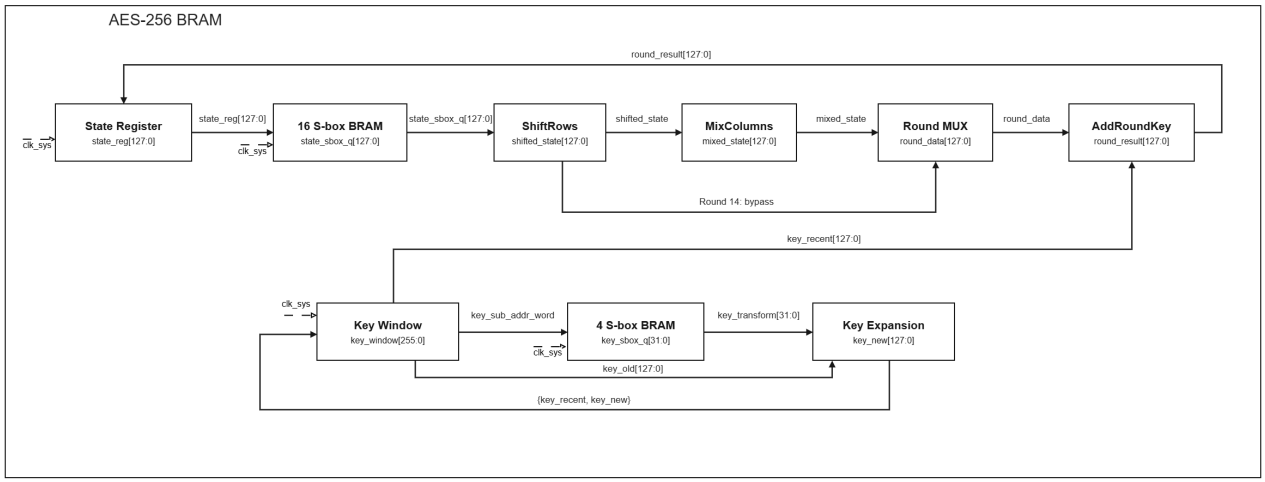
3.3.2 Biến thể BRAM S-box

Biến thể BRAM trong `aes256_bram.sv` thay đổi cả cách truy xuất S-box lẫn trình tự điều khiển. Module `aes_sbox_bram` vẫn lưu bảng thay thế 256 byte nhưng đầu ra được cập nhật tại

cạnh lên của Clock thông qua phép gán `q <= rom[addr]`. Như vậy, dữ liệu đọc không còn được sử dụng tức thời theo địa chỉ như ở biến thể LUT. Thiết kế phải dành một pha để ghi nhận kết quả tra cứu trước khi cập nhật State.

Trong Cipher Core có 16 S-box BRAM hoạt động song song. Chúng nhận các byte từ `state_reg[127:0]` và tạo bus `state_sbox_q[127:0]`.

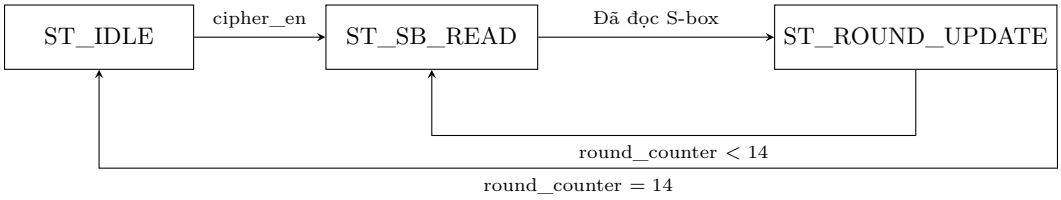
Bốn thể hiện khác phục vụ Key Expansion, với đầu vào 32 bit `key_sub_addr_word` và đầu ra `key_sbox_q[31:0]`. Hai nhóm tra cứu dùng chung Clock `clk_sys` nên kết quả S-box của State và Key Expansion được ghi nhận trong cùng pha đọc. Các phép ShiftRows, MixColumns, chọn nhánh Round cuối và AddRoundKey tiếp tục được triển khai bằng logic tổ hợp sau đầu ra S-box đã chốt.



Hình 14: Cấu trúc dữ liệu của biến thể BRAM, gồm các S-box đọc đồng bộ cho State và Key Expansion.

Khác với biến thể LUT, `aes256_bram` tổ chức State và cửa sổ khóa ngay trong cùng một module, lần lượt qua các thanh ghi `state_reg[127:0]` và `key_window[255:0]`. Khi nhận `cipher_en` ở trạng thái nghỉ, module thực hiện Initial AddRoundKey bằng `plain_text` $\oplus$ `cipher_key[255:128]`, đồng thời nạp khóa 256 bit vào `key_window`. Trong các Round tiếp theo, `key_recent = key_window[127:0]` được sử dụng làm Round Key, còn `key_new` được tính bằng chuỗi XOR từ `key_old` và `key_transform`. Cửa sổ khóa chỉ dịch sau khi hoàn thành một Round chưa phải Round cuối, tránh cập nhật khóa trong pha truy xuất S-box.

Để đồng bộ các thao tác trên, thiết kế dùng FSM ba trạng thái `ST_IDLE`, `ST_SB_READ` và `ST_ROUND_UPDATE`. Sơ đồ chuyển trạng thái ở Hình 15 được rút trực tiếp từ khối điều khiển trong RTL.



Hình 15: Trình tự điều khiển ba trạng thái của biến thể BRAM.

Ở ST_IDLE, bộ điều khiển chờ lệnh Start và nạp State, khóa ban đầu. Tại ST_SB_READ, các S-box nhận địa chỉ từ thanh ghi hiện tại; dữ liệu đọc được ghi vào đầu ra q tại cạnh Clock. Đến ST_ROUND_UPDATE, logic sử dụng kết quả vừa đọc để tính round_result, ghi lại state_reg và, nếu chưa đến Round 14, cập nhật key_window cùng bộ đếm Round. Khi xử lý Round 14, round_result được ghi vào State, cipher_ready được đưa lên mức 1 và FSM trở lại ST_IDLE. Điều kiện chọn dữ liệu Round cuối được thể hiện bằng:

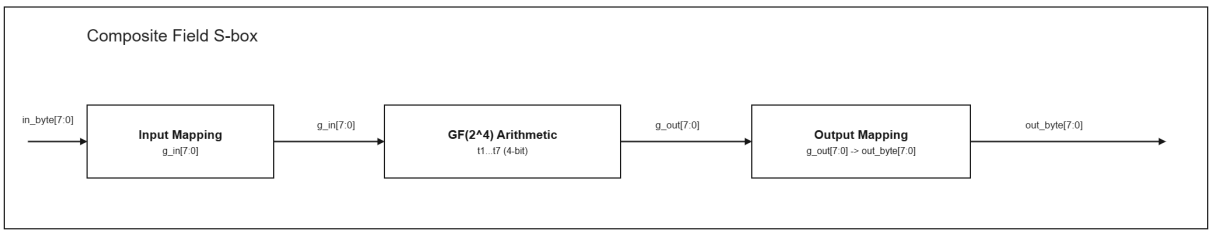
$$\texttt{round_data} = \begin{cases} \texttt{shifted_state}, & \texttt{round_counter} = 14, \\ \texttt{mixed_state}, & \texttt{round_counter} \neq 14. \end{cases}$$

Cơ chế đọc đồng bộ làm tăng số pha xử lý so với LUT, nhưng tách việc đọc S-box khỏi bước cập nhật Round, qua đó thay đổi cấu trúc đường Timing. Một điểm cần phân biệt là aes_sbox_bram hiện sử dụng thuộc tính ramstyle = "M10K" trong RTL. Thuộc tính này không phải chỉ thị ram_style = "block" thường dùng của Vivado; vì vậy không thể chỉ dựa vào tên biến thể hoặc thuộc tính RTL để kết luận các bảng tra đã được ánh xạ thành BRAM vật lý trên KV260. Việc xác nhận phải căn cứ vào báo cáo tổng hợp và sử dụng tài nguyên FPGA ở Chương 5.

3.3.3 Biến thể Composite Field

Biến thể Composite Field được mô tả trong tệp aes_composite.sv. Khác với hai phương án sử dụng bảng tra, S-box của biến thể này được hiện thực bằng mạng logic tổ hợp dựa trên các phép toán của Composite Field. Lõi mã hóa vẫn giữ 128 bit State, cấu trúc Round và cách truyền Round Key tương tự biến thể LUT; phần được thay thế trực tiếp là module aes_sbox trong Cipher Core và Key Expansion. Theo RTL hiện tại, Cipher Core tạo 16 thể hiện aes_sbox cho SubBytes; Key Expansion gọi cùng loại S-box bốn lần để xử lý after_rotW[31:0].

Cấu trúc bên trong mỗi S-box chia thành ba bước xử lý. Trước tiên, tám bit in_byte[7:0] được biến đổi qua các tổ hợp XOR để tạo g_in[7:0], tương ứng với ánh xạ từ biểu diễn byte AES sang biểu diễn Composite Field. Tiếp theo, g_in được tách thành hai nửa bốn bit và xử lý thông qua các tín hiệu trung gian t1 đến t7. Cuối cùng, hai kết quả bốn bit được ghép thành g_out[7:0]; mạng XOR/XNOR đầu ra thực hiện ánh xạ ngược kết hợp biến đổi Affine để tạo out_byte[7:0].



Hình 16: Các khối chức năng trong một S-box Composite Field.

Phần số học trường con được cấu tạo theo phân cấp. Module `GF_2_4_MUL_Nw_PolyPoly` thực hiện phép nhân trên trường bốn bit; `GF_2_4_INV_Nw_PolyPoly` thực hiện phép nghịch đảo; còn `GF_2_4_SQR_SCL_N_N2_Nw_PolyPoly` đảm nhiệm phép bình phương kết hợp nhân với hằng số. Các module này tiếp tục sử dụng những khối xử lý hai bit như `GF_2_2_MUL_Poly`, `GF_2_2_INV_Poly` và các phép biến đổi tuyến tính khác. Như vậy, thao tác thay thế byte được biểu diễn bằng những mạng XOR, AND và các tổ hợp trường hữu hạn thay vì truy xuất một mảng ROM. Cơ sở toán học của cách phân rã trường đã được trình bày ở Chương 2; tại đây, vấn đề quan tâm là quan hệ giữa các module trong RTL.

Toàn bộ module `aes_sbox` là logic tổ hợp, không chứa thanh ghi Clock bên trong. Vì vậy, biến thể Composite không cần bổ sung pha đọc S-box đồng bộ như BRAM và vẫn sử dụng cơ chế điều khiển Round tương tự biến thể LUT. Tuy nhiên, thời gian của đường tổ hợp có thể chịu ảnh hưởng từ các tầng nhân, nghịch đảo trường con và mạng XOR ở đầu ra. Không nên mặc định Composite Field sẽ nhanh hơn hay ít tốn LUT hơn phương án bảng tra: điều này phụ thuộc cách công cụ tổng hợp tối ưu cấu trúc logic trên thiết bị mục tiêu. Đánh giá định lượng được thực hiện ở phần kết quả triển khai.

3.3.4 Biến thể Dummy Noise

Biến thể Dummy Noise trong `dummy_noise.sv` không thay đổi các phép biến đổi AES chính mà bổ sung một nhánh logic có hoạt động chuyển mạch trong lúc lõi mã hóa vận hành. Wrapper `dummy_noise` vẫn kết nối Cipher Core và Key Expansion theo giao diện thông thường. Điểm khác biệt nằm trong `aes256_cipher_core_cpng`, nơi có thêm module `aes256_full_cpng` với các thanh ghi và S-box riêng. Thiết kế này được sử dụng để nghiên cứu ảnh hưởng của hoạt động logic bổ sung đối với đặc tính công suất. Nhánh CPNG nhận dữ liệu State 128 bit từ Cipher Core. Cổng đầu vào này có tên `state_before_addroundkey[127:0]`.

Dữ liệu được nối từ tín hiệu `addRoundKey_in[127:0]` của lõi AES.

Thanh ghi `kinterf_reg[127:0]` được cập nhật theo hàm LFSR. Ở mỗi cạnh lên khi `update_en` được kích hoạt, các thanh ghi được cập nhật theo các biểu thức dưới đây, trong đó C là `cpng_state_reg`, K là `kinterf_reg` và S_{preARK} là `state_before_addroundkey`:

$$C^{+} = S_{\text{preARK}} \oplus K, \tag{27}$$

$$K^{+} = \text{LFSR128}(K). \tag{28}$$

Trong đó, phép cập nhật LFSR dịch chuỗi 128 bit sang trái một vị trí và tính bit phản hồi từ bốn vị trí của trạng thái cũ:

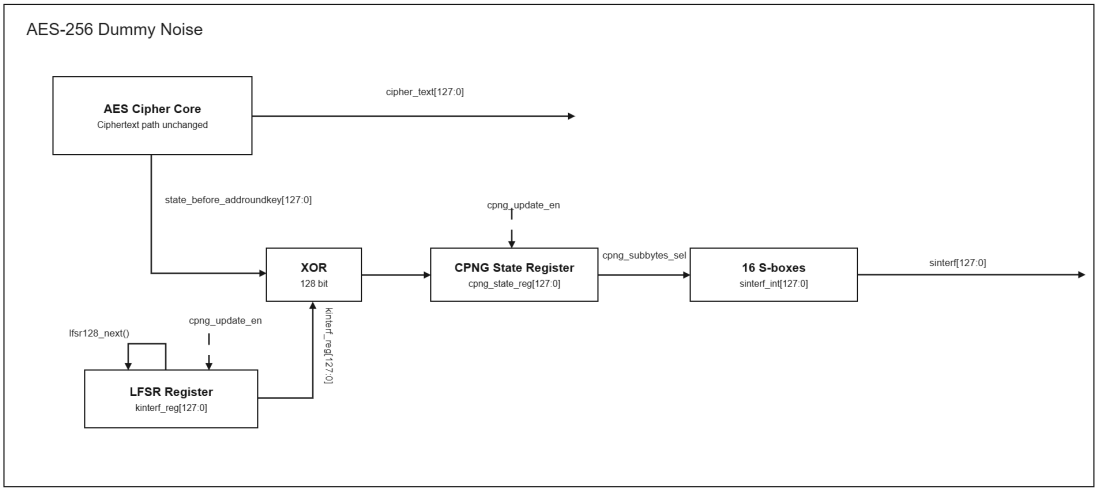
$$f = k[127] \oplus k[125] \oplus k[100] \oplus k[98], \qquad k^{+} = \{k[126{:}0], f\}.$$

Ở trạng thái Reset, `cpng_state_reg` được đưa về 0, trong khi `kinterf_reg` được nạp giá trị `SEED` cố định khai báo trong module. Điều này có nghĩa chuỗi trạng thái LFSR là xác định nếu cùng điều kiện khởi tạo và Enable; nguồn này không phải bộ sinh số ngẫu nhiên vật lý. Tín

hiệu cho phép cập nhật được tạo từ trạng thái hoạt động của AES:

$$\texttt{cpng_update_en} = \texttt{cipher_en} \vee \big(\texttt{rkey_en} \wedge \neg\texttt{cipher_complete}\big).$$

Sau khi được chốt, `cpng_state_reg` đi qua mạng hoán vị byte `cpng_subbytes_sel` và 16 lần gọi hàm S-box, tạo `sinterf_int[127:0]`. Đầu ra này được nối tới `sinterf` của CPNG và `cpng_sinterf` trong Cipher Core. Trong RTL được khảo sát, `cpng_sinterf` không tham gia phép tính `after_addRoundkey` và không được nối vào bus Ciphertext. Vì vậy, nhánh CPNG về mặt chức năng được tách khỏi đường dữ liệu tạo kết quả mã hóa; sự hiện diện của nó hướng tới tạo thêm chuyển mạch nội bộ chứ không làm biến đổi thuật toán AES.



Hình 17: Nhánh CPNG hoạt động song song với Cipher Core trong biến thể Dummy Noise.

Trong RTL, một số thanh ghi và module CPNG có sử dụng các thuộc tính như `KEEP`, `DONT_TOUCH` và `KEEP_HIERARCHY` nhằm hạn chế công cụ loại bỏ hoặc tái cấu trúc nhánh logic không ảnh hưởng đến đầu ra AES. Tuy nhiên, mức độ bảo toàn logic thực tế vẫn cần được kiểm tra trên netlist sau tổng hợp và Implementation. Tương tự, việc có thêm các nút chuyển mạch không đủ để kết luận mức tiêu thụ công suất đã tăng theo một tỷ lệ nhất định hoặc khả năng chống phân tích công suất đã được cải thiện. Các kết luận này chỉ có thể đưa ra sau khi đo hoặc mô phỏng công suất theo cùng điều kiện giữa các biến thể.

3.3.5 So sánh cấu trúc các biến thể

Bốn biến thể có thể được phân biệt theo phương pháp thực hiện S-box, trạng thái lưu trữ và logic điều khiển bổ sung. Bảng 10 tổng hợp các đặc điểm được xác nhận từ RTL, chưa bao gồm kết quả đo tài nguyên hay hiệu năng sau khi đưa lên FPGA.

Bảng 10: Đối chiếu cấu trúc RTL của bốn biến thể AES-256.

Tiêu chí	LUT	BRAM	Composite Field	Dummy Noise
S-box chính	ROM tổ hợp, hướng đến distributed LUT	ROM đọc đồng bộ theo Clock	Mạng số học Composite Field	Tra bảng bằng hàm tổ hợp như lõi nền
S-box ở mức RTL	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa	16 cho State; 4 cho khóa; thêm 16 S-box CPNG
Điều khiển Round	Bộ đếm Round	FSM ba trạng thái và bộ đếm Round	Bộ đếm Round	Bộ đếm Round; thêm Enable cho CPNG
Đường dữ liệu bổ sung	Không	Đường đọc S-box có chốt đầu ra	Logic trường con thay ROM	LFSR, thanh ghi CPNG và mạng S-box riêng
Đặc điểm cần đánh giá	LUT và Timing đường tổ hợp	Memory mapping, số chu kỳ, Timing	LUT, chiều sâu logic và Timing	Tài nguyên logic, công suất và switching activity

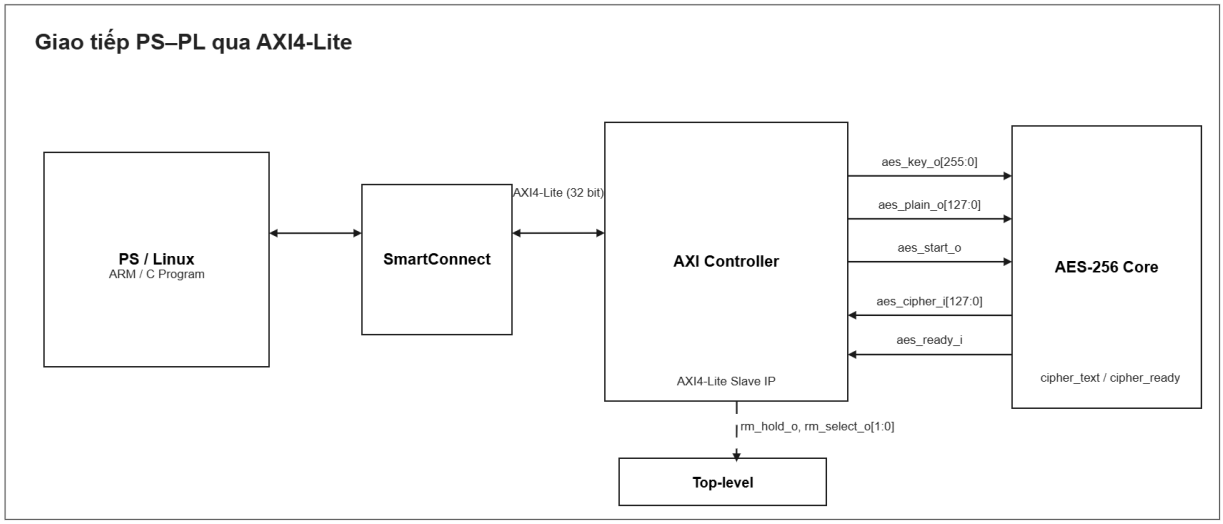
Biến thể LUT có ưu điểm về cấu trúc điều khiển đơn giản và phù hợp làm mốc tham chiếu. BRAM thay đổi cách truy xuất S-box, đồng thời cần FSM để bảo đảm dữ liệu đọc được sử dụng đúng thời điểm. Composite Field giữ tiến trình xử lý theo Round nhưng thay bảng tra bằng mạng số học trường con. Dummy Noise lại giữ đường dữ liệu AES và bổ sung một nhánh CPNG hoạt động song song. Nhờ đó, phép so sánh giữa các biến thể không chỉ phản ánh sự thay đổi một module S-box, mà còn cho thấy ảnh hưởng của lựa chọn kiến trúc đến cách tổ chức dữ liệu và điều khiển.

3.4 Thiết kế giao tiếp PS–PL

Giao tiếp giữa Processing System (PS) và Programmable Logic (PL) được xây dựng để phần mềm trên ARM có thể nạp khóa, cung cấp Plaintext, khởi động lõi AES-256 và đọc Ciphertext mà không cần thao tác trực tiếp với các tín hiệu nội bộ của Cipher Core. Trong thiết kế của nhóm, lớp giao tiếp này là một AXI4-Lite Slave IP tùy chỉnh, với dữ liệu thanh ghi rộng 32 bit. Mục 3.4 tập trung vào giao diện và ánh xạ thanh ghi; cách kết nối cụ thể với Reconfigurable Partition và các Replica được trình bày ở Mục 3.5.

3.4.1 Kiến trúc kết nối PS–PL

Trên nền tảng KV260, chương trình C chạy dưới Linux ở PS thực hiện các giao dịch đọc/ghi tại vùng địa chỉ AXI được ánh xạ cho bộ điều khiển. Đường truy cập đi qua khối kết nối AXI trong PL đến Custom AXI Controller. Từ các thanh ghi 32 bit, bộ điều khiển tạo hai bus song song `aes_key_o[255:0]` và `aes_plain_o[127:0]`, đồng thời phát xung `aes_start_o` khi có lệnh bắt đầu. Chiều ngược lại, lõi mã hóa cung cấp `aes_cipher_i[127:0]` cùng `aes_ready_i` để bộ điều khiển lưu kết quả và đưa trạng thái về PS.



Hình 18: Kết nối dữ liệu và điều khiển giữa PS, AXI4-Lite Controller và AES-256 Core.

Trong Vivado Block Design, hệ thống sử dụng các khối `zynq_ultra_ps_e_0`, `smartconnect_0` và `proc_sys_reset_0`. Custom AXI Controller được kết nối ở phía Slave của mạng AXI. Tầng này chỉ thực hiện chức năng giao tiếp, không chứa các phép biến đổi AES. Xung Clock và Reset được phân phối từ hệ thống PS–PL để đồng bộ việc truy cập thanh ghi với phần cứng. Đối với mô hình DPR, các tín hiệu từ AXI Controller còn đi qua logic điều khiển và cách ly RM; đối với Replica, tín hiệu chọn biến thể được sử dụng ở tầng Top-level. Những khác biệt này không làm thay đổi quy ước truy cập thanh ghi của phần mềm.

3.4.2 Custom AXI4-Lite Controller

Module `aes256_axi_controller_slave_lite_v1_0_S00_AXI` có bus dữ liệu AXI rộng 32 bit và bus địa chỉ rộng 7 bit. Phần giao thức có các kênh ghi địa chỉ (AW), ghi dữ liệu (W), phản hồi ghi (B), đọc địa chỉ (AR) và đọc dữ liệu (R) theo AXI4-Lite [9]. Bộ điều khiển tiếp nhận giao dịch ghi khi địa chỉ và dữ liệu hợp lệ đồng thời với tín hiệu sẵn sàng tương ứng; phản hồi ghi dùng mã `OKAY`. Với thao tác đọc, địa chỉ được chốt tại kênh AR, dữ liệu 32 bit được chọn từ Register Map và trả qua kênh R. Các tín hiệu `WSTRB[3:0]` cho phép ghi theo từng byte, tránh thay đổi các byte không được chọn khi cập nhật một thanh ghi dữ liệu.

Về chức năng, RTL gồm ba phần: giải mã giao dịch AXI, nhóm thanh ghi điều khiển và dữ liệu, và logic chuyển đổi giữa thanh ghi 32 bit với các cổng dữ liệu AES. Các từ khóa được ghép theo thứ tự `{KEY7, ..., KEY0}` để tạo khóa 256 bit; tương tự, bốn từ Plaintext tạo thành bus 128 bit. Chiều đọc sử dụng thanh ghi `cipher_latched[127:0]` để lưu kết quả mã hóa và tách thành bốn từ 32 bit. Nhờ đó, giao diện của lõi AES không phải thay đổi khi chuyển giữa bốn biến thể phần cứng.

Lệnh Start được hiện thực như một xung điều khiển, không phải một bit lưu liên tục trong thanh ghi. Khi có giao dịch ghi vào `COMMAND` với bit 0 của dữ liệu bằng 1 và `WSTRB[0]` được bật, RTL đưa `aes_start_o` lên mức 1 trong một chu kỳ `S_AXI_ACLK`; tại các chu kỳ còn lại tín

hiệu này mặc định trở về 0. Cùng thao tác Start, cờ `done_latched` của yêu cầu trước được xóa. Việc phát xung một chu kỳ giúp Cipher Core nhận một yêu cầu mới mà không phụ thuộc thời gian chương trình giữ dữ liệu ghi trên bus AXI.

3.4.3 Register Map và tín hiệu điều khiển

Register Map được tổ chức theo các offset 4 byte, phù hợp với dữ liệu AXI4-Lite rộng 32 bit. Trong hệ thống thí nghiệm, vùng thanh ghi được ánh xạ tại địa chỉ cơ sở `0xA0000000`; đây là địa chỉ cấu hình của hệ thống tích hợp chứ không phải hằng số được cố định bên trong RTL của Slave IP. Bảng 11 trình bày các vùng địa chỉ và chiều truy cập tương ứng.

Bảng 11: Ánh xạ thanh ghi của AXI4-Lite Controller.

Offset	Thanh ghi	R/W	Chức năng
`0x00`	COMMAND	W	Bit 0 phát xung `START`; đọc về 0.
`0x04`	STATUS	R	Bit 0: `DONE` (chốt); bit 1: `AES_READY` trực tiếp.
`0x08`	RM_CONTROL	R/W	Bit 0: `RM_HOLD`; bit [2:1]: `RM_SELECT`.
`0x0C`	RESERVED	–	Chưa sử dụng.
`0x10-0x2C`	KEY0–KEY7	R/W	Tám từ 32 bit tạo `aes_key_o[255:0]`.
`0x30-0x3C`	PLAIN0–PLAIN3	R/W	Bốn từ 32 bit tạo `aes_plain_o[127:0]`.
`0x40-0x4C`	CIPHER0–CIPHER3	R	Bốn từ 32 bit từ `cipher_latched[127:0]`.

Với nhóm KEY, địa chỉ `0x10` chứa `cipher_key[31:0]` và địa chỉ `0x2C` chứa `cipher_key[255:224]`; các từ giữa tiếp tục tăng theo từng 32 bit. Quy ước tương tự được áp dụng với Plaintext và Ciphertext. Chẳng hạn, `CIPHER0` đọc phần thấp `cipher_latched[31:0]`, còn `CIPHER3` đọc phần cao `cipher_latched[127:96]`. Sự thống nhất thứ tự từ giữa phần mềm và RTL là điều kiện quan trọng để kết quả trên KV260 trùng với vector kiểm chứng AES.

Trường `RM_CONTROL[0]` được đưa ra cổng `rm_hold_o`, sử dụng khi cần giữ hoặc cách ly Reconfigurable Module trong kiến trúc DPR. Hai bit `RM_CONTROL[2:1]` được đưa ra `rm_select_o[1:0]` với quy ước `00`: LUT, `01`: BRAM, `10`: Composite Field và `11`: Dummy Noise. Việc giải thích cơ chế chọn RM và chuyển đổi cấu hình thuộc Mục 3.5; trong phạm vi 3.4, các bit này được xem là giao diện điều khiển xuất phát từ Register Map.

3.4.4 Handshake và trình tự trao đổi dữ liệu

Một yêu cầu mã hóa được thực hiện qua các bước truy cập thanh ghi. Trước tiên, phần mềm ghi tám từ KEY và bốn từ Plaintext theo đúng thứ tự địa chỉ. Nếu sử dụng Replica, phần mềm chọn biến thể qua `RM_SELECT`; nếu sử dụng DPR, việc giữ RM trong quá trình đổi cấu hình được quản lý riêng và phải được giải phóng trước khi bắt đầu mã hóa. Sau đó, ghi giá trị có bit 0 bằng 1 vào `COMMAND` để phát Start. Lõi AES nhận dữ liệu đã nạp, tiến hành mã hóa và cập nhật tín hiệu hoàn tất khi kết quả sẵn sàng. Cuối cùng, phần mềm đọc `STATUS`, lấy bốn thanh ghi Ciphertext và ghép chúng lại thành kết quả 128 bit. Việc đối chiếu Ciphertext với vector tham chiếu và lưu thời gian thực thi thuộc Chương 5–6.

Ở chiều phản hồi, AXI Controller sử dụng cặp tín hiệu `aes_ready_i` và `aes_cipher_i[127:0]`. RTL hiện tại có điều kiện `if (aes_ready_i)` để chốt Ciphertext và đưa `done_latched` lên mức 1. Vì vậy, bit `STATUS[0]` phản ánh cờ DONE đã lưu, trong khi `STATUS[1]` phản ánh trực tiếp mức tín hiệu `aes_ready_i`. Hai bit có ý nghĩa khác nhau và không nên sử dụng thay thế cho nhau khi phân tích waveform.

`aes_ready_i` ở một số biến thể có thể duy trì mức 1 trong trạng thái nghỉ. Trong khi đó, điều kiện chốt DONE của phiên bản AXI Controller đang kiểm tra mức cao, chưa kiểm tra cạnh lên mới của READY sau khi phát Start. Do đó, nếu DONE được xóa tại thời điểm Start nhưng READY vẫn đang cao trong một chu kỳ tiếp theo, cờ này có khả năng được đặt lại trước khi phép mã hóa mới hoàn tất. Đây là điểm cần được kiểm chứng ở mức tích hợp; một hướng hoàn thiện là chỉ cho phép chốt kết quả sau khi đã ghi nhận pha Busy (READY xuống 0) của yêu cầu hiện tại. Phần mô tả này phản ánh đúng hành vi RTL hiện có, không giả định cơ chế cải tiến đã được tích hợp.

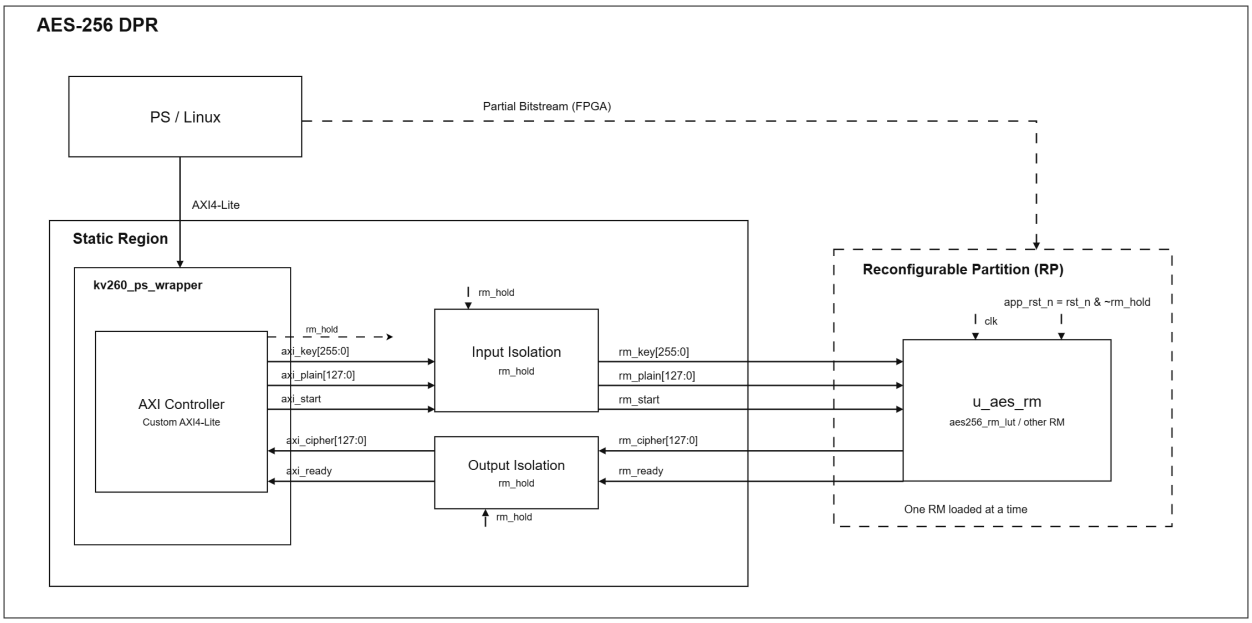
Như vậy, AXI4-Lite Controller đóng vai trò giao diện ổn định giữa phần mềm và các lõi AES-256. Các khác biệt giữa DPR và Replica tập trung ở tầng điều khiển và liên kết phần cứng phía sau giao diện này, trong khi phần mềm vẫn sử dụng các nhóm thanh ghi KEY, PLAIN, COMMAND, STATUS và CIPHER theo cùng quy ước.

3.5 Thiết kế kiến trúc DPR và Replica

Sau khi thống nhất giao diện AES-256 và lớp thanh ghi AXI4-Lite, nhóm xây dựng hai phương án tích hợp các biến thể trên KV260. Phương án Dynamic Partial Reconfiguration (DPR) sử dụng một Reconfigurable Partition (RP) để lần lượt triển khai các Reconfigurable Module (RM). Trong khi đó, phương án Replica giữ bốn lõi AES-256 thường trú trong PL và lựa chọn lõi hoạt động bằng tín hiệu điều khiển. Nội dung mục này tập trung vào tổ chức Top-level, các đường kết nối và cơ chế chuyển đổi được hiện thực trong RTL. Nguyên lý chung của DPR và Replica đã trình bày ở Chương 2, còn việc tạo bitstream và triển khai trên FPGA được trình bày ở Chương 4.

3.5.1 Kiến trúc Top-level DPR

Kiến trúc DPR được mô tả tại `top_dfx_aes256.sv`. Thành phần `kv260_ps_wrapper` cung cấp Clock, Reset, dữ liệu và tín hiệu điều khiển từ hệ thống PS–AXI. Về phía giao diện AES, ba tín hiệu xuất từ phần tĩnh gồm `axi_key[255:0]`, `axi_plain[127:0]` và `axi_start`; chiều ngược lại sử dụng `axi_cipher[127:0]` và `axi_ready`. Các tín hiệu này đi qua lớp cách ly trước khi kết nối với RM. Nhờ đó, các thanh ghi AXI và phần mềm điều khiển không phải thay đổi khi thay thế một biến thể AES-256 bằng biến thể khác.



Hình 19: Kết nối Top-level DPR với đường dữ liệu qua logic cách ly và một Reconfigurable Partition.

Trong Hình 19, RP chứa thể hiện `u_aes_rm`. Đây là tên instance được giữ cố định trong Top-level để phục vụ việc gắn với định nghĩa phân vùng tái cấu hình trong quy trình DFX. Phiên bản RTL đang sử dụng `aes256_rm_lut` tại vị trí này; các wrapper `aes256_rm_bram`, `aes256_rm_composite` và `aes256_rm_dummy_noise` có cùng tập cổng, dùng làm các phương án RM thay thế. Mỗi wrapper nhận `clk_sys`, `rst_n`, Key, Plaintext và Start, đồng thời trả về Ciphertext cùng Ready. Tại một thời điểm vận hành, RP chỉ chứa RM tương ứng với cấu hình đã nạp, không phải bốn lõi hoạt động đồng thời.

Lớp cách ly được điều khiển bằng `rm_hold`, lấy từ thanh ghi `RM_CONTROL[0]`. Khi tín hiệu này bằng 1, RTL đưa `rm_key`, `rm_plain`, `rm_start` về 0 và cũng chặn Ciphertext, Ready từ RM ở chiều phản hồi. Khi `rm_hold` bằng 0, các đường dữ liệu được nối thông tương ứng giữa phía AXI và RM. Bên cạnh đó, Reset cấp vào RM được xác định theo:

$$\texttt{app_rst_n} = \texttt{rst_n} \ \wedge \ \neg\texttt{rm_hold}.$$

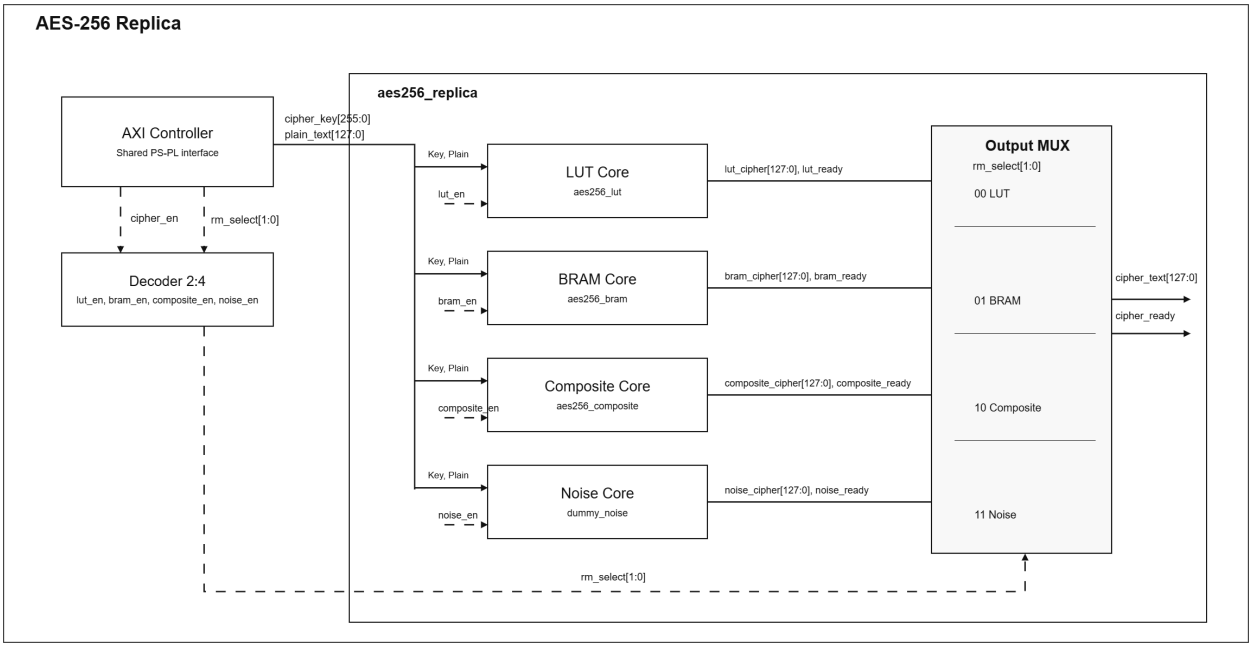
Do đó, đưa `rm_hold` lên 1 vừa đặt RM về trạng thái Reset, vừa ngăn các tín hiệu không xác định từ vùng đang thay đổi lan sang các khối điều khiển tĩnh. Phần AXI vẫn duy trì hoạt động để phần mềm tiếp tục truy cập thanh ghi. Đây là cơ chế bảo vệ giao diện ở mức RTL, không thay thế các yêu cầu về cách ly và kiểm tra tính hợp lệ của thiết kế trong DFX flow.

Việc lựa chọn biến thể của DPR không thực hiện bằng `rm_select`. Phần mềm trên PS quản lý tệp Partial Bitstream và yêu cầu hệ thống nạp cấu hình mới thông qua FPGA Manager. Một trình tự điều khiển thích hợp là kết thúc tác vụ đang xử lý, đặt `rm_hold=1`, nạp cấu hình RM mới, đưa `rm_hold=0` và chỉ sau đó phát Start. Như vậy, đường nạp cấu hình là luồng điều khiển riêng với đường AXI4-Lite truyền Key, Plaintext và Ciphertext. Mục 3.5 chỉ mô tả cấu trúc kết nối; các bước tạo RP, bố trí Pblock và nạp Partial Bitstream thực tế sẽ được trình bày ở

Chương 4.

3.5.2 Kiến trúc Top-level Replica

Phương án Replica được mô tả trong module `aes256_replica.sv`, khởi tạo bốn lõi độc lập: `u_lut`, `u_bram`, `u_composite` và `u_noise`. Cả bốn cùng nhận Key 256 bit, Plaintext 128 bit, Clock và Reset. Tuy nhiên, tín hiệu Start không được phân phối đồng thời đến tất cả các lõi. RTL giải mã `rm_select[1:0]` để chỉ cấp xung Start cho biến thể đang được chọn; các lõi còn lại không nhận yêu cầu mã hóa mới.



Hình 20: Kết nối các lõi AES-256 trong kiến trúc Replica với logic giải mã Start và lựa chọn đầu ra.

Ở Hình 20, đường Key và Plaintext được dùng chung, còn bốn tín hiệu `lut_en`, `bram_en`, `composite_en` và `noise_en` là các nhánh Start riêng. Các nhánh nét đứt được nhận diện bằng nhãn net tương ứng để tránh tạo nhiều đường giao nhau trên hình. Điều kiện kích hoạt từng lõi được viết gọn như sau:

$$\texttt{en}_i = \texttt{cipher_en} \ \wedge \ (\texttt{rm_select} = i), \quad i \in \{0, 1, 2, 3\}.$$

Tại đầu ra, mỗi lõi cung cấp cặp tín hiệu Ciphertext và Ready độc lập. Khối Output MUX chọn đồng thời hai tín hiệu của cùng một lõi theo giá trị `rm_select[1:0]`. Ánh xạ lựa chọn trong RTL được thể hiện tại Bảng 12.

Bảng 12: Ánh xạ tín hiệu chọn biến thể của kiến trúc Replica.

rm_select	Lõi nhận Start	Cặp tín hiệu đầu ra được chọn
2'b00	LUT	lut_cipher, lut_ready
2'b01	BRAM	bram_cipher, bram_ready
2'b10	Composite Field	composite_cipher, composite_ready
2'b11	Dummy Noise	noise_cipher, noise_ready

Bộ chọn đầu ra được mô tả bằng `always_comb`, vì vậy thay đổi `rm_select` làm thay đổi trực tiếp đường Ciphertext và Ready đang quan sát, không có thanh ghi chốt lựa chọn trong `aes256_replica`. Để tránh đọc nhầm kết quả của lõi khác, phần mềm cần xác lập giá trị chọn trước khi Start và giữ nguyên cho đến khi yêu cầu mã hóa hiện tại hoàn tất. Trong kiến trúc này, việc đổi biến thể không đòi hỏi nạp Partial Bitstream mới, nhưng mọi lõi vẫn hiện diện trong thiết kế nên chi phí tài nguyên đồng thời có thể cao hơn DPR. Mức sử dụng LUT, FF, BRAM và thời gian thực thi thực tế sẽ được xác định bằng kết quả tổng hợp và đo kiểm ở các chương sau.

Cần đồng bộ trong mã nguồn là instance `u_composite` của `aes256_replica.sv` hiện gọi module `aes256_composite`, trong khi file `aes_composite.sv` được cung cấp khai báo module cấp cao `aes_composite`. Trước khi tổng hợp Replica, nhóm cần sửa tên instance hoặc bổ sung wrapper phù hợp; phần mô tả kiến trúc ở đây không mặc nhiên khẳng định hai tên đã tương thích trong bộ mã nguồn hiện tại.

3.5.3 Giao diện chung và cơ chế chuyển đổi biến thể

Hai kiến trúc sử dụng cùng định dạng Key 256 bit, Plaintext 128 bit, Ciphertext 128 bit và cặp tín hiệu Start–Ready. Vì vậy, chương trình trên PS có thể tiếp tục sử dụng Register Map ở Mục 3.4 cho cả hai trường hợp. Sự khác biệt nằm ở thao tác chuẩn bị phần cứng trước mỗi yêu cầu mã hóa: DPR phải bảo đảm đã nạp đúng RM và giải phóng cách ly, còn Replica chỉ cần đặt lựa chọn MUX phù hợp. Bảng 13 tổng hợp các điểm khác nhau trực tiếp thể hiện trong Top-level RTL.

Bảng 13: So sánh tổ chức và cơ chế chuyển đổi tại mức Top-level RTL.

Nội dung	DPR	Replica
Lõi AES trong PL	Một `u_aes_rm` tại RP ở một thời điểm	Bốn thể hiện AES tồn tại trong RTL đồng thời
Lựa chọn biến thể	Nạp Partial Bitstream tương ứng từ PS	`rm_select[1:0]` điều khiển Start Decoder và Output MUX
`RM_HOLD`	Dùng để cách ly đầu vào, đầu ra và giữ Reset RM	Không tham gia chọn lõi trong `aes256_replica`
Thao tác trước Start	Hoàn tất nạp RM, bỏ Hold, sau đó mới Start	Ghi Select, giữ ổn định khi xử lý, sau đó Start
Điều kiện đổi biến thể	Cần thời gian nạp lại cấu hình RM	Đổi đường chọn logic, không cần tái cấu hình PL
Giao diện phần mềm	KEY, PLAIN, COMMAND, STATUS, CIPHER	KEY, PLAIN, COMMAND, STATUS, CIPHER

Việc duy trì cùng giao diện phần mềm là cơ sở để sử dụng các vector kiểm chứng và quy trình thu thập kết quả thống nhất khi so sánh hai kiến trúc. Tuy nhiên, thao tác chọn biến thể chưa đủ để khẳng định một yêu cầu mới đã hoàn tất: phần mềm vẫn cần tuân thủ cơ chế xác nhận trạng thái mã hóa và các giới hạn của logic DONE đã phân tích ở Mục 3.4.4.

CHƯƠNG 4: HIỆN THỰC HỆ THỐNG TRÊN FPGA VÀ ARM/LINUX

4.1 Môi trường và quy trình hiện thực

Chương 3 đã xác định cấu trúc các lõi AES-256, giao diện AXI4-Lite và hai phương án tổ chức phần cứng DPR, Replica. Từ các mô tả đó, công việc hiện thực cần bảo đảm ba yêu cầu: các thành phần RTL được tích hợp vào cùng hệ thống PS–PL, các biến thể AES-256 được triển khai theo đúng mô hình tổ chức đã chọn, và kết quả của quá trình Synthesis–Implementation có thể sử dụng để tạo cấu hình cho thiết bị. Chương này trình bày quá trình chuyển từ thiết kế RTL sang các cấu hình phần cứng trên nền tảng Kria KV260. Phần kiểm chứng tính đúng đắn, phân tích Timing, tài nguyên và thực nghiệm chuyển đổi biến thể được dành cho Chương 5 và Chương 6.

4.1.1 Nền tảng phần cứng và công cụ thiết kế

Nền tảng thực nghiệm sử dụng Kria KV260 với thiết bị thuộc họ Zynq UltraScale+ MPSoC. Đây là kiến trúc có Processing System (PS) và Programmable Logic (PL) trong cùng một thiết bị: PS đảm nhiệm chạy hệ điều hành, quản lý các giao dịch điều khiển và cấu hình, còn PL hiện thực phần cứng mã hóa AES-256. Cách phân chia này cho phép chương trình phía PS truy cập khối mã hóa qua các thanh ghi AXI4-Lite mà không phải chuyển toàn bộ thuật toán sang phần mềm.

Phần thiết kế phần cứng được tổ chức trong Vivado Design Suite. Các module SystemVerilog mô tả AES-256 và các Top-level kết nối được thêm vào Design Sources; khối PS và các IP hạ tầng được xây dựng bằng IP Integrator. Trong phương án DPR, Vivado DFX flow còn được dùng để định nghĩa Reconfigurable Partition, gắn các Reconfigurable Module và xây dựng từng Configuration Run. Với Replica, toàn bộ bốn lõi được tổng hợp và triển khai trong một thiết kế tĩnh. Các ảnh chụp Vivado trong các mục tiếp theo là tư liệu trực tiếp từ project của nhóm; chúng không phải hình minh họa tạo lại.

Tại giai đoạn hiện tại, nhóm đã cung cấp bằng chứng Block Design, Address Editor, RTL Hierarchy, Design Runs và Device View cho quá trình hiện thực. Các ảnh này cho phép phân tích cấu trúc và trạng thái của từng Run, nhưng không cung cấp phiên bản Vivado cụ thể, báo cáo Utilization hay báo cáo Timing. Vì vậy, chương này không gán các thông số đó khi chưa có tài liệu công cụ tương ứng.

4.1.2 Phân chia quy trình triển khai

Quy trình hiện thực được chia theo các đầu ra kỹ thuật cần đạt, thay vì mô tả tuần tự từng thao tác trên giao diện Vivado. Trước tiên, nhóm tích hợp PS, mạng AXI và bộ điều khiển AES vào một hệ thống SoC, sau đó nối phần hạ tầng này với Top-level RTL. Tiếp theo, từng kiến trúc được xử lý theo quy trình riêng: DPR cần cấu hình các RM, quản lý các Configuration Runs và phân vùng vật lý cho RP; Replica cần tổng hợp toàn bộ bốn lõi trong một thiết kế và

thực hiện Implementation để tạo bitstream. Cuối cùng, các file cấu hình được chuyển sang môi trường ARM/Linux trên KV260 để nạp và điều khiển; nội dung đó thuộc Mục 4.5–4.6.

Bảng 14: Các giai đoạn hiện thực và tư liệu xác nhận trong Vivado.

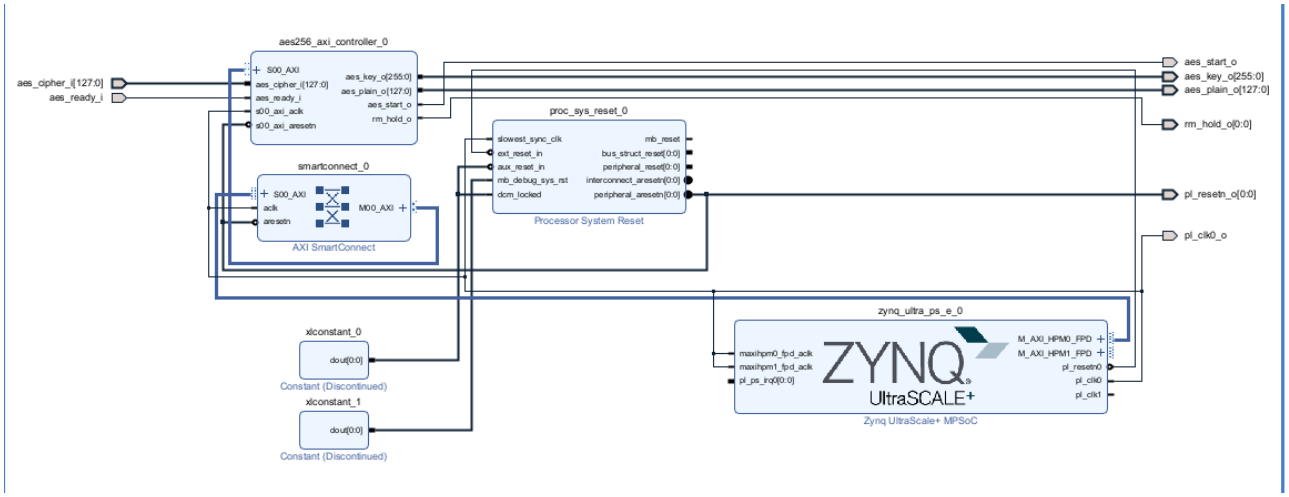
Giai đoạn	Nội dung cần thực hiện	Tư liệu đối chiếu
Tích hợp PS–PL	Tạo hệ thống AXI, cấu hình Clock/Reset, ánh xạ địa chỉ và nối tới Top-level.	Block Design, Address Editor, RTL Hierarchy.
Hiện thực DPR	Tổng hợp các RM, lập Configuration, định nghĩa Pblock và chạy Parent/Child Implementation.	DFX Configurations, Design Runs, Device View.
Hiện thực Replica	Tích hợp đồng thời bốn Core, chạy Synthesis/Implementation và sinh bitstream thiết kế tĩnh.	RTL Hierarchy, Design Runs, Device View.
Triển khai trên board	Chuẩn bị file cấu hình và điều khiển AES từ môi trường ARM/Linux.	File cấu hình, mã C và log trên KV260 (Mục 4.5–4.6).

Mục 4.2 trình bày lớp hạ tầng PS–PL dùng chung; Mục 4.3 và 4.4 lần lượt phân tích các bước triển khai đặc thù của DPR và Replica. Cách tổ chức này giúp tách biệt phần tích hợp SoC với các quyết định về phân vùng hay nhân bản phần cứng, đồng thời không lặp lại các sơ đồ chức năng đã trình bày ở Chương 3.

4.2 Tích hợp hệ thống SoC trong Vivado

4.2.1 Xây dựng Block Design và các kết nối PS–PL

Nhóm sử dụng IP Integrator để tạo hệ thống hạ tầng giao tiếp trong PL. Hình 21 là Block Design thực tế, gồm khối `zynq_ultra_ps_e_0`, `smartconnect_0`, `proc_sys_reset_0` và IP `aes256_axi_controller_0`. Hai khối `xlconstant_0` và `xlconstant_1` được sử dụng cho các kết nối cố định của hệ thống. Đây là cấu trúc triển khai giao tiếp, không phải sơ đồ nội bộ của thuật toán AES-256.



Hình 21: Block Design tích hợp Zynq UltraScale+ MPSoC, AXI SmartConnect, Reset và Custom AXI Controller trong Vivado.

`zynq_ultra_ps_e_0` là đầu mối phát các giao dịch AXI từ PS, trong khi `smartconnect_0` đóng

vai trò kết nối AXI Master với Slave IP. Ở phía đích, giao diện S00_AXI của aes256_axi_controller_0 nhận các giao dịch truy cập thanh ghi. Bằng cách sử dụng SmartConnect, thiết kế không cần hiện thực riêng cơ chế phân phối giao dịch giữa PS và Slave. Điểm quan trọng đối với đề tài là kênh điều khiển mang dữ liệu qua AXI4-Lite, trong khi Key, Plaintext và Ciphertext sau khi đi qua các thanh ghi được chuyển thành các bus song song tại giao diện RTL của bộ mã hóa.

Trong Block Design, các cổng phía ứng dụng của IP được đưa ra bên ngoài, gồm aes_key_o[255:0], aes_plain_o[127:0], aes_start_o, aes_cipher_i[127:0], aes_ready_i và rm_hold_o. Những cổng này không nối đến một AES Core nằm trong sơ đồ IP Integrator; chúng được kết nối ở RTL Top-level. Vì vậy, chỉ nhìn ảnh Block Design sẽ chưa thấy cấu trúc RP hay bốn Replica, mặc dù giao tiếp PS–PL đã được triển khai đầy đủ.

Bảng 15: Vai trò các IP chính trong Block Design của nhóm.

IP trong Vivado	Vai trò ở mức hiện thực
zynq_ultra_ps_e_0	Cấu hình Processing System, xuất Clock/Reset và giao diện AXI của PS.
smartconnect_0	Kết nối luồng truy cập AXI từ PS đến Custom AXI Slave.
proc_sys_reset_0	Tổ chức các tín hiệu Reset tương ứng miền Clock cho hệ thống PL.
aes256_axi_controller_0	Chuyển giao dịch thanh ghi thành bus dữ liệu và các tín hiệu điều khiển AES-256.

4.2.2 Phân phối Clock, Reset và giao diện Top-level

Block Design sử dụng Clock từ PS, thể hiện bằng các cổng pl_clk0 và pl_clk0_o trong hệ thống. Clock này được dùng cho mạng AXI và các thành phần logic phía PL theo kết nối của thiết kế. Khối proc_sys_reset_0 cung cấp tín hiệu Reset phù hợp với miền Clock; tại giao diện Top-level, nhóm đưa ra pl_resetn_o. Việc tổ chức Clock và Reset thành các đường riêng giúp phần RTL bên ngoài Block Design sử dụng cùng nguồn đồng bộ với bộ điều khiển AXI, thay vì giả định có một Clock độc lập cho AES.

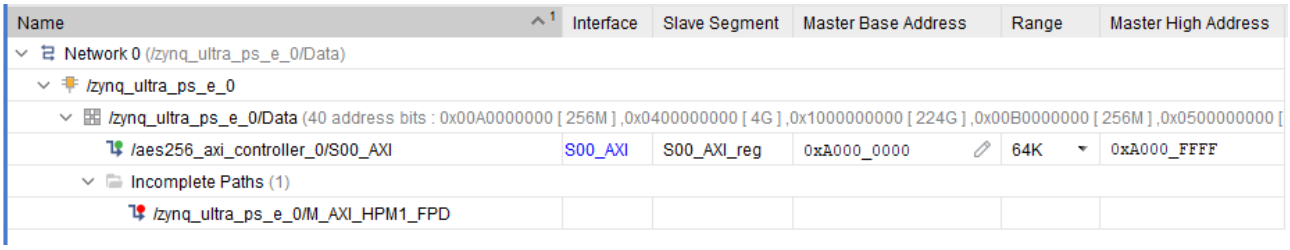
Trong phương án DPR, top_dfx_aes256.sv tạo instance u_kv260_ps của kv260_ps_wrapper và kết nối các cổng AXI-side với u_aes_rm. Ngoài Reset của PS–PL, tín hiệu rm_hold được xử lý thêm để tạo app_rst_n và cách ly giao diện RM. Đây là phần logic nằm trong Top-level, không phải chức năng của SmartConnect hay IP Reset. Đối với Replica, Top-level sử dụng instance u_aes_replica; khác biệt về nhân bản lõi và lựa chọn đầu ra được trình bày tại Mục 4.4.

Từ góc độ tích hợp, cần phân biệt *IP Integrator Block Design* với *toàn bộ thiết kế FPGA*. Block Design chỉ tập hợp các IP nền tảng và công cụ tạo wrapper; RTL Top-level mới là nơi gắn wrapper với biến thể mã hóa. Do đó, quá trình Implementation phải xét toàn bộ hierarchy của thiết kế, bao gồm cả các module bên ngoài sơ đồ IP Integrator.

4.2.3 Gán địa chỉ cho AXI Controller

Việc gán địa chỉ được thực hiện tại Vivado Address Editor. Hình 22 cho thấy đoạn Slave /aes256_axi_controller_0/S00_AXI có Base Address 0xA0000000, High Address 0xA000FFFF

và Range 64K. Đây là không gian địa chỉ mà phần mềm phía PS sử dụng để đọc/ghi các thanh ghi của bộ điều khiển. Dữ liệu được chuyển từ địa chỉ hệ thống sang offset bên trong AXI Slave; việc mã hóa các offset cụ thể đã trình bày ở Mục 3.4 nên không lặp lại bảng Register Map tại đây.



Hình 22: Cửa sổ Address Editor xác nhận vùng ánh xạ 0xA0000000–0xA000FFFF cho Custom AXI Controller.

Khoảng địa chỉ 64K là cửa sổ phân bổ trong không gian địa chỉ của PS; không đồng nghĩa tất cả các địa chỉ trong khoảng này đều đã được định nghĩa thành thanh ghi. RTL Slave thực tế chỉ giải mã các offset đã dùng cho lệnh, trạng thái, Key, Plaintext và Ciphertext. Đặt vùng ánh xạ cố định tại 0xA0000000 cho phép các chương trình truy cập MMIO sử dụng cùng địa chỉ cơ sở khi triển khai các biến thể AES-256. Tuy nhiên, việc truy cập thành công trên ARM/Linux còn cần cấu hình phần cứng đã nạp phù hợp và chương trình sử dụng đúng độ rộng truy cập; phần đó sẽ được trình bày riêng trong Mục 4.5–4.6.

Như vậy, kết quả tích hợp tại Vivado đã xác lập một tầng giao tiếp PS–PL có địa chỉ và tín hiệu cụ thể. Tầng này là cơ sở chung cho DPR và Replica, còn việc xác định Reconfigurable Partition hoặc giữ thường trú nhiều lõi AES-256 được thực hiện ở các quy trình Implementation tương ứng.

4.3 Hiện thực kiến trúc DPR

4.3.1 Xác định vùng tái cấu hình trong Top-level

Thiết kế DPR dùng top_dfx_aes256 làm Top-level, với hai instance chính: u_kv260_ps cho phần hạ tầng PS–AXI và u_aes_rm cho vùng AES có khả năng thay đổi cấu hình. Cây Design Sources trong Hình 23 ghi nhận u_aes_rm đang gắn với module aes256_rm_lut. Việc duy trì cố định tên instance này có ý nghĩa đối với DFX, vì Vivado dùng cùng một vị trí hierarchy để liên kết các RM và Pblock qua nhiều Configuration.



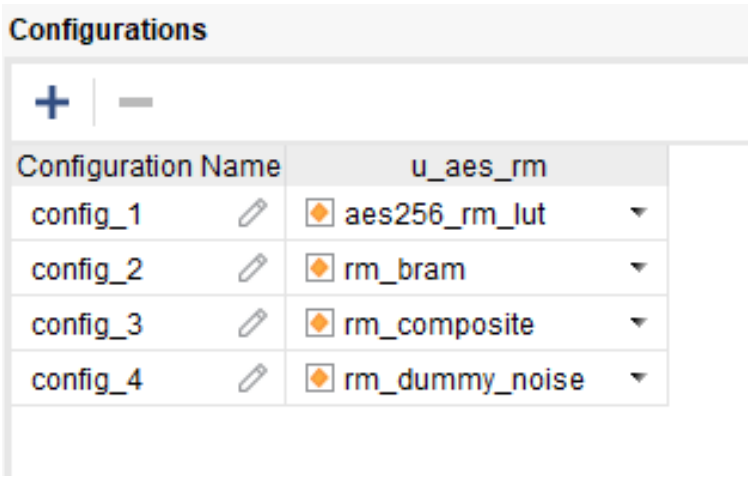
Hình 23: RTL Hierarchy của phương án DPR với Top-level top_dfx_aes256 và instance u_aes_rm.

Ở phạm vi RTL, u_aes_rm sử dụng giao diện Key 256 bit, Plaintext 128 bit, Start, Ciphertext

128 bit, Ready, Clock và Reset. Các wrapper RM tương ứng bốn biến thể giữ chung tập cổng, cho phép thay đổi module ở bên trong RP mà không phải thiết kế lại phần giao tiếp tĩnh. Vùng tĩnh duy trì khối AXI, Clock/Reset và logic cách ly được điều khiển bởi `rm_hold`. Phần này đã được phân tích dưới góc độ kiến trúc tại Mục 3.5; ở đây trọng tâm là cách Vivado tổ chức, tổng hợp và bố trí các Configuration khác nhau trên cùng ranh giới phân vùng.

4.3.2 Xây dựng các DFX Configurations và Synthesis Runs

Trong DFX Wizard, nhóm khai báo bốn Configurations dùng chung cột RM Instance `u_aes_rm`. Hình 24 cung cấp ánh xạ thực tế: `config_1` dùng `aes256_rm_lut`, `config_2` dùng `rm_bram`, `config_3` dùng `rm_composite` và `config_4` dùng `rm_dummy_noise`. Như vậy, mỗi Configuration là một phương án hoàn chỉnh gồm phần tĩnh chung với một RM cụ thể; bốn Configuration không đồng nghĩa bốn lõi cùng được đặt trong RP tại một thời điểm.



Hình 24: Bốn DFX Configurations được gắn lần lượt với bốn biến thể tại cùng RM Instance `u_aes_rm`.

Các RM có Synthesis Runs riêng ở nhóm *Out-of-Context Module Runs*. Cách tổng hợp này tách việc xây dựng netlist của từng biến thể khỏi phần hạ tầng tĩnh, phù hợp với yêu cầu quản lý nhiều cấu hình phần cứng trong DFX flow [10]. Trong hình Design Runs, các hàng `aes256_rm_lut_synth_1`, `rm_bram_synth_1`, `rm_composite_synth_1` và `rm_dummy_noise_synth_1` đều báo `synth_design Complete!`. Điều này xác nhận công cụ đã hoàn tất bước Synthesis tương ứng với các RM; nó chưa thay thế được báo cáo về tài nguyên hoặc Timing sau Implementation.

Bảng 16: Ánh xạ giữa DFX Configuration và RM trong project DPR.

Configuration	RM	Implementation Run	Kết quả trong ảnh
config_1	aes256_rm_lut	impl_1	route_design Complete!
config_2	rm_bram	child_0_impl_1	route_design Complete!
config_3	rm_composite	child_1_impl_1	route_design Complete!
config_4	rm_dummy_noise	child_2_impl_1	route_design Complete!

4.3.3 Parent Run, Child Runs và kết quả Routing

Project đang dùng DFX Mode `STANDARD`. Hình 25 thể hiện `impl_1` là Parent Implementation Run của `config_1`; ba Child Runs `child_0_impl_1`, `child_1_impl_1` và `child_2_impl_1` lần lượt triển khai những Configuration còn lại. Cả bốn dòng Implementation đều hiển thị trạng thái `route_design Complete!`.

Name	Configuration	DFX Mode	RM Instance	Constraints	Status
synth_1 (active)				**constrs_1**	**synth_design Complete!**
impl_1 (active)	**config_1**	**STANDARD**		**constrs_1**	**route_design Complete!**
child_0_impl_1	config_2	STANDARD			route_design Complete!
child_1_impl_1	config_3	STANDARD			route_design Complete!
child_2_impl_1	config_4	STANDARD			route_design Complete!
Out-of-Context Module Runs					
aes256_rm_lut_synth_1				aes256_rm_lut	synth_design Complete!
rm_bram_synth_1				rm_bram	synth_design Complete!
rm_composite_synth_1				rm_composite	synth_design Complete!
kv260_ps					Submodule Runs Complete
rm_dummy_noise_synth_1				rm_dummy_noise	synth_design Complete!

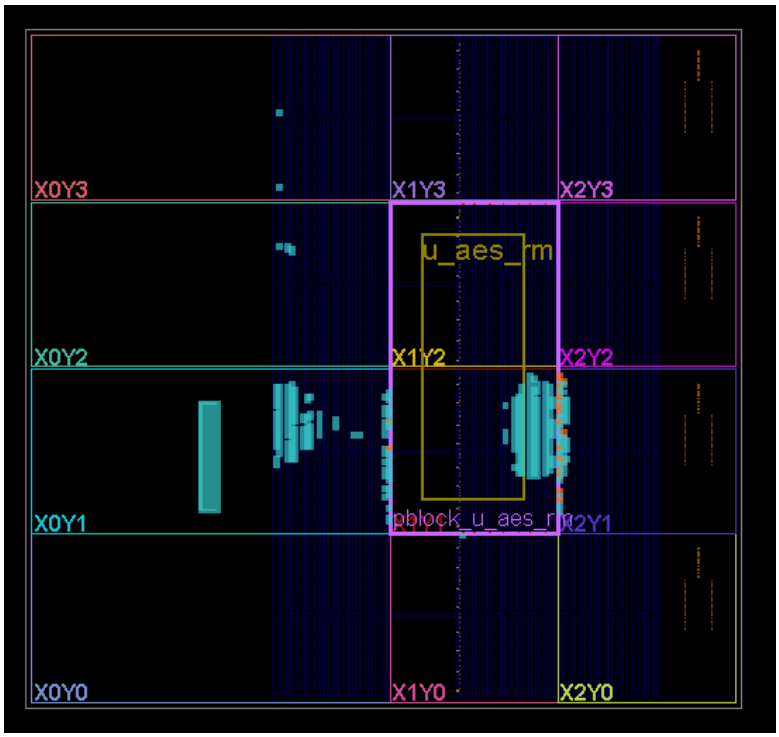
Hình 25: Design Runs của phương án DPR: Parent Run, ba Child Runs và các Out-of-Context Synthesis Runs.

Đối với Standard DFX Flow, Parent Run được triển khai trước để xác định kết quả phần tĩnh và một Configuration tham chiếu. Những Child Runs sau đó dùng lại thông tin phần tĩnh đã cố định, kết hợp với netlist của RM tương ứng để hoàn thiện Configuration mới [10]. Điều này bảo đảm cùng một vùng phần cứng tĩnh được giữ ổn định qua các phương án thay thế RM, thay vì đặt và nối lại toàn bộ hệ thống cho mỗi biến thể. Trong trường hợp của nhóm, `config_1` dùng LUT làm cấu hình tham chiếu; BRAM, Composite và Dummy Noise được triển khai ở ba Child Runs.

Cần phân biệt `route_design Complete!` với `write_bitstream Complete!`. Ảnh Design Runs ở đây xác nhận Routing, không hiển thị trạng thái tạo Partial Bitstream cho từng Child Run. Để mô tả các file cấu hình phục vụ nạp trên KV260, cần đối chiếu thư mục file đầu ra hoặc log tạo/nạp bitstream, sẽ được xử lý ở Mục 4.5. Tương tự, dòng trạng thái Routing chưa chứng minh Timing đã đạt yêu cầu, vì điều đó cần các báo cáo Timing Summary cho từng Configuration.

4.3.4 Phân vùng vật lý và Pblock

Một khác biệt quan trọng giữa DFX và Implementation tĩnh là Reconfigurable Partition phải được gắn với vùng tài nguyên vật lý xác định trên chip. Trong Vivado, vùng này được mô tả bằng Pblock, làm cơ sở để giới hạn nơi đặt logic của RM và quản lý ranh giới với phần tĩnh. Ảnh Device View ở Hình 26 cho thấy nhóm đã tạo `pblock_u_aes_rm`, bao quanh vùng triển khai của `u_aes_rm`. Phần được làm nổi bật nằm tại khu vực giữa của Device View và trải qua một số vùng tài nguyên logic; các khu vực ngoài Pblock thuộc phần còn lại của thiết kế và hạ tầng FPGA.



Hình 26: Device View của phương án DPR với Pblock `pblock_u_aes_rm` dành cho Reconfigurable Partition.

Sự tồn tại của một Pblock chung là điều kiện tổ chức cần thiết để các RM được thay thế tại cùng vị trí trong kiến trúc DFX. Khi lựa chọn vùng vật lý, thiết kế phải đáp ứng được nhu cầu của biến thể lớn nhất, đồng thời phù hợp quy tắc ranh giới và tài nguyên của thiết bị. Tuy nhiên, từ hình Device View này không thể suy ra chính xác tổng số LUT, FF, BRAM của từng RM, mức lấp đầy Pblock hay mức dự trữ tài nguyên cho các biến thể khác. Những kết luận định lượng chỉ được đưa ra khi có báo cáo Utilization và các kiểm tra DFX tương ứng.

Tổng hợp các bằng chứng, project DPR đã có bốn Configuration được ánh xạ vào cùng `u_aes_rm`, các RM Synthesis Runs hoàn tất và bốn Implementation Runs hoàn tất Routing, kèm Pblock vật lý của RP. Đây là kết quả hiện thực ở mức Vivado. Việc tạo/nạp cấu hình từng phần và kiểm chứng vận hành khi chuyển RM trên KV260 được trình bày riêng, tránh dùng trạng thái công cụ làm bằng chứng thay cho kết quả thực nghiệm.

4.4 Hiện thực kiến trúc Replica

4.4.1 Tích hợp bốn lõi trong một thiết kế tĩnh

Trong phương án Replica, nhóm không sử dụng Reconfigurable Partition mà tích hợp bốn biến thể AES-256 vào một hierarchy phần cứng thường trú. Hình 27 cho thấy Top-level `top_replica_aes256`, instance `u_kv260_ps` và `u_aes_replica`. Bên trong `u_aes_replica` có bốn instance `u_lut`, `u_bram`, `u_composite` và `u_noise`, lần lượt liên kết đến các tệp RTL tương ứng. Cây hierarchy này là bằng chứng về cấu trúc tích hợp được Vivado nhận diện trong project Replica.



Hình 27: RTL Hierarchy của Replica cho thấy bốn lõi AES-256 được khởi tạo đồng thời dưới `u_aes_replica`.

Bốn lõi cùng tồn tại về mặt cấu trúc, nhưng cách phân phối Start không đồng nghĩa cả bốn cùng thực hiện một yêu cầu mã hóa. Theo `aes256_replica.sv`, tín hiệu `cipher_en` được kết hợp với `rm_select[1:0]` để tạo bốn Enable riêng; chỉ lõi được chọn mới nhận Start của yêu cầu đó. Các bus Key và Plaintext được kết nối tới cả bốn lõi, còn Ciphertext và Ready của lõi được chọn được đưa ra qua bộ chọn tổ hợp. Chi tiết hàm chọn đã phân tích ở Mục 3.5, vì vậy tại đây chỉ xem cơ chế này như một điều kiện tích hợp trong Synthesis và Implementation.

Việc giữ cả bốn lõi trong cùng một thiết kế có nghĩa Vivado phải tổng hợp và bố trí logic của tất cả các biến thể cùng các khối chọn. Đây là khác biệt cơ bản về tổ chức tài nguyên so với một RP chỉ chứa RM đang hoạt động của DPR. Dù vậy, không thể kết luận ngay tỷ lệ tăng tài nguyên từ số lõi, bởi sau tổng hợp công cụ có thể tối ưu logic hoặc ánh xạ mỗi biến thể sang các loại tài nguyên khác nhau. Việc so sánh định lượng cần được thực hiện trên các báo cáo Synthesis và Implementation tương ứng.

4.4.2 Trạng thái Synthesis, Implementation và Bitstream

Project Replica sử dụng quy trình hiện thực tĩnh. Khác với DPR, tab Design Runs không có Parent–Child Configurations để lần lượt thay thế RM. Hình 28 cho thấy `synth_1` ở trạng thái `synth_design Complete!`, `impl_1` ở trạng thái `write_bitstream Complete!`, và nhóm Out-of-Context `kv260_ps` báo `Submodule Runs Complete`. Kết quả này xác nhận Vivado đã hoàn thành quá trình tạo bitstream của phương án Replica tại thời điểm chụp.

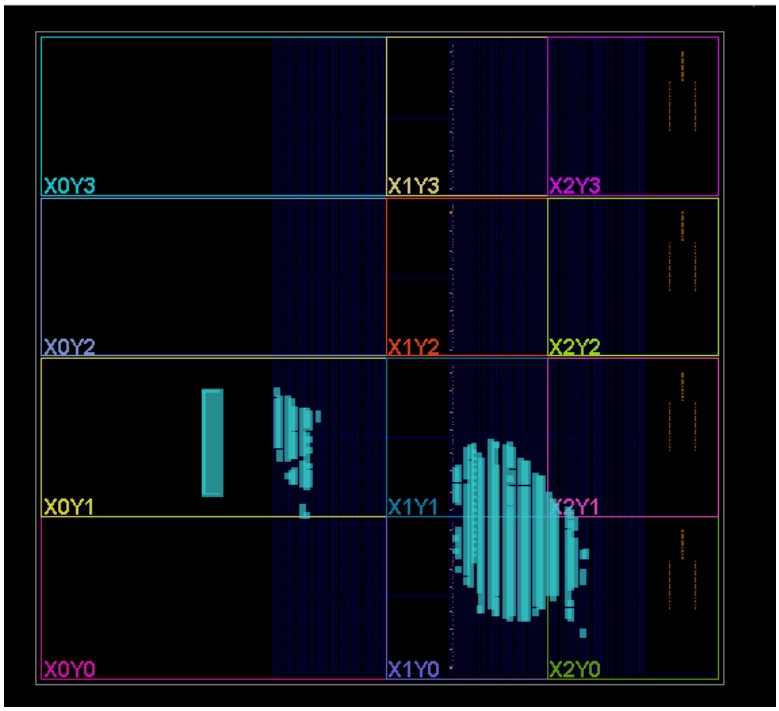
Name	Constraints	Status
✓ **synth_1** (active)	**constrs_1**	**synth_design Complete!**
✓ impl_1	constrs_1	write_bitstream Complete!
Out-of-Context Module Runs		
✓ kv260_ps		Submodule Runs Complete

Hình 28: Design Runs của Replica với `write_bitstream Complete!` tại Implementation Run `impl_1`.

Trạng thái `write_bitstream Complete!` có mức xác nhận khác với `route_design Complete!` quan sát ở ảnh DPR. Nó cho biết thao tác tạo file cấu hình của Run này đã hoàn tất, nhưng không tự xác nhận bitstream đã được nạp lên KV260 hoặc dữ liệu mã hóa chạy đúng trên thiết bị. Những bằng chứng liên quan đến nạp cấu hình và thực thi chương trình ARM/Linux cần được trình bày riêng ở Mục 4.5–4.6. Ngược lại, việc có bitstream cũng chưa thay thế cho báo cáo Timing, nên không nên suy ra Fmax hoặc Throughput từ trạng thái Design Runs.

4.4.3 Bố trí phần cứng trên FPGA

Hình 29 là Device View của phương án Replica sau Implementation. Các phần tử logic được đặt tại những vùng tài nguyên trên chip; khác với ảnh DPR, hình không đánh dấu Pblock dành cho một Reconfigurable Partition. Quan sát này phù hợp với cách tổ chức Replica thành thiết kế tĩnh chứa các lõi thường trú, nhưng không nên suy ra rằng mọi vùng có phần tử được tô màu đều thuộc AES, bởi trong cùng thiết kế còn có PS wrapper, bộ điều khiển AXI và các logic phụ trợ.



Hình 29: Device View sau Implementation của phương án Replica trên FPGA KV260.

Ảnh Device View cung cấp chứng cứ trực quan về việc triển khai thiết kế, song không đủ để gán từng cụm tài nguyên cho một Core cụ thể hoặc so sánh mức tiêu thụ tài nguyên với DPR. Việc đối chiếu đó đòi hỏi thêm báo cáo Utilization theo hierarchy hoặc các báo cáo thực thi của từng Configuration. Trong Chương 4, kết quả được xác nhận ở mức tổ chức thiết kế và tiến độ Implementation: bốn biến thể xuất hiện trong hierarchy, `impl_1` hoàn tất `write_bitstream`, và đã có Device View tương ứng.

Như vậy, hai phương án đã được hiện thực theo hai quy trình Vivado khác nhau: DPR sử dụng nhiều Configuration Runs trên cùng vùng RP, trong khi Replica triển khai bốn Core trong một thiết kế tĩnh. Điểm khác biệt ở đây được chứng minh bằng hierarchy, trạng thái Run và

floorplan thực tế; các so sánh về tài nguyên, Timing hoặc thời gian chuyển đổi sẽ được thực hiện ở các chương đánh giá để bảo đảm kết luận có số liệu đo kiểm.

4.5 Triển khai hệ thống trên Kria KV260

Các Bitstream được tạo từ Vivado được triển khai trên Kria KV260 thông qua môi trường ARM/Linux. Với DPR, Full Bitstream thiết lập cấu hình ban đầu, còn các Partial Bitstream được dùng để thay đổi RM. Replica tích hợp bốn lõi AES-256 trong một Full Bitstream và lựa chọn lõi bằng thanh ghi điều khiển.

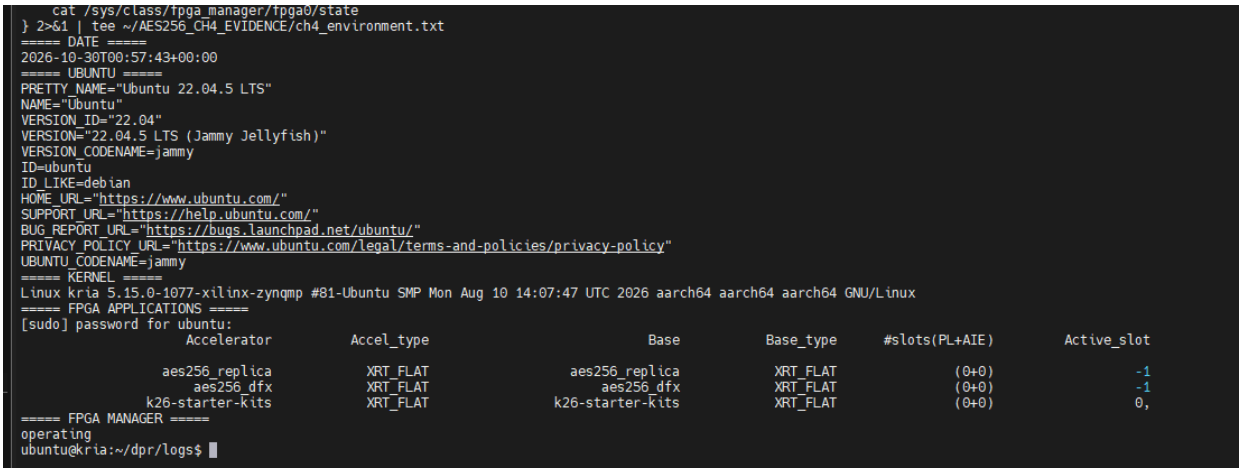
Quá trình triển khai sử dụng các công cụ `xmutil`, `fpgautil` và FPGA Manager.

4.5.1 Môi trường ARM/Linux trên KV260

Hệ thống thực nghiệm sử dụng Ubuntu 22.04.5 LTS với Linux Kernel `5.15.0-1077-xilinx-zynqmp` trên kiến trúc `aarch64`. Hệ điều hành được thực thi trên PS của Zynq UltraScale+ MPSoC, trong khi các lõi AES-256 được hiện thực trong Programmable Logic (PL). Chương trình phía PS chịu trách nhiệm lựa chọn biến thể, điều khiển quá trình mã hóa và thu nhận kết quả từ phần cứng.

Việc quản lý các ứng dụng FPGA được thực hiện thông qua công cụ `xmutil`. Công cụ này cho phép liệt kê các ứng dụng đã được đăng ký, nạp ứng dụng vào PL và giải phóng cấu hình đang hoạt động. Đối với việc thay đổi RM trong kiến trúc DPR, nhóm sử dụng thêm `fpgautil` để nạp trực tiếp các Partial Bitstream thông qua FPGA Manager.

Trạng thái cấu hình của FPGA có thể được đọc từ giao diện `/sys/class/fpga_manager/fpga0/state`. Trong quá trình thực nghiệm, giá trị `operating` cho biết FPGA Manager đang ở trạng thái vận hành. Tuy nhiên, trạng thái này không xác định riêng kiến trúc nào đang được nạp. Vì vậy, nhóm kết hợp kết quả kiểm tra FPGA Manager với thông tin `Active_slot` từ `xmutil listapps` và log nạp cấu hình để xác nhận trạng thái triển khai.



Hình 30: Thông tin Ubuntu, Linux Kernel và trạng thái FPGA Manager trên Kria KV260.

Hình 30 thể hiện kết quả kiểm tra môi trường thực thi từ Terminal. Việc xác nhận hệ điều hành, Kernel và khả năng truy cập FPGA Manager là điều kiện cần trước khi thực hiện các

thao tác nạp cấu hình và truy cập phần cứng từ chương trình C.

4.5.2 Chuẩn bị Bitstream và Device Tree Overlay

Các file cấu hình được tạo từ Vivado được chuyển sang hệ thống file Ubuntu để phục vụ quá trình triển khai. Đối với cả hai kiến trúc, Full Bitstream được sử dụng để thiết lập cấu hình phần cứng ban đầu. Riêng DPR có thêm bốn Partial Bitstream tương ứng với LUT, BRAM, Composite và Noise.

Các ứng dụng FPGA được tổ chức trong thư mục `/lib/firmware/xilinx/`. Mỗi ứng dụng có file cấu hình phần cứng, Device Tree Overlay và file mô tả ứng dụng `shell.json`. Device Tree Overlay được sử dụng để cung cấp thông tin mô tả phần cứng cần thiết cho Linux khi triển khai ứng dụng FPGA.

Bên cạnh thư mục ứng dụng, các Partial Bitstream của DPR được lưu tại `/home/ubuntu/` để chương trình điều khiển có thể truy cập trực tiếp trong quá trình thay đổi RM. Cách tổ chức này tách biệt thao tác nạp ứng dụng ban đầu với việc thay thế từng module trong vùng RP.

Bảng 17: Các file cấu hình được sử dụng trên KV260.

Thành phần	Tên file	Kích thước (byte)
DPR Full Bitstream	`aes256_dfx.bit.bin`	7 797 692
Replica Full Bitstream	`aes256_replica.bit.bin`	7 797 692
DPR – LUT	`aes256_lut_partial.bit.bin`	1 558 232
DPR – BRAM	`aes256_bram_partial.bit.bin`	1 558 232
DPR – Composite	`aes256_composite_partial.bit.bin`	1 558 232
DPR – Noise	`aes256_dummy_noise_partial.bit.bin`	1 558 232
DPR Device Tree Overlay	`aes256_dfx.dtbo`	2 328
Replica Device Tree Overlay	`aes256_replica.dtbo`	2 332

Các kích thước trong Bảng 17 được lấy từ danh sách file thực tế trên KV260. Bốn Partial Bitstream có cùng kích thước do được tạo cho cùng một vùng RP, mặc dù nội dung cấu hình và cách sử dụng tài nguyên của từng RM khác nhau. Full Bitstream của hai kiến trúc cũng có cùng kích thước trong bộ file hiện tại, nhưng điều này không có nghĩa hai thiết kế sử dụng cùng lượng tài nguyên logic.

Thông tin kích thước file được ghi nhận tại đây nhằm mô tả các thành phần đã triển khai. Việc đánh giá dung lượng cấu hình, chi phí tái cấu hình và sự đánh đổi giữa DPR với Replica sẽ được thực hiện trong Chương 6.

4.5.3 Nạp cấu hình và thay đổi Reconfigurable Module của DPR

Trong kiến trúc DPR, trước tiên hệ thống được cấu hình bằng ứng dụng `aes256_dfx`, chứa Static Region và vùng RP dành cho lõi AES-256. Sau khi cấu hình ban đầu được thiết lập, các RM có thể được thay thế mà không cần nạp lại toàn bộ thiết kế.

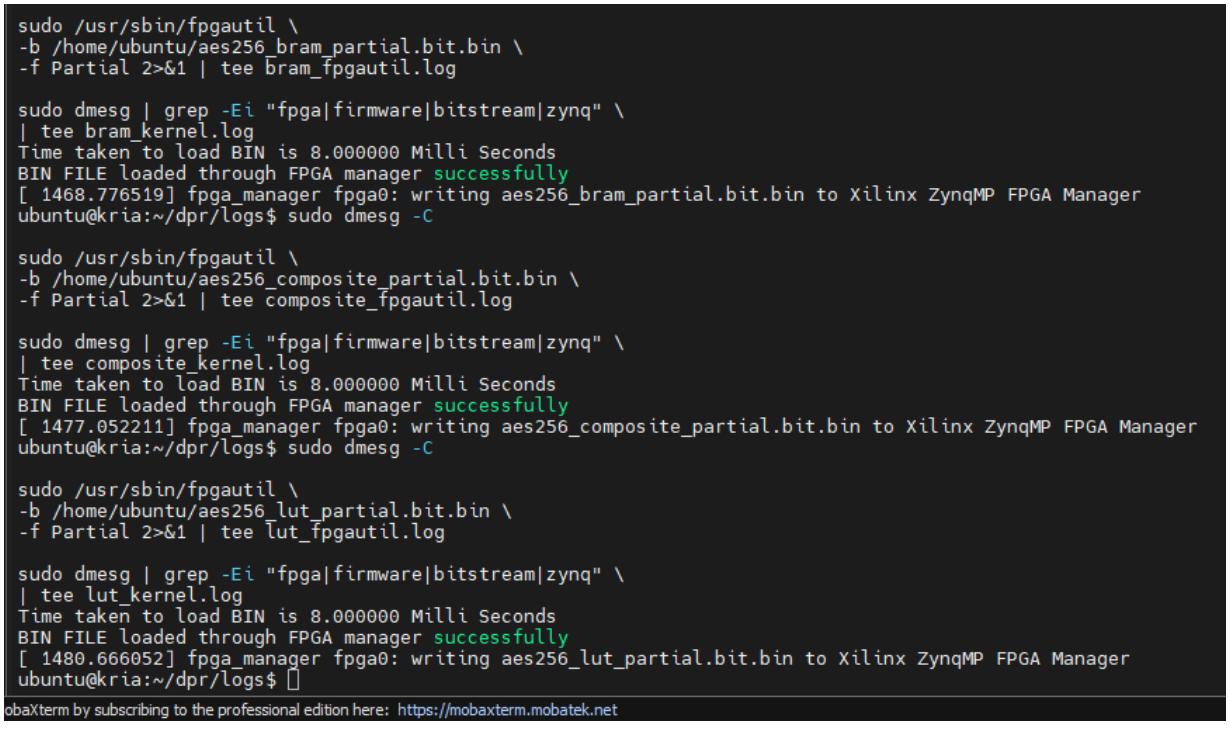
Nhóm sử dụng `fpgautil` với tùy chọn `Partial` để nạp file cấu hình của RM cần sử dụng. Ví dụ, thao tác nạp biến thể BRAM được thực hiện bằng lệnh:

```
sudo /usr/sbin/fpgautil \
    -b /home/ubuntu/aes256_bram_partial.bit.bin \
    -f Partial
```

Đối với các biến thể LUT, Composite và Noise, quy trình tương tự được thực hiện với Partial Bitstream tương ứng.

Trong quá trình thay đổi RM, phần mềm cần cô lập giao diện của vùng tái cấu hình trước khi tiến hành nạp. Cụ thể, chương trình điều khiển đặt tín hiệu `RM_HOLD` thông qua thanh ghi `RM_CONTROL`, đưa giao diện kết nối với RM về trạng thái an toàn. Sau khi `fpgautil` hoàn tất và FPGA Manager được xác nhận đang hoạt động, chương trình giải phóng `RM_HOLD` để module mới có thể tiếp nhận yêu cầu mã hóa.

Cơ chế cô lập này được thực hiện bằng logic điều khiển trong RTL của nhóm, không sử dụng một DFX Decoupler IP riêng biệt. Việc kiểm soát tín hiệu tại ranh giới RP giúp hạn chế truyền các yêu cầu Start hoặc dữ liệu không hợp lệ đến module trong thời gian tái cấu hình.



Hình 31: Log nạp Partial Bitstream của các biến thể AES-256 thông qua FPGA Manager trên KV260.

Hình 31 thể hiện các lần nạp Partial Bitstream cho LUT, BRAM và Composite. Trong cả ba trường hợp, công cụ hiển thị thông báo `BIN FILE loaded through FPGA manager successfully`. Nhóm cũng đã thực hiện quy trình tương ứng với biến thể Noise và ghi nhận kết quả nạp thành công.

Các kết quả này xác nhận rằng bốn RM có thể được nạp thông qua FPGA Manager trong môi trường Linux thực tế. Tuy nhiên, thông báo nạp thành công chỉ phản ánh trạng thái cấu hình;

việc xác định module có thực hiện đúng AES-256 sau khi được nạp cần kết hợp với phép kiểm chứng chức năng. Nội dung kiểm chứng đó được trình bày trong Chương 5.

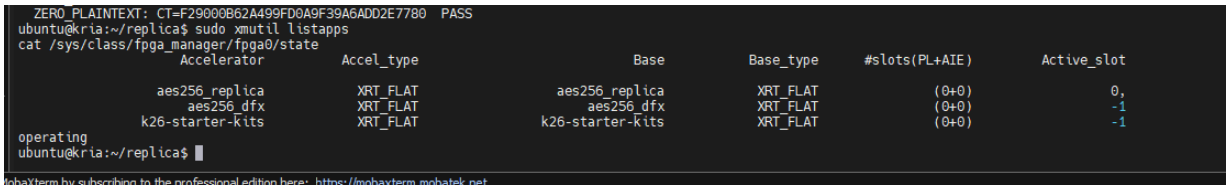
4.5.4 Nạp Full Bitstream của kiến trúc Replica

Đối với Replica, toàn bộ bốn lõi AES-256 được triển khai trong một cấu hình phần cứng tĩnh. Vì vậy, hệ thống chỉ cần nạp Full Bitstream của ứng dụng `aes256_replica` khi chuyển từ một ứng dụng FPGA khác sang kiến trúc Replica.

Quá trình quản lý ứng dụng được thực hiện bằng `xmutil`. Trước khi nạp Replica, ứng dụng FPGA đang chiếm vùng cấu hình được giải phóng. Sau đó, hệ thống nạp ứng dụng Replica bằng các lệnh:

```
sudo xmutil unloadapp
sudo xmutil loadapp aes256_replica
sudo xmutil listapps
```

Khi ứng dụng đã được nạp, nhóm kiểm tra danh sách ứng dụng để xác nhận `aes256_replica` đang được gán vào slot hoạt động. Kết quả cuối cùng cho thấy `Active_slot = 0` đối với Replica, trong khi `aes256_dfx` có giá trị `-1`. Đồng thời, FPGA Manager báo trạng thái `operating`.



Hình 32: Trạng thái `xmutil listapps` xác nhận ứng dụng Replica đang hoạt động tại slot 0 trên KV260.

Hình 32 cung cấp bằng chứng rằng thiết kế Replica đã được kích hoạt thay cho ứng dụng DPR. Đây là bước xác nhận cần thiết, bởi hai kiến trúc sử dụng cùng vùng địa chỉ AXI4-Lite. Nếu không kiểm tra ứng dụng đang hoạt động, một chương trình đọc được Ciphertext hợp lệ chưa đủ để xác định kết quả đó đến từ DPR hay Replica.

Sau khi Full Bitstream Replica được nạp, việc lựa chọn giữa LUT, BRAM, Composite và Noise không yêu cầu thao tác cấu hình lại FPGA. Thay vào đó, chương trình phía PS ghi giá trị lựa chọn vào thanh ghi `RM_CONTROL`. Phương pháp này giúp tách thao tác quản lý ứng dụng FPGA khỏi các giao dịch mã hóa thông thường.

Như vậy, cả hai phương án đã được triển khai trên cùng nền tảng KV260. DPR thay đổi phần cứng đang hoạt động bằng Partial Bitstream, còn Replica sử dụng Full Bitstream cố định và lựa chọn lõi thông qua thanh ghi. Cách điều khiển cụ thể của mỗi phương án được phân tích trong phần tiếp theo.

4.6 Chương trình điều khiển ARM/Linux

Nhóm xây dựng hai chương trình C chạy trên Ubuntu của KV260: `aes256_dpr_control.c` và `aes256_replica_control.c`. Cả hai sử dụng cùng giao diện AXI4-Lite để trao đổi dữ liệu

AES-256. Chương trình DPR quản lý việc nạp Partial Bitstream, còn Replica lựa chọn Core bằng thanh ghi điều khiển.

4.6.1 Truy cập các thanh ghi AXI4-Lite từ Linux

Khối điều khiển AES-256 được ánh xạ vào không gian địa chỉ vật lý của PS tại địa chỉ cơ sở `0xA0000000`, với vùng địa chỉ được cấp phát có kích thước `0x10000` byte. Để truy cập trực tiếp vùng thanh ghi này từ chương trình User Space, nhóm sử dụng thiết bị `/dev/mem` kết hợp với lời gọi hệ thống `mmap()`.

Chương trình mở `/dev/mem` với các cờ `O_RDWR` và `O_SYNC`, sau đó ánh xạ vùng địa chỉ AXI sang không gian địa chỉ ảo của tiến trình. Con trỏ kết quả được khai báo dưới dạng `volatile uint32_t *`, cho phép truy cập các thanh ghi 32 bit tại các Offset tương ứng.

Đối với một Offset tính theo byte, vị trí thanh ghi được xác định bằng cách chia Offset cho bốn. Do đó, các thao tác đọc và ghi được thực hiện bằng các hàm `mmio_read()` và `mmio_write()` thay vì truy cập trực tiếp địa chỉ vật lý trong toàn bộ chương trình.

Cả hai chương trình đều sử dụng cùng Register Map đã được thiết kế tại Mục 3.4. Những vùng thanh ghi trực tiếp tham gia quá trình điều khiển được tóm tắt trong Bảng 18.

Bảng 18: Các nhóm thanh ghi AXI4-Lite được sử dụng bởi chương trình ARM/Linux.

Thanh ghi	Offset	Chức năng trong chương trình
COMMAND	`0x00`	Phát yêu cầu START để kích hoạt một giao dịch mã hóa.
STATUS	`0x04`	Kiểm tra trạng thái READY và DONE của AES Core.
RM_CONTROL	`0x08`	Điều khiển HOLD và lựa chọn biến thể trong kiến trúc Replica.
KEY0–KEY7	`0x10-0x2C`	Ghi khóa AES-256 dưới dạng tám từ 32 bit.
PT0–PT3	`0x30-0x3C`	Ghi Plaintext 128 bit dưới dạng bốn từ 32 bit.
CT0–CT3	`0x40-0x4C`	Đọc Ciphertext 128 bit sau khi mã hóa hoàn tất.

Trong chương trình C, khóa 256 bit được chia thành tám từ 32 bit và ghi lần lượt vào các thanh ghi KEY0 đến KEY7. Tương tự, Plaintext được ghi vào bốn thanh ghi PT0 đến PT3. Thứ tự các từ được thiết lập phù hợp với cách ghép bus trong AXI Controller, theo đó thanh ghi có chỉ số lớn hơn chứa các bit có trọng số cao hơn của dữ liệu.

Sau khi chuẩn bị Key và Plaintext, phần mềm ghi giá trị `0x1` vào COMMAND để kích hoạt AES Core. Chương trình theo dõi thanh ghi STATUS, trong đó bit 0 biểu diễn DONE và bit 1 biểu diễn READY. Khi giao dịch hoàn tất, bốn từ Ciphertext được đọc từ CT0 đến CT3 và ghép lại thành dữ liệu 128 bit.

Để tránh trường hợp phần cứng không phản hồi làm chương trình bị treo vô thời hạn, hai chương trình sử dụng hàm kiểm tra trạng thái có giới hạn thời gian. Thời gian chờ được theo dõi bằng `CLOCK_MONOTONIC_RAW`, với ngưỡng Timeout được thiết lập trong mã nguồn. Các thao tác truy cập thanh ghi cũng sử dụng Memory Barrier để duy trì thứ tự các lệnh đọc và ghi trong quá trình điều khiển.

Cơ chế giao tiếp này cho phép phần mềm ARM/Linux sử dụng cùng một giao diện để điều khiển cả hai kiến trúc, trong khi sự khác biệt về tổ chức phần cứng được xử lý bởi các thao tác lựa chọn hoặc tái cấu hình tương ứng.

4.6.2 Chương trình điều khiển kiến trúc DPR

Chương trình `aes256_dpr_control.c` cho phép lựa chọn một trong bốn RM gồm LUT, BRAM, Composite và Noise thông qua tham số dòng lệnh. Mỗi lựa chọn được ánh xạ đến đường dẫn Partial Bitstream tương ứng trong hệ thống file Linux.

Sau khi xác định biến thể đích, chương trình mở vùng nhớ AXI4-Lite và bắt đầu quá trình thay đổi cấu hình. Trình tự thực hiện gồm bốn bước chính.

Trước tiên, chương trình ghi `0x1` vào RM_CONTROL để kích hoạt HOLD. Tín hiệu này được đưa xuống PL và tác động đến logic cô lập tại ranh giới RP. Trong thời gian HOLD được kích hoạt, các tín hiệu giao tiếp với RM được đặt về trạng thái xác định nhằm tránh phát sinh giao dịch mã hóa trong khi cấu hình đang thay đổi.

Tiếp theo, chương trình tạo một tiến trình con bằng `fork()` và gọi `fpgautil` thông qua `execl()`. Công cụ nhận đường dẫn Partial Bitstream và tùy chọn `Partial` để thực hiện thao tác nạp. Tiến trình chính sử dụng `waitpid()` để chờ tiến trình nạp kết thúc và kiểm tra mã trả về.

Sau khi `fpgautil` hoàn tất, chương trình đọc trạng thái FPGA Manager từ `/sys/class/fpga_manager/fpga0/state`. Chỉ khi quá trình nạp không báo lỗi và FPGA Manager ở trạng thái `operating`, chương trình mới tiếp tục giải phóng HOLD.

Cuối cùng, phần mềm ghi `0x0` vào RM_CONTROL và đọc lại thanh ghi để xác nhận giá trị điều khiển đã được cập nhật. Tại thời điểm này, RM mới được cho phép tiếp nhận các giao dịch mã hóa.

Như vậy, chương trình DPR không chỉ lựa chọn chức năng AES mà còn quản lý trình tự cấu hình phần cứng. Việc kết hợp HOLD, nạp Partial Bitstream, kiểm tra FPGA Manager và RELEASE tạo thành một quy trình điều khiển thống nhất giữa PS và PL.

Trong trường hợp công cụ nạp hoặc FPGA Manager báo lỗi, chương trình không tiếp tục thực hiện phần kiểm chứng AES. Thay vào đó, nó thông báo lỗi và kết thúc quá trình điều khiển. Cách xử lý này giúp phân biệt lỗi cấu hình phần cứng với lỗi phát sinh trong quá trình mã hóa.

Chương trình còn hỗ trợ tùy chọn `-verify`. Khi tùy chọn này được kích hoạt, phần mềm sử dụng hai bộ dữ liệu kiểm tra gồm `FIPS_STANDARD` và `ZERO_PLAINTEXT`. Với mỗi bộ dữ liệu, chương trình ghi Key và Plaintext, kích hoạt mã hóa, chờ DONE rồi so sánh Ciphertext nhận được với giá trị kỳ vọng.

```
ubuntu@kria:~/dpr$ sudo ./aes256_dpr_control lut --verify 2>&1 | \
tee aes256_dpr_lut_verify.log
Time taken to load BIN is 1192.000000 Milli Seconds
BIN FILE loaded through FPGA manager successfully
DPR target     : LUT
Partial file   : /home/ubuntu/aes256_lut_partial.bit.bin
RM readback    : 0x00000000
DPR time       : 1232160353 ns (1232160.353 us, 1232.160353 ms)
AES-256 verification:
  FIPS_STANDARD: CT=8EA2B7CA516745BFEAFC49904B496089  PASS
  ZERO_PLAINTEXT: CT=F29000B62A499FD0A9F39A6ADD2E7780  PASS
ubuntu@kria:~/dpr$
```

Hình 33: Kết quả chạy chương trình điều khiển DPR với biến thể LUT và tùy chọn `-verify`.

Hình 33 thể hiện kết quả thực thi chương trình trên KV260. Sau khi nạp Partial Bitstream LUT, thanh ghi RM_CONTROL được đọc lại với giá trị `0x00000000`. Hai bộ dữ liệu kiểm tra đều trả về Ciphertext đúng với giá trị kỳ vọng và được chương trình đánh dấu PASS.

Kết quả này xác nhận chuỗi thao tác điều khiển từ phần mềm Linux đến phần cứng AES-256 đã được thực hiện thành công đối với biến thể LUT. Việc kiểm chứng chức năng trên toàn bộ bốn RM và các Test Vector NIST được trình bày riêng trong Chương 5.

4.6.3 Chương trình điều khiển kiến trúc Replica

Chương trình `aes256_replica_control.c` được xây dựng trên cùng giao diện AXI4-Lite, nhưng không thực hiện thao tác nạp Partial Bitstream. Điều này xuất phát từ đặc điểm của kiến trúc Replica: bốn biến thể AES-256 đã cùng tồn tại trong PL sau khi Full Bitstream được nạp.

Để lựa chọn Core, phần mềm ghi hai giá trị liên tiếp vào RM_CONTROL. Giá trị đầu tiên kích hoạt HOLD cùng mã lựa chọn, trong khi giá trị thứ hai giải phóng HOLD và giữ nguyên mã lựa chọn Core. Theo thiết kế thanh ghi, bit 0 dùng cho HOLD và hai bit [2:1] dùng cho `rm_select[1:0]`.

Các giá trị được sử dụng trong chương trình được trình bày ở Bảng 19.

Bảng 19: Giá trị điều khiển lựa chọn Core trong kiến trúc Replica.

Core	`rm_select`	HOLD	RELEASE
LUT	00	0x1	0x0
BRAM	01	0x3	0x2
Composite	10	0x5	0x4
Noise	11	0x7	0x6

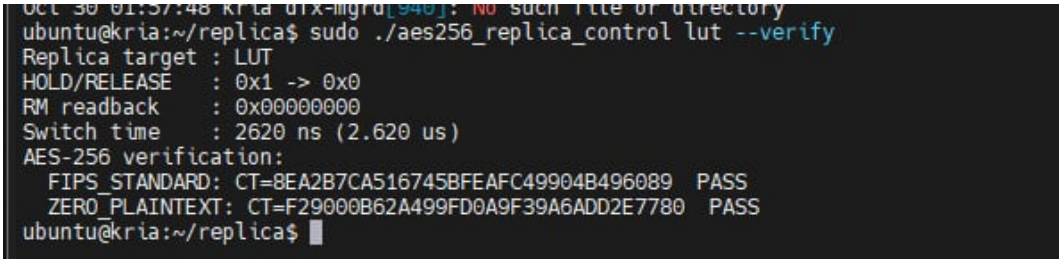
Sau khi ghi giá trị RELEASE, chương trình đọc lại RM_CONTROL và so sánh với giá trị mong đợi. Nếu Readback khớp, Core được xem là đã được lựa chọn theo thanh ghi điều khiển. Phần mềm sau đó có thể thực hiện giao dịch AES với Core tương ứng.

Khác với DPR, toàn bộ thao tác lựa chọn trong Replica được thực hiện thông qua các giao dịch ghi và đọc thanh ghi AXI4-Lite. Không có tiến trình `fpgautil`, thao tác thay thế RM hay

bước kiểm tra kết quả nạp Partial Bitstream trong mỗi lần chuyển Core.

Tuy nhiên, chương trình phải duy trì mã lựa chọn Core trong suốt một giao dịch mã hóa. Do kết quả Ciphertext và READY được chọn theo `rm_select`, việc thay đổi lựa chọn khi một Core đang xử lý có thể khiến phần mềm đọc dữ liệu hoặc trạng thái của một Core khác. Vì vậy, trình tự điều khiển được tổ chức theo nguyên tắc hoàn tất giao dịch hiện tại trước khi chọn biến thể tiếp theo.

Tương tự chương trình DPR, Replica Control hỗ trợ tùy chọn `-verify` để thực hiện kiểm chứng nhanh sau khi chọn Core. Chương trình sử dụng cùng Key và hai bộ Plaintext kiểm tra, sau đó đối chiếu Ciphertext nhận được với dữ liệu kỳ vọng.



Hình 34: Kết quả chạy chương trình Replica Control với biến thể LUT và tùy chọn `-verify`.

Trong Hình 34, chương trình ghi cặp giá trị HOLD/RELEASE `0x1` và `0x0` để lựa chọn LUT. Giá trị đọc lại từ RM_CONTROL là `0x00000000`, phù hợp với lựa chọn đã gửi.

Hai phép kiểm tra `FIPS_STANDARD` và `ZERO_PLAINTEXT` đều được đánh dấu PASS. Kết hợp với trạng thái `Active_slot = 0` của ứng dụng Replica đã xác nhận ở Mục 4.5, kết quả này cho thấy chương trình ARM/Linux có thể giao tiếp với thiết kế Replica đang hoạt động trên KV260.

4.6.4 Đối chiếu quy trình điều khiển

Hai chương trình C sử dụng chung giao diện MMIO và quy trình trao đổi Key, Plaintext, Ciphertext qua AXI4-Lite. Khác biệt nằm ở bước chuẩn bị Core: DPR thực hiện HOLD, nạp Partial Bitstream, kiểm tra FPGA Manager rồi RELEASE; Replica lựa chọn Core bằng cách ghi `RM_CONTROL`.

Cả hai chương trình hỗ trợ kiểm chứng Ciphertext sau khi chọn biến thể. Thời gian thực thi các thao tác điều khiển bao gồm chi phí Linux và giao tiếp AXI, không đại diện cho độ trễ xử lý thuần của AES Core.

Các phép kiểm chứng NIST và đo hiệu năng được trình bày lần lượt trong Chương 5 và Chương 6.

CHƯƠNG 5: KIỂM CHỨNG VÀ ĐÁNH GIÁ THIẾT KẾ AES256

5.1 Phương pháp và phạm vi kiểm chứng

Việc đánh giá thiết kế AES-256 sử dụng ba nguồn dữ liệu: RTL Simulation, kiểm chứng chức năng trên Kria KV260 và báo cáo Implementation của Vivado. Mô phỏng xác nhận hoạt động mã hóa và tín hiệu điều khiển; phép thử trên KV260 kiểm tra giao dịch qua PS–AXI–PL; các báo cáo Vivado cung cấp số liệu tài nguyên và Timing sau Routing.

5.1.1 Tiêu chí đánh giá chức năng

Ở mỗi phép thử, một khóa 256 bit và một khối Plaintext 128 bit được đưa đến lõi mã hóa. Khi nhận Start, lõi thực hiện AES-256 Encryption và trả về Ciphertext 128 bit cùng tín hiệu Ready. Giá trị quan sát được chỉ được chấp nhận khi tác vụ hoàn tất; các trạng thái trung gian của bus Ciphertext trong lúc xử lý không được xem là kết quả hợp lệ. Một testcase đạt yêu cầu khi phần cứng sẵn sàng nhận lệnh, hoàn thành giao dịch trong giới hạn Timeout và trả về Ciphertext trùng giá trị tham chiếu:

$$\mathrm{PASS}_i = (\mathrm{READY}_i = 1) \wedge (\mathrm{DONE}_i = 1) \wedge (C_i^{\mathrm{actual}} = C_i^{\mathrm{expected}}) \wedge (\mathrm{Timeout}_i = 0). \tag{29}$$

Ngoài giá trị bản mã, mô phỏng còn quan sát Reset, xung Start và số chu kỳ từ lúc yêu cầu được chấp nhận đến khi hoàn tất. Với Replica, cần kiểm tra thêm việc lựa chọn lần lượt bốn lõi bằng `rm_select[1:0]`; với DPR trên board, phải xác nhận đúng RM đã được nạp trước khi mã hóa. Các phép đo độ trễ hoặc thao tác chuyển đổi cấu hình thực tế không được suy ra từ Waveform RTL mà cần ghi nhận riêng từ chương trình chạy trên KV260.

5.1.2 Bộ dữ liệu chuẩn và phạm vi Test Vector

Phần kiểm chứng sử dụng một Known Answer Test của AES-256 với khóa tuần tự 256 bit và Plaintext 128 bit. Đây là cặp dữ liệu đã được đưa vào các Testbench LUT, BRAM và Replica của nhóm. Ciphertext tham chiếu tương ứng là `8EA2B7CA516745BFEAFC49904B496089`. Test Vector này phù hợp để quan sát trực tiếp một giao dịch mã hóa hoàn chỉnh trên Waveform và kiểm tra cơ chế Start–Ready.

Để mở rộng phạm vi, nhóm có bốn tệp `.rsp` của bộ CAVS 11.1 dành cho AES-256 ở chế độ ECB: `ECBGFSbox256`, `ECBKeySbox256`, `ECBVarKey256` và `ECBVarTxt256`. Các tệp đều có hai phần `[ENCRYPT]` và `[DECRYPT]`, nhưng RTL hiện tại chỉ thực hiện Encryption. Vì vậy, số trường hợp cần sử dụng được thống kê từ phần `[ENCRYPT]` của từng tệp, không cộng gộp cả phần giải mã.

Bảng 20: Số lượng testcase AES-256 Encryption chuẩn NIST

Tệp dữ liệu	Số testcase	Đặc điểm bộ dữ liệu
ECBGFSbox256.rsp	5	Các mẫu kiểm tra nhóm phép biến đổi AES.
ECBKeySbox256.rsp	16	Các mẫu liên quan đến khóa và phép thế byte.
ECBVarKey256.rsp	256	Thay đổi khóa trong các trường hợp kiểm thử.
ECBVarTxt256.rsp	128	Thay đổi Plaintext trong các trường hợp kiểm thử.
Tổng cộng	**405**	**405 testcase ENCRYPT, không tính DECRYPT.**

Bảng 20 tổng hợp 405 Test Vector Encryption của bốn bộ dữ liệu NIST AESAVS. Mỗi biến thể AES-256 được kiểm chứng bằng toàn bộ tập dữ liệu này trên KV260; kết quả được trình bày trong Mục 5.3. Đây là phép kiểm chứng do nhóm thực hiện, không phải chứng nhận chính thức của NIST.

5.1.3 Nguồn dữ liệu đánh giá phần cứng

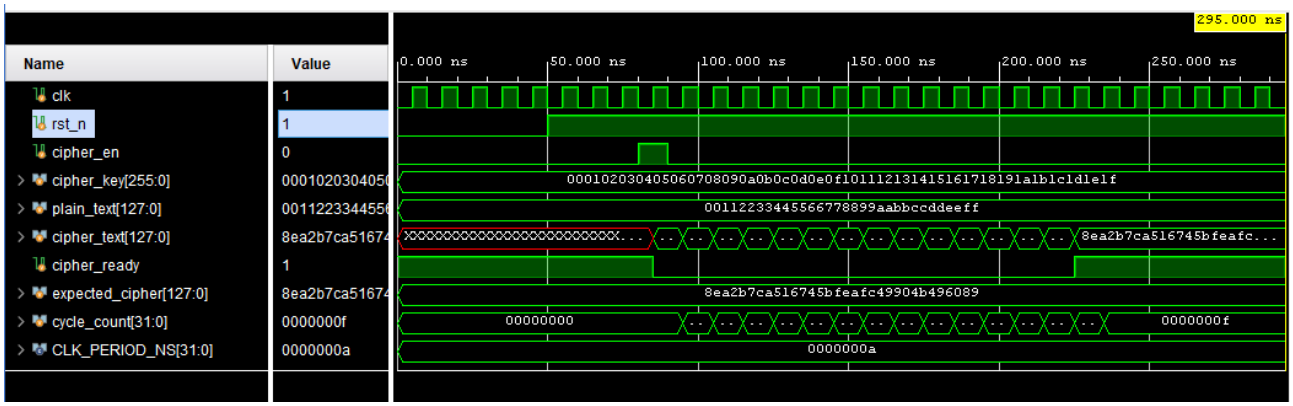
Phần đánh giá tài nguyên và thời gian sử dụng hai loại báo cáo Vivado: Utilization theo hierarchy và Timing Summary. Nhóm đã xuất dữ liệu ở trạng thái `Design State: Routed` từ bốn Implementation Runs của DPR và một Implementation Run của Replica, cùng thiết bị đích `xck26-sfvc784-2LV-c`. Các số liệu sẽ được tách thành hai cấp: toàn bộ hệ thống PS–PL và phần lõi AES.

5.2 Kiểm chứng chức năng bằng mô phỏng RTL

Mô phỏng RTL được thực hiện trước khi đối chiếu kết quả trên FPGA nhằm kiểm tra trực tiếp giao thức điều khiển của lõi mã hóa. Ba nhóm bằng chứng hiện có gồm Testbench độc lập cho LUT, Testbench độc lập cho BRAM và Testbench Replica với bốn lần lựa chọn lõi. Các hình được lấy từ Vivado Simulator và phần log của Testbench; chúng phản ánh kết quả tại mức RTL, không phải dữ liệu thời gian đo qua giao diện AXI trên KV260.

5.2.1 Kiểm chứng lõi AES-256 LUT

Trong lần kiểm chứng LUT, Testbench thiết lập Clock chu kỳ 10 ns, đặt Key và Plaintext theo mẫu AES-256 chuẩn, sau đó tạo xung `cipher_en`. Trên Waveform ở Hình 35, tín hiệu `cipher_ready` thay đổi theo trạng thái xử lý, còn `cipher_text` đi đến giá trị cuối cùng khi tác vụ hoàn tất. Các trạng thái chuyển tiếp của bus dữ liệu trong thời gian chờ không phải cơ sở để đối chiếu tính đúng đắn.



Hình 35: Waveform RTL của AES-256 LUT: Start, Ready và dữ liệu mã hóa với chu kỳ Clock 10 ns.

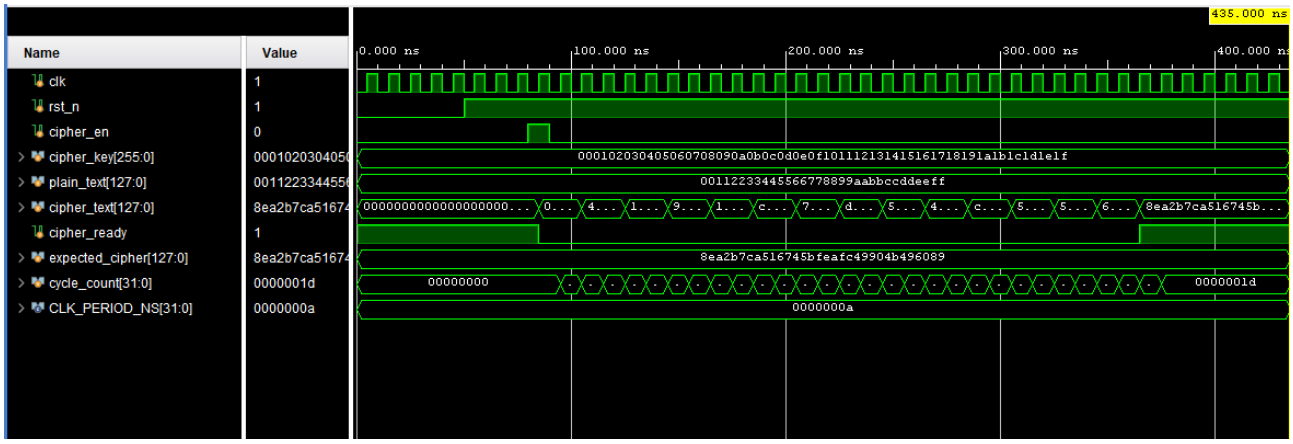
Log ở Hình 36 ghi `ACTUAL` trùng hoàn toàn với `EXPECTED`, đồng thời cho biết độ trễ 15 chu kỳ và kết quả `PASS`. Do đó, trong phép thử đã quan sát, phiên bản LUT tạo đúng Ciphertext và hoàn thành theo số chu kỳ do Testbench ghi nhận.

```
# run 1000ns
==================================================
 AES-256 LUT RTL VERIFICATION
==================================================
KEY       = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PLAINTEXT= 00112233445566778899aabbccddeeff
EXPECTED = 8ea2b7ca516745bfeafc49904b496089
--------------------------------------------------
ACTUAL    = 8ea2b7ca516745bfeafc49904b496089
LATENCY   = 15 clock cycles
[PASS] AES-256 LUT ciphertext correct
OVERALL RESULT : PASS
==================================================
```

Hình 36: Log Testbench AES-256 LUT: kết quả đối chiếu Ciphertext và độ trễ 15 chu kỳ.

5.2.2 Kiểm chứng lõi AES-256 BRAM

Cùng dữ liệu đầu vào được áp dụng cho biến thể BRAM để bảo đảm phép đối chiếu chức năng nhất quán. Hình 37 cho thấy sau xung Start, lõi cần một khoảng xử lý trước khi `cipher_ready` trở lại trạng thái hoàn tất. Khác với LUT, biến thể này ghi nhận nhiều chu kỳ hơn; các giá trị xuất hiện trên `cipher_text` trong lúc đang xử lý không được hiểu là những Ciphertext hợp lệ độc lập.



Hình 37: Waveform RTL AES-256 BRAM với tín hiệu Start–Ready và bus Ciphertext.

Log ở Hình 38 cho thấy cùng một Ciphertext tham chiếu, độ trễ 29 chu kỳ và kết quả `PASS`. Như vậy, với mẫu kiểm tra này, BRAM cho kết quả đúng nhưng độ trễ theo bộ đếm của Testbench lớn hơn LUT 14 chu kỳ. Với Clock mô phỏng 10 ns, hai giá trị đếm tương ứng 150 ns và 290 ns nếu quy đổi theo chu kỳ; đây không phải kết quả đo thời gian thực thi bằng chương trình ARM/Linux.

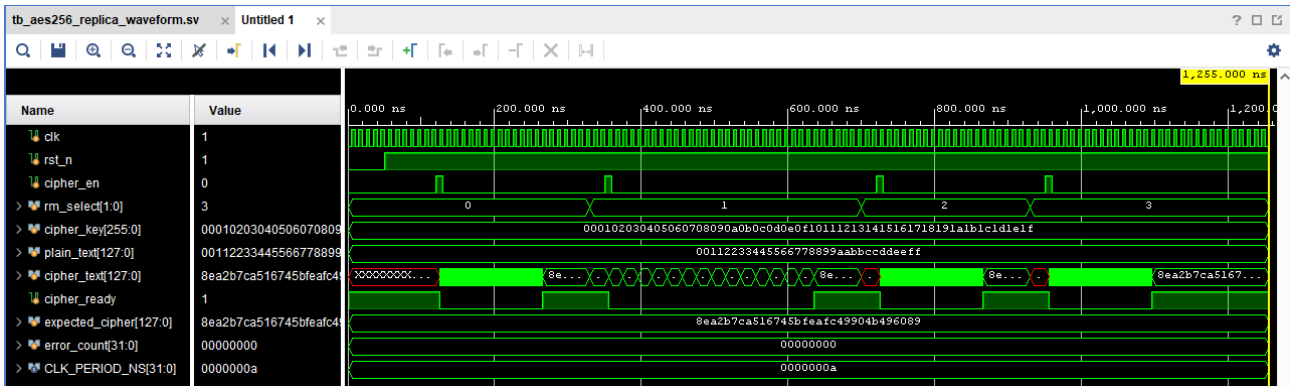
```
# run 1000ns
====================================================
 AES-256 BRAM RTL VERIFICATION
====================================================
KEY       = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PLAINTEXT = 00112233445566778899aabbccddeeff
EXPECTED  = 8ea2b7ca516745bfeafc49904b496089
----------------------------------------------------
ACTUAL    = 8ea2b7ca516745bfeafc49904b496089
LATENCY   = 29 clock cycles
[PASS] AES-256 BRAM ciphertext correct
OVERALL RESULT : PASS
====================================================
```

Hình 38: Log Testbench AES-256 BRAM: Ciphertext đúng và độ trễ 29 chu kỳ.

5.2.3 Kiểm chứng cơ chế chọn lõi trong Replica

Trong Replica, bốn lõi LUT, BRAM, Composite và Noise tồn tại đồng thời ở mức RTL. Testbench lần lượt đặt `rm_select` bằng `00`, `01`, `10` và `11`. Theo mã `aes256_replica.sv`, chỉ lõi được chọn nhận xung Start; Ciphertext và Ready được ghép ra đầu ra chung tương ứng với giá trị chọn. Do bộ chọn là tổ hợp, Testbench phải giữ `rm_select` ổn định cho đến khi kết thúc giao dịch mã hóa đang kiểm tra.

Hình 39 thể hiện tuần tự bốn giá trị lựa chọn trong cùng một phiên mô phỏng. Các xung `cipher_en` xuất hiện cho từng yêu cầu mã hóa và `cipher_ready` được sử dụng để xác định thời điểm đọc kết quả. Tín hiệu `error_count` trong hình giữ giá trị bằng không ở trạng thái quan sát cuối cùng.



Hình 39: Waveform kiểm chứng Replica khi `rm_select` lần lượt nhận bốn giá trị từ 0 đến 3.

Log tổng hợp ở Hình 40 ghi nhận bốn lõi được lựa chọn, cả bốn lần kiểm tra đều đạt, không có trường hợp thất bại và kết quả tổng hợp là `PASS`. Trong cả bốn lượt, Ciphertext trả về trùng với giá trị tham chiếu đã sử dụng cho Testbench LUT và BRAM. Kết quả xác nhận hoạt động chức năng của đường chọn lõi và dữ liệu đầu ra trong *một bộ Test Vector* của Testbench Replica; nó chưa chứng minh toàn bộ 405 vector CAVS 11.1 đã được mô phỏng ở mức RTL hoặc đã PASS trên FPGA thực tế.

```
==================================================
 AES-256 REPLICA RTL VERIFICATION
==================================================
KEY       = 000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f
PLAINTEXT = 00112233445566778899aabbccddeeff
EXPECTED  = 8ea2b7ca516745bfeafc49904b496089
==================================================
[LUT]       CT = 8ea2b7ca516745bfeafc49904b496089
[PASS] LUT
[BRAM]      CT = 8ea2b7ca516745bfeafc49904b496089
[PASS] BRAM
[COMPOSITE] CT = 8ea2b7ca516745bfeafc49904b496089
[PASS] COMPOSITE
[NOISE]     CT = 8ea2b7ca516745bfeafc49904b496089
[PASS] NOISE

==================================================
 REPLICA VERIFICATION SUMMARY
==================================================
Selected cores : 4
Passed         : 4
Failed         : 0
OVERALL RESULT : PASS
==================================================
```

Hình 40: Log kiểm chứng bốn lõi trong Replica: 4 lần PASS và không phát hiện sai khác Ciphertext.

Bảng 21: Tổng hợp kết quả RTL Simulation đã có bằng chứng trực tiếp.

Testbench	Phạm vi đã kiểm tra	Latency ghi nhận	Kết quả
LUT độc lập	Một Known Answer Test AES-256	15 chu kỳ	PASS
BRAM độc lập	Cùng Known Answer Test	29 chu kỳ	PASS
Replica	Bốn lựa chọn LUT, BRAM, Composite, Noise với cùng Test Vector	Chưa có số theo từng lõi	4/4 PASS

5.3 Kiểm chứng chức năng trên phần cứng Kria KV260

Sau khi các thiết kế vượt qua những phép thử RTL cơ bản, nhóm tiếp tục kiểm chứng hoạt động mã hóa AES-256 trực tiếp trên Kria KV260. Mục tiêu của giai đoạn này không chỉ là xác nhận Ciphertext đúng, mà còn kiểm tra khả năng thực hiện một giao dịch mã hóa hoàn chỉnh thông qua phần mềm ARM/Linux, giao diện AXI4-Lite và phần cứng trong PL.

Hai kiến trúc DPR và Replica được kiểm chứng bằng cùng bộ dữ liệu NIST AESAVS, gồm bốn Test Suites đã trình bày trong Mục 5.1. Mỗi biến thể AES-256 phải thực hiện 405 trường hợp Encryption, tương ứng với các cặp Key, Plaintext và Ciphertext tham chiếu. Việc áp dụng cùng một bộ dữ liệu cho tám cấu hình kiểm chứng giúp đối chiếu tính đúng đắn giữa các biến thể mà không phụ thuộc vào phương thức triển khai phần cứng.

5.3.1 Phương pháp kiểm chứng trên hệ thống ARM/Linux

Nhóm phát triển hai chương trình kiểm chứng bằng ngôn ngữ C, gồm `aes256_dpr_nist.c` và `aes256_replica_nist.c`. Cả hai chương trình đều truy cập các thanh ghi AXI4-Lite thông qua `/dev/mem` và `mmap()`, sử dụng địa chỉ cơ sở `0xA0000000`. Sự khác biệt giữa hai chương trình nằm ở bước lựa chọn biến thể AES-256 trước khi thực hiện kiểm chứng.

Trong DPR, chương trình lần lượt nạp bốn Partial Bitstream thông qua `fpgautil`. Sau mỗi lần nạp, chương trình xác nhận trạng thái FPGA Manager và tiến hành kiểm chứng toàn bộ 405 Test Vector trên RM đang hoạt động. Đối với Replica, bốn Core đã được tích hợp trong cùng một Full Bitstream. Chương trình lần lượt thay đổi thanh ghi `RM_CONTROL` để lựa chọn Core, sau đó thực hiện cùng bộ Test Vector mà không cần tái cấu hình FPGA.

Dữ liệu kiểm chứng được đọc trực tiếp từ các file `.rsp` trong thư mục `/home/ubuntu/benchmarks`. Chương trình phân tích các trường `COUNT`, `KEY`, `PLAINTEXT` và `CIPHERTEXT`, đồng thời chỉ xử lý dữ liệu Encryption. Phần Decryption trong các file NIST không được thực thi vì các lõi phần cứng hiện tại được xây dựng cho chức năng AES-256 Encryption.

Đối với mỗi Test Vector, phần mềm kiểm tra trạng thái READY, ghi Key và Plaintext vào các thanh ghi đầu vào, sau đó phát yêu cầu START. Khi trạng thái DONE được xác nhận, chương trình đọc bốn thanh ghi Ciphertext và đối chiếu từng từ 32 bit với giá trị tham chiếu.

Các chương trình sử dụng giới hạn Timeout 100 ms cho thao tác chờ READY và DONE. Kết quả của từng testcase được phân loại thành PASS, FAIL, READY_TIMEOUT hoặc

DONE_TIMEOUT. PASS chỉ được ghi nhận khi phần cứng hoàn thành giao dịch và toàn bộ 128 bit Ciphertext trùng với giá trị kỳ vọng.

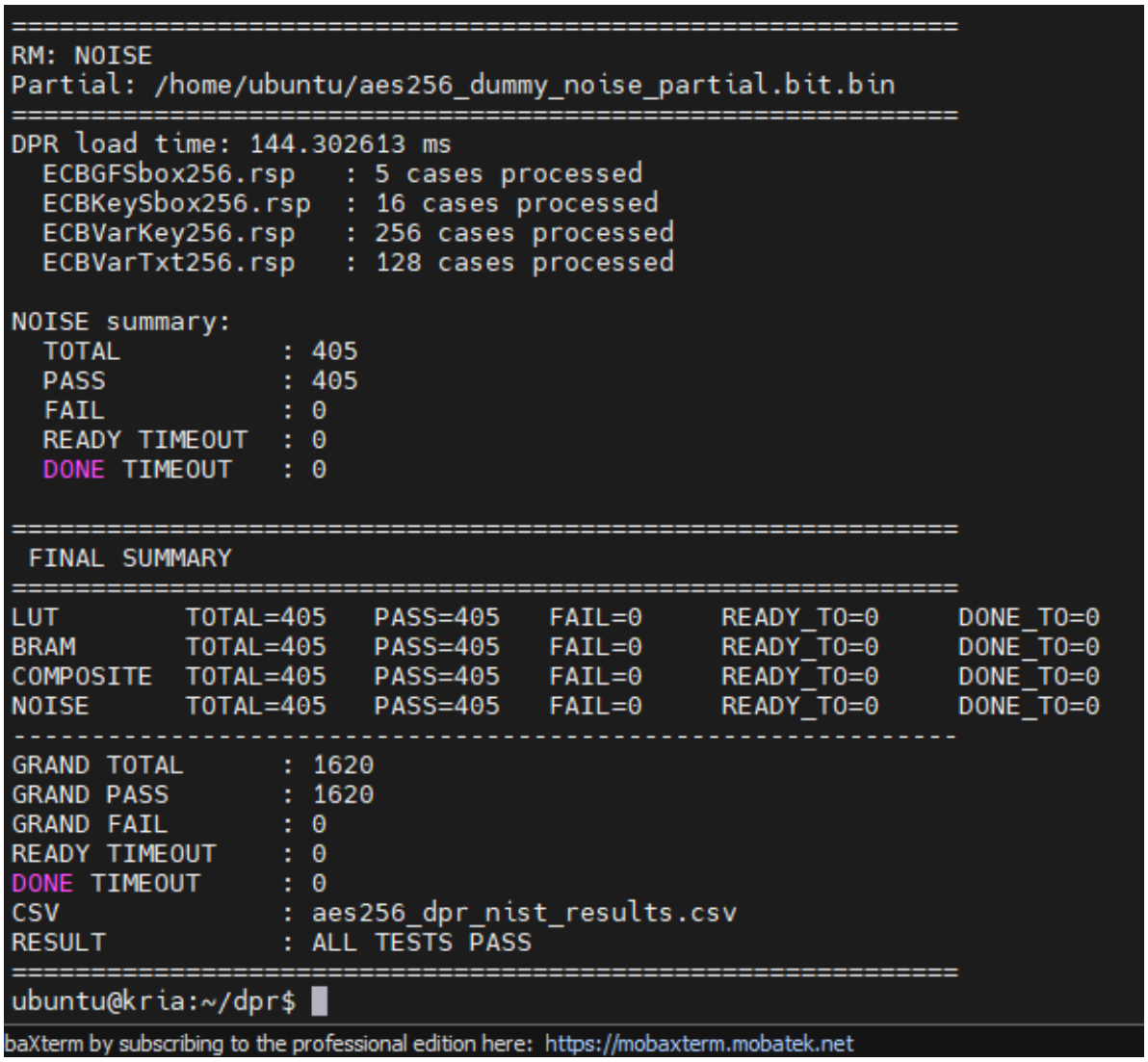
Mỗi testcase được ghi vào CSV cùng tên biến thể, Test Suite, COUNT, Key, Plaintext, Ciphertext thực tế, trạng thái phần cứng và kết quả PASS/FAIL hoặc Timeout. Ciphertext tham chiếu được đọc từ các file NIST khi thực thi. CSV và log tổng kết được sử dụng để đối chiếu giữa DPR và Replica.

5.3.2 Kết quả kiểm chứng kiến trúc DPR

Đối với DPR, quá trình kiểm chứng được tiến hành tuần tự trên bốn RM: LUT, BRAM, Composite và Noise. Trước khi chạy mỗi bộ dữ liệu, chương trình đưa giao diện của RP vào trạng thái HOLD, nạp Partial Bitstream tương ứng, kiểm tra kết quả nạp và giải phóng HOLD.

Sau khi RM mới được kích hoạt, chương trình thực hiện 405 phép mã hóa AES-256 từ bốn file NIST. Mỗi phép thử sử dụng dữ liệu đầu vào riêng và đối chiếu Ciphertext với giá trị tham chiếu tương ứng. Quy trình được lặp lại cho đến khi hoàn thành cả bốn RM.

Log thực nghiệm ghi nhận toàn bộ các Partial Bitstream được nạp thành công. Với mỗi RM, chương trình xử lý lần lượt 5 testcase từ `ECBGFSbox256.rsp`, 16 testcase từ `ECBKeySbox256.rsp`, 256 testcase từ `ECBVarKey256.rsp` và 128 testcase từ `ECBVarTxt256.rsp`.



Hình 41: Log kiểm chứng NIST AES-256 trên kiến trúc DPR, thể hiện kết quả của bốn RM và tổng kết 1 620 testcase.

Hình 41 thể hiện phần cuối quá trình kiểm chứng DPR trên KV260. Kết quả tổng hợp ghi nhận mỗi RM hoàn thành 405 testcase, trong đó toàn bộ đều PASS. Không có trường hợp Ciphertext sai, READY Timeout hoặc DONE Timeout.

Như vậy, kiến trúc DPR hoàn thành tổng cộng 1 620 phép thử Encryption, tương ứng với 405 Test Vector trên bốn cấu hình phần cứng khác nhau. Kết quả này cho thấy cả bốn RM đều thực hiện đúng chức năng AES-256 sau khi được nạp thông qua FPGA Manager.

Mỗi RM được nạp một lần trước khi chạy 405 Test Vector. Phép thử này xác nhận chức năng mã hóa sau tái cấu hình; độ ổn định của những lần chuyển đổi liên tiếp được đánh giá trong Mục 6.2.

5.3.3 Kết quả kiểm chứng kiến trúc Replica

Kiểm chứng Replica được thực hiện sau khi Full Bitstream của ứng dụng `aes256_replica` đã được nạp thành công trên KV260. Trạng thái `Active_slot = 0` của ứng dụng được xác nhận

thông qua `xmutil`, như đã trình bày trong Chương 4.

Khác với DPR, chương trình Replica không thực hiện thao tác nạp lại Bitstream giữa các biến thể. Thay vào đó, phần mềm lần lượt chọn LUT, BRAM, Composite và Noise bằng các cặp giá trị HOLD/RELEASE trên thanh ghi `RM_CONTROL`.

Các giá trị RELEASE tương ứng là `0x0`, `0x2`, `0x4` và `0x6`. Sau mỗi thao tác lựa chọn, chương trình đọc lại thanh ghi để xác nhận mã Core đã được cập nhật trước khi thực hiện giao dịch mã hóa.

```
./aes256_replica_nist /home/ubuntu/benchmarks \
2>&1 | tee aes256_replica_nist.log
============================================================
 AES-256 REPLICA NIST ECB KAT VERIFICATION
============================================================
NIST directory: /home/ubuntu/benchmarks
Replica must already be Active_slot = 0
============================================================

============================================================
CORE: LUT
HOLD/RELEASE: 0x1 -> 0x0
============================================================
Replica switch time: 2.680 us
RM_CONTROL readback: 0x00000000
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

LUT summary:
  TOTAL          : 405
  PASS           : 405
  FAIL           : 0
  READY TIMEOUT  : 0
  DONE TIMEOUT   : 0

============================================================
CORE: BRAM
HOLD/RELEASE: 0x3 -> 0x2
============================================================
Replica switch time: 0.770 us
RM_CONTROL readback: 0x00000002
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

BRAM summary:
  TOTAL          : 405
  PASS           : 405
  FAIL           : 0
  READY TIMEOUT  : 0
  DONE TIMEOUT   : 0
```

baXterm by subscribing to the professional edition here: https://mobaxterm.mobatek.net

Hình 42: Log kiểm chứng hai Core LUT và BRAM trong kiến trúc Replica, gồm thao tác lựa chọn Core, đọc lại RM_CONTROL và thực thi bộ Test Vector NIST.

Hình 42 thể hiện kết quả kiểm chứng LUT và BRAM. Với LUT, chương trình ghi cặp giá trị `0x1` và `0x0`, sau đó đọc lại `RM_CONTROL = 0x00000000`. Đối với BRAM, cặp giá trị tương ứng là `0x3` và `0x2`, với giá trị Readback bằng `0x00000002`.

Cả hai Core đều hoàn thành 405 testcase, không ghi nhận FAIL hoặc Timeout. Các giá trị Readback phù hợp với lựa chọn của chương trình, trong khi kết quả mã hóa được xác nhận bằng Ciphertext thực tế.

Quy trình tương tự được áp dụng cho Composite và Noise. Hai Core này sử dụng các cặp HOLD/RELEASE lần lượt là `0x5`/`0x4` và `0x7`/`0x6`. Giá trị Readback tương ứng trùng với mã lựa chọn đã gửi, đồng thời toàn bộ testcase đều đạt yêu cầu.

```
============================================================
CORE: COMPOSITE
HOLD/RELEASE: 0x5 -> 0x4
============================================================
Replica switch time: 0.790 us
RM_CONTROL readback: 0x00000004
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

COMPOSITE summary:
  TOTAL          : 405
  PASS           : 405
  FAIL           : 0
  READY TIMEOUT  : 0
  DONE TIMEOUT   : 0

============================================================
CORE: NOISE
HOLD/RELEASE: 0x7 -> 0x6
============================================================
Replica switch time: 0.780 us
RM_CONTROL readback: 0x00000006
  ECBGFSbox256.rsp    : 5 cases processed
  ECBKeySbox256.rsp   : 16 cases processed
    ECBVarKey256.rsp: 50 cases, PASS=71 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 100 cases, PASS=121 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 150 cases, PASS=171 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 200 cases, PASS=221 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarKey256.rsp: 250 cases, PASS=271 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarKey256.rsp    : 256 cases processed
    ECBVarTxt256.rsp: 50 cases, PASS=327 FAIL=0 READY_TO=0 DONE_TO=0
    ECBVarTxt256.rsp: 100 cases, PASS=377 FAIL=0 READY_TO=0 DONE_TO=0
  ECBVarTxt256.rsp    : 128 cases processed

NOISE summary:
  TOTAL          : 405
  PASS           : 405
  FAIL           : 0
  READY TIMEOUT  : 0
  DONE TIMEOUT   : 0

============================================================
 FINAL SUMMARY
============================================================
LUT        TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
BRAM       TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
COMPOSITE  TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
NOISE      TOTAL=405   PASS=405   FAIL=0     READY_TO=0     DONE_TO=0
------------------------------------------------------------
GRAND TOTAL      : 1620
GRAND PASS       : 1620
GRAND FAIL       : 0
READY TIMEOUT    : 0
```

Hình 43: Log kiểm chứng Composite và Noise trong Replica, kèm bảng tổng kết kết quả kiểm chứng của bốn Core.

Hình 43 trình bày kết quả của hai Core còn lại và phần tổng kết chương trình. Theo log, Replica hoàn thành 1 620 phép thử, trong đó 1 620 trường hợp PASS, không có trường hợp Ciphertext sai hoặc Timeout.

Kết quả cho thấy các Core trong Replica có thể được lựa chọn bằng giao diện AXI4-Lite và thực hiện đúng AES-256 Encryption với toàn bộ Test Vector đã sử dụng. Kết hợp với kết quả RTL Simulation ở Mục 5.2, phép kiểm chứng trên KV260 xác nhận cả chức năng mã hóa lẫn đường điều khiển lựa chọn Core trong các điều kiện đã thử nghiệm.

5.3.4 Tổng hợp và đối chiếu kết quả kiểm chứng

Kết quả của hai kiến trúc được tổng hợp từ các file log và CSV do chương trình kiểm chứng tạo ra. Đối với mỗi Core hoặc RM, số lượng testcase được thực hiện đúng bằng số lượng Test Vector Encryption trong bốn file NIST.

Bảng 22 trình bày kết quả theo từng biến thể và kiến trúc.

Bảng 22: Kết quả kiểm chứng NIST AES-256 trên Kria KV260.

Kiến trúc	LUT	BRAM	Composite	Noise	Tổng PASS
DPR	405/405	405/405	405/405	405/405	1 620
Replica	405/405	405/405	405/405	405/405	1 620

Các trường hợp kiểm chứng đều cho kết quả PASS. Trong cả hai kiến trúc, không ghi nhận FAIL, READY Timeout hoặc DONE Timeout. Thanh ghi STATUS tại thời điểm hoàn tất được ghi nhận với giá trị `0x00000003` trên toàn bộ 3 240 dòng dữ liệu, tương ứng với hai bit trạng thái READY và DONE được thiết lập.

Để xác định phạm vi kiểm chứng của từng Test Suite, Bảng 23 tổng hợp số testcase đã đạt khi cộng kết quả của bốn biến thể trong mỗi kiến trúc.

Bảng 23: Kết quả kiểm chứng theo từng bộ Test Vector NIST.

Test Suite	Vector/Core	DPR PASS	Replica PASS
`ECBGFSbox256`	5	20/20	20/20
`ECBKeySbox256`	16	64/64	64/64
`ECBVarKey256`	256	1 024/1 024	1 024/1 024
`ECBVarTxt256`	128	512/512	512/512
Tổng cộng	**405**	**1 620/1 620**	**1 620/1 620**

Ngoài việc đối chiếu các giá trị PASS/FAIL do chương trình tạo ra, nhóm tiến hành rà soát lại dữ liệu CSV. Mỗi kiến trúc có đúng 1 620 bản ghi kết quả, với 405 tổ hợp Test Suite–COUNT riêng biệt cho từng biến thể. Không phát hiện testcase bị thiếu hoặc lặp trong phạm vi mỗi Core.

Việc so sánh Key, Plaintext và Ciphertext giữa hai file CSV cũng cho thấy các biến thể tương ứng của DPR và Replica nhận cùng dữ liệu đầu vào và tạo cùng kết quả đầu ra. Đồng thời, khi kiểm tra lại các cặp Key–Plaintext bằng một thư viện AES độc lập trên phần mềm, toàn bộ 3 240 Ciphertext được ghi trong CSV đều trùng với kết quả tính toán.

Như vậy, tập dữ liệu thực nghiệm gồm 405 Test Vector khác nhau được chạy trên tám cấu hình kiểm chứng, tạo thành 3 240 phép mã hóa AES-256 đã hoàn thành. Kết quả xác nhận tính đúng đắn chức năng của cả hai kiến trúc trong phạm vi các bộ dữ liệu Encryption được sử dụng.

Tuy nhiên, kết quả PASS không phản ánh trực tiếp hiệu năng mã hóa hay chi phí chuyển đổi giữa các Core. Các chương trình NIST sử dụng một khoảng chờ ngắn trước khi kiểm tra DONE

và chỉ đo thời gian lựa chọn hoặc nạp cấu hình một lần cho mỗi biến thể. Những giá trị thời gian này được ghi lặp lại trên các dòng CSV của cùng biến thể, nên không thể xem là các mẫu độ trễ độc lập.

Vì vậy, dữ liệu NIST trong mục này được sử dụng để đánh giá tính đúng đắn chức năng. Các phép đo độ trễ xử lý AES-256 và chi phí chuyển đổi DPR–Replica được thực hiện bằng Benchmark riêng trong Chương 6.

5.4 Đánh giá tài nguyên và Timing sau Implementation

Các báo cáo Vivado sử dụng trong mục này được xuất từ năm thiết kế đã hoàn tất Routing, gồm bốn Configuration Runs của DPR và một Run của Replica. Cả năm cùng nhắm đến thiết bị `xck26-sfvc784-2LV-c`, được phân tích bằng Vivado 2025.2. Việc sử dụng dữ liệu ở cùng giai đoạn thực thi giúp tránh so sánh báo cáo Synthesis của một phương án với báo cáo Implementation của phương án còn lại. Đơn vị tài nguyên trong các bảng bên dưới là số lượng phần tử logic vật lý do công cụ báo cáo, không phải tỷ lệ phần trăm hoặc dung lượng bộ nhớ tính bằng byte.

5.4.1 Mức sử dụng tài nguyên của toàn thiết kế

Bảng 24 trình bày số LUT, Flip-Flop (FF), RAMB18 và DSP ở mức Top-level. Với DPR, mỗi dòng đại diện cho một Configuration hoàn chỉnh gồm vùng Static và một RM cụ thể, không phải bốn lõi được cộng tài nguyên với nhau. Với Replica, bốn lõi cùng tồn tại trong một bitstream và số liệu phản ánh toàn bộ logic đi kèm bộ chọn, điều khiển và khối PS–PL.

Bảng 24: Tài nguyên toàn thiết kế sau Routing trên thiết bị Kria K26.

Phương án / biến thể	LUT	FF	RAMB18	DSP
DPR – LUT	2 596	1 796	0	0
DPR – BRAM	1 890	1 796	10	0
DPR – Composite	2 820	1 788	0	0
DPR – Noise	2 979	2 044	8	0
Replica	6 715	3 346	18	0

Ở cùng cấu hình DPR, BRAM dùng ít LUT nhất (1 890 LUT), nhưng sử dụng 10 RAMB18. Điều này cho thấy thiết kế đã được ánh xạ thực tế sang Block RAM, không chỉ được đặt tên là "BRAM" trong RTL. Ngược lại, LUT và Composite không dùng RAMB18 trong các báo cáo này. Biến thể Noise dùng 8 RAMB18 và có số FF cao hơn ba phương án còn lại, phù hợp với việc chứa thêm phần logic tạo nhiễu công suất có tương quan (CPNG). Tuy nhiên, số tài nguyên không phải phép đo trực tiếp công suất tiêu thụ; để kết luận về công suất cần báo cáo Power hoặc đo trên board.

Replica có 6 715 LUT, 3 346 FF và 18 RAMB18 ở mức hệ thống. Các số liệu lớn hơn bất kỳ cấu hình DPR riêng lẻ nào là hệ quả phù hợp với việc giữ đồng thời bốn biến thể, song không được phép quy toàn bộ chênh lệch cho lõi AES vì phần kết nối và logic phụ trợ của hai Top-level khác nhau.

5.4.2 Phân tách tài nguyên lõi mã hóa và phần giao tiếp

Báo cáo `Utilization by Hierarchy` cho phép chọn đúng instance thuộc lõi. Trong DPR, đó là `u_aes_rm` mang thuộc tính `Reconfigurable`; trong Replica là `u_aes_replica` chứa bốn instance con. Khi chỉ xét riêng phần này, kết quả được tổng hợp ở Bảng 25.

Bảng 25: Tài nguyên theo hierarchy của phần AES, không bao gồm toàn bộ PS–PL.

Hierarchy	LUT	FF	RAMB18	Kiến trúc
`u_aes_rm` – LUT	1 340	397	0	DPR
`u_aes_rm` – BRAM	634	397	10	DPR
`u_aes_rm` – Composite	1 564	389	0	DPR
`u_aes_rm` – Noise	1 723	645	8	DPR
`u_aes_replica` (4 lõi)	3 892	1 827	18	Replica

Điểm đáng chú ý là ở cả bốn Configuration DPR, hierarchy `u_kv260_ps` giữ nguyên mức 1 256 LUT và 1 399 FF. Đây là bằng chứng về phần Static chung được duy trì trong các báo cáo DPR. Mỗi RM đóng góp phần tài nguyên riêng, chẳng hạn BRAM là 634 LUT và 10 RAMB18, còn Noise là 1 723 LUT, 645 FF và 8 RAMB18. Các số liệu này có ý nghĩa trực tiếp hơn khi thảo luận tác động của cách hiện thực AES so với số lượng toàn thiết kế.

Trong Replica, `u_aes_replica` sử dụng 3 892 LUT, 1 827 FF và 18 RAMB18. Bên trong khối này, báo cáo ghi 959 LUT cho `u_lut`, 598 LUT cho `u_bram`, 1 127 LUT cho `u_composite`, và 1 208 LUT cho `u_noise`. Độ lớn của từng lõi khi tích hợp vào Replica không hoàn toàn trùng số của RM tương ứng trong DPR. Điều này có thể chịu ảnh hưởng của ranh giới hierarchy, tối ưu hóa và ràng buộc triển khai khác nhau; không nên mặc nhiên xem hai cách ánh xạ là một phép so sánh IP độc lập đã được cố định điều kiện. Mặc dù tổng số LUT của bốn instance bằng tổng LUT tại `u_aes_replica` trong báo cáo này, điều đó không loại bỏ chi phí khác nằm bên ngoài hierarchy.

5.4.3 Kết quả Timing sau Routing

Các báo cáo `report_timing_summary` đều xác nhận hai Clock `clk_pl_0` và `clk_pl_1` có chu kỳ ràng buộc 10.000 ns, tương ứng 100 MHz. Tất cả năm Run ghi `All user specified timing constraints are met`; đồng thời WNS dương, TNS bằng không và không có Setup Failing Endpoints trong phần tổng hợp. Điều này cho thấy các cấu hình đã đáp ứng các ràng buộc Setup được khai báo tại tần số mục tiêu, không có nghĩa 100 MHz là Fmax được xác định bằng một phép quét tần số độc lập.

Bảng 26: Kết quả Timing Summary sau Routing với Clock mục tiêu 100 MHz.

Thiết kế	WNS (ns)	TNS (ns)	WHS (ns)	THS (ns)
DPR – LUT	+4.317	0.000	+0.027	0.000
DPR – BRAM	+3.469	0.000	+0.027	0.000
DPR – Composite	+3.167	0.000	+0.027	0.000
DPR – Noise	+2.981	0.000	+0.027	0.000
Replica	+3.766	0.000	+0.015	0.000

WNS nhỏ nhất trong bốn phương án DPR thuộc Noise (+2.981 ns), còn LUT đạt WNS lớn nhất (+4.317 ns). Replica có WNS +3.766 ns, nằm giữa các giá trị ghi nhận ở DPR. Các con số này mô tả biên độ Timing của *toàn thiết kế dưới các ràng buộc được khai báo*, do đó không đủ để kết luận một AES Core có độ trễ mã hóa theo chu kỳ thấp hơn hay có Throughput thực tế cao hơn. Kết quả WNS cũng chịu tác động của đường Timing tới hạn trong mạng logic và giao tiếp, không nhất thiết chỉ nằm trong Datapath AES.

Mặt khác, WHS dương và THS bằng không ở cả năm Run cho thấy không có vi phạm Hold được thống kê trong phần Timing Summary sau Routing. Đây là một điểm quan trọng của Replica: báo cáo Synthesis trước đó không phản ánh trạng thái cuối cùng của thiết kế; việc đánh giá phải dùng báo cáo đã được định tuyến, nơi WHS đạt +0.015 ns. Các cảnh báo phương pháp luận được Vivado liệt kê ở bước phân tích cần được giữ lại như giới hạn của quá trình đánh giá nếu chúng liên quan đến ràng buộc hoặc cách sử dụng phần tử, thay vì coi kết quả Timing dương là bảo đảm tuyệt đối cho mọi chế độ hoạt động.

Tổng hợp Mục 5.4 cho thấy hai kiến trúc đều đạt mục tiêu Timing 100 MHz trong các báo cáo đã cung cấp, nhưng khác nhau về mức sử dụng tài nguyên: DPR giữ một RM trên cùng vùng tái cấu hình, Replica dành tài nguyên cho cả bốn lõi. Phép đánh giá này chỉ phản ánh chi phí triển khai phần cứng và ràng buộc Timing; thời gian tái cấu hình DPR, thời gian điều khiển ARM/Linux và khả năng hoạt động ổn định qua nhiều lần chuyển đổi cần được kiểm tra bằng dữ liệu đo trên KV260 ở các mục thực nghiệm tiếp theo.

5.5 Phân tích và đánh giá tổng hợp

RTL Simulation, kiểm chứng NIST trên KV260 và báo cáo Implementation lần lượt cung cấp bằng chứng về chức năng, khả năng vận hành và kết quả hiện thực phần cứng. Phần dưới tổng hợp những kết quả chính, làm cơ sở cho các phép đo hiệu năng ở Chương 6.

5.5.1 Đánh giá tính đúng đắn chức năng

Mô phỏng RTL và kiểm chứng NIST trên KV260 xác nhận chức năng mã hóa của cả bốn biến thể AES-256. Tổng cộng 3 240 phép thử Encryption hoàn thành với Ciphertext trùng giá trị tham chiếu, không ghi nhận FAIL hoặc Timeout.

Trong DPR, các RM hoạt động đúng sau khi được nạp bằng Partial Bitstream. Với Replica, chương trình chọn từng Core qua AXI4-Lite và thu được kết quả mã hóa tương ứng. Các kết luận này giới hạn trong phạm vi Test Vector và điều kiện thực nghiệm đã sử dụng.

5.5.2 Đánh giá sự đánh đổi về tài nguyên và Timing

Về tài nguyên FPGA, Bảng 24 cho thấy Replica sử dụng 6 715 LUT và 3 346 FF ở mức Top-level. Trong khi đó, các cấu hình DPR sử dụng từ 1 890 đến 2 979 LUT và từ 1 788 đến 2 044 FF, tùy thuộc RM đang được triển khai.

Sự khác biệt này phù hợp với cách tổ chức hai kiến trúc. DPR duy trì một RM trong vùng tái cấu hình tại mỗi thời điểm, nhờ đó có thể sử dụng cùng một phần tài nguyên vật lý cho nhiều biến thể AES-256. Ngược lại, Replica giữ đồng thời cả bốn Core trong PL, khiến tổng tài nguyên phải đáp ứng nhu cầu của nhiều khối logic cùng lúc.

Đối với biến thể BRAM, số LUT được giảm xuống còn 1 890 trong cấu hình DPR, nhưng thiết kế sử dụng 10 RAMB18. Đây là sự đánh đổi giữa tài nguyên logic và tài nguyên bộ nhớ chuyên dụng. Composite và LUT không sử dụng RAMB18 trong các Configuration tương ứng, còn Noise sử dụng thêm Block RAM và Flip-Flop cho phần logic bổ sung.

Những khác biệt này cũng được thể hiện ở mức hierarchy của AES Core trong Bảng 25. Tuy nhiên, số LUT của một lõi khi tích hợp vào Replica không hoàn toàn trùng với RM tương ứng trong DPR. Vì vậy, khi so sánh hai phương án, cần phân biệt tài nguyên của riêng AES Core với tài nguyên của toàn bộ hệ thống, bao gồm giao diện AXI, điều khiển và các khối phụ trợ.

Về Timing, cả bốn Configuration DPR và thiết kế Replica đều đáp ứng các ràng buộc đã khai báo ở Clock mục tiêu 100 MHz. Các giá trị Worst Negative Slack (WNS) đều dương và Total Negative Slack (TNS) bằng không sau Routing.

Trong bốn cấu hình DPR, WNS dao động từ +2.981 ns đến +4.317 ns, trong khi Replica ghi nhận +3.766 ns. Điều này cho thấy các thiết kế đều đáp ứng yêu cầu Setup Timing trong điều kiện phân tích của Vivado, dù biên độ Timing khác nhau giữa các Configuration.

5.5.3 Giới hạn đánh giá và định hướng thực nghiệm tiếp theo

Chương 5 xác nhận tính đúng đắn chức năng, mức sử dụng tài nguyên và khả năng đáp ứng Timing của các thiết kế. Dữ liệu NIST không được sử dụng để xác định AES latency hoặc switching latency, vì chương trình kiểm chứng có thêm thời gian chờ và chi phí điều khiển từ Linux.

DPR tái sử dụng tài nguyên trong RP nhưng phải nạp Partial Bitstream khi thay đổi RM. Replica duy trì đồng thời bốn Core và lựa chọn qua AXI4-Lite. Chương 6 đánh giá sự đánh đổi này bằng các phép đo chuyển đổi, workload AES-256 và công suất ước lượng từ Vivado.

CHƯƠNG 6: THỰC NGHIỆM VÀ KIỂM TRA HỆ THỐNG

6.1 Thiết lập thực nghiệm

Các phép thực nghiệm trong chương này được thực hiện trên bo mạch AMD Kria KV260 sử dụng K26 Zynq UltraScale+ MPSoC. Thiết kế phần cứng được xây dựng bằng Vivado 2025.2, trong khi các chương trình điều khiển và thu thập dữ liệu được thực thi trên Ubuntu 22.04 LTS ở phía Processing System. Phần Programmable Logic hoạt động ở tần số xấp xỉ 100 MHz trong toàn bộ các phép đo runtime.

Kiến trúc DPR gồm bốn Reconfigurable Module tương ứng với các biến thể LUT, BRAM, Composite và Noise. Tại một thời điểm chỉ một RM được nạp vào Reconfigurable Partition, trong khi phần static và giao diện AXI4-Lite được giữ nguyên. Kiến trúc Replica được triển khai trên cùng nền tảng với bốn Core AES-256 tồn tại đồng thời trong PL và được sử dụng làm phương án đối chiếu trong quá trình đánh giá. Hai kiến trúc sử dụng cùng vùng thanh ghi AXI4-Lite tại địa chỉ cơ sở `0xA0000000`, cùng cách truyền Key, Plaintext và Ciphertext giữa phần mềm và phần cứng.

Thực nghiệm được chia thành hai nhóm chính. Nhóm thứ nhất thực hiện 10 000 lần chuyển đổi giữa các biến thể với seed `3152026`, đồng thời kiểm tra kết quả AES-256 sau mỗi lần chuyển đổi. Dữ liệu thu được được sử dụng để đánh giá độ ổn định của quá trình tái cấu hình, phân bố lựa chọn RM, các hướng chuyển đổi và thời gian nạp Partial Bitstream. Nhóm thứ hai sử dụng cùng một workload gồm 10 000 phép mã hóa AES-256 cho DPR và Replica. Mỗi biến thể xử lý 2 500 khối dữ liệu, toàn bộ quá trình gồm 40 batch và 39 lần chuyển đổi. Hai kiến trúc sử dụng cùng tập Plaintext, cùng Key và cùng thứ tự biến thể để tạo điều kiện đối chiếu về thời gian xử lý và chi phí chuyển đổi.

Bên cạnh các phép đo trên KV260, công suất của các cấu hình được lấy từ báo cáo sau implementation trong Vivado. Các báo cáo được tạo riêng cho bốn cấu hình DPR và cho kiến trúc Replica, với các thông số gồm Total On-Chip Power, Dynamic Power và Device Static Power. Các giá trị này được sử dụng để so sánh công suất ước lượng giữa các phương án hiện thực trong cùng điều kiện thiết kế.

Bảng 27 tóm tắt các điều kiện chính của quá trình thực nghiệm.

Bảng 27: Điều kiện thực nghiệm sử dụng trong Chương 6.

Thông số	Giá trị
Nền tảng	AMD Kria KV260, K26 Zynq UltraScale+ MPSoC
Công cụ thiết kế	Vivado 2025.2
Hệ điều hành	Ubuntu 22.04 LTS
Tần số PL	Xấp xỉ 100 MHz
Giao tiếp PS–PL	AXI4-Lite
Các biến thể AES-256	LUT, BRAM, Composite và Noise
Thực nghiệm chuyển đổi	10 000 lần chuyển đổi, seed `3152026`
Workload AES-256	10 000 phép mã hóa, 40 batch, 39 lần chuyển đổi
Đánh giá công suất	Vivado post-implementation power report

Các kết quả của hai nhóm thực nghiệm và số liệu công suất được trình bày trong các mục tiếp theo, trong đó DPR là kiến trúc được phân tích chính và Replica được sử dụng làm cơ sở đối chiếu.

6.2 Đánh giá quá trình chuyển đổi giữa các biến thể AES-256

Phép thử chuyển đổi ngẫu nhiên được thực hiện nhằm đánh giá khả năng hoạt động liên tục của cơ chế DPR khi thay đổi giữa bốn RM LUT, BRAM, Composite và Noise. Chương trình thực hiện 10 000 lần chuyển đổi với seed cố định `3152026`. Sau mỗi lần nạp RM mới, một phép mã hóa AES-256 được thực hiện và Ciphertext được đối chiếu với giá trị tham chiếu.

Biến thể tiếp theo được lựa chọn ngẫu nhiên trong ba RM còn lại, không chọn lại RM đang hoạt động. Cùng chuỗi lựa chọn được sử dụng cho kiến trúc Replica để tạo dữ liệu đối chiếu. Không có khoảng chờ nhân tạo được chèn giữa các lần chuyển đổi.

6.2.1 Độ ổn định của quá trình tái cấu hình

Trong 10 000 lần chuyển đổi DPR, toàn bộ Partial Bitstream được nạp thành công và phép mã hóa thực hiện sau tái cấu hình đều cho Ciphertext đúng. Không ghi nhận trường hợp lỗi tái cấu hình, sai Ciphertext, READY Timeout hoặc DONE Timeout.

Kết quả tổng hợp của phép thử được trình bày trong Bảng 28.

Bảng 28: Kết quả phép thử chuyển đổi ngẫu nhiên của kiến trúc DPR.

Thông số	Kết quả
Số lần chuyển đổi	10 000
Chuyển đổi thành công	10 000
AES-256 PASS	10 000
Lỗi DPR	0
Ciphertext sai	0
READY/DONE Timeout	0
Tỷ lệ hoàn thành	100%

Kết quả cho thấy Reconfigurable Partition có thể được thay đổi liên tục giữa bốn RM mà không làm mất chức năng mã hóa trong phạm vi số lần thử đã thực hiện. Khác với kiểm chứng NIST ở Chương 5, trong đó mỗi RM được nạp một lần trước khi chạy toàn bộ Test Vector, phép thử này thực hiện tái cấu hình ở từng iteration và kiểm tra lại AES-256 ngay sau mỗi lần chuyển đổi. Vì vậy, dữ liệu thu được phản ánh trực tiếp hơn độ ổn định của quá trình thay đổi RM khi hệ thống hoạt động liên tục.

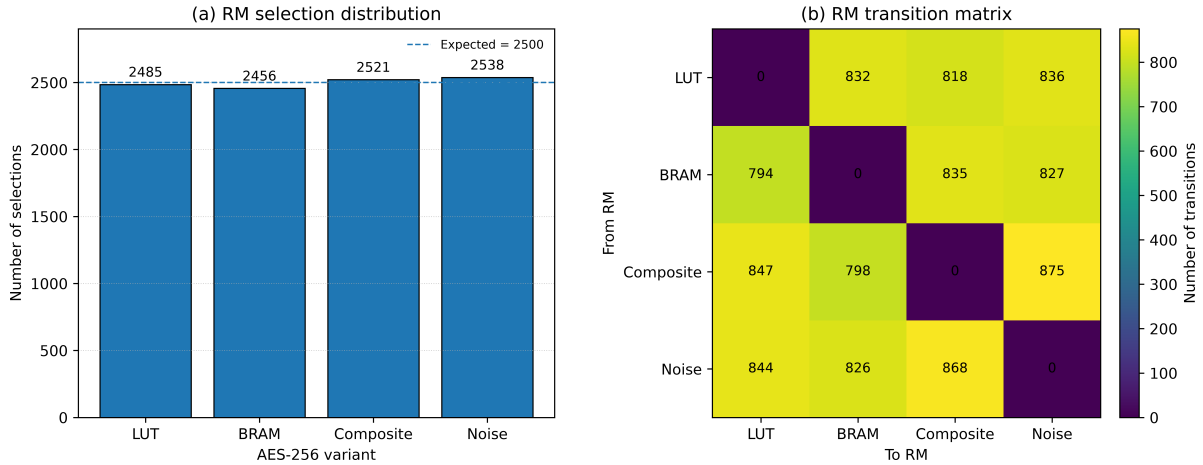
6.2.2 Phân bố lựa chọn RM và các hướng chuyển đổi

Sau 10 000 lần chuyển đổi, số lần bốn RM được lựa chọn lần lượt là 2 485 đối với LUT, 2 456 đối với BRAM, 2 521 đối với Composite và 2 538 đối với Noise. Mỗi biến thể vì vậy chiếm xấp xỉ một phần tư tổng số lần lựa chọn.

Phép kiểm định Chi-square trên bốn nhóm cho giá trị $p \approx 0.655$. Với tập dữ liệu đã thu thập, không quan sát thấy sự lệch đáng kể về tần suất lựa chọn giữa bốn RM.

Bên cạnh số lần lựa chọn từng RM, chương trình ghi nhận cả biến thể trước và sau mỗi lần chuyển đổi. Do thuật toán không cho phép chọn lại RM hiện tại, ma trận chuyển đổi gồm 12 hướng khác nhau và không có phần tử trên đường chéo chính.

Các số liệu được trình bày trong Hình 44.



Hình 44: Phân bố lựa chọn bốn RM và ma trận chuyển đổi trong 10 000 lần thử nghiệm DPR.

Ba hướng xuất phát từ LUT có số lần tương ứng là 832, 818 và 836; từ BRAM là 794, 835 và 827; từ Composite là 847, 798 và 875; từ Noise là 844, 826 và 868. Các giá trị tập trung quanh mức kỳ vọng khoảng 833 lần cho mỗi hướng. Kiểm định Chi-square trên 12 hướng chuyển đổi cho kết quả $p \approx 0.744$.

Kết quả trên cho thấy chuỗi thử nghiệm không bị tập trung rõ rệt vào một RM hoặc một cặp chuyển đổi cụ thể. Trong phạm vi chương này, phép phân tích được sử dụng để kiểm tra sự cân bằng của workload chuyển đổi giữa các RM. Giá trị này không được sử dụng như một đánh giá về độ an toàn mật mã của bộ sinh số giả ngẫu nhiên.

6.2.3 Thời gian tái cấu hình DPR

Thời gian tái cấu hình được ghi nhận cho từng lần nạp Partial Bitstream thông qua FPGA Manager. Thống kê của 10 000 mẫu được trình bày trong Bảng 29.

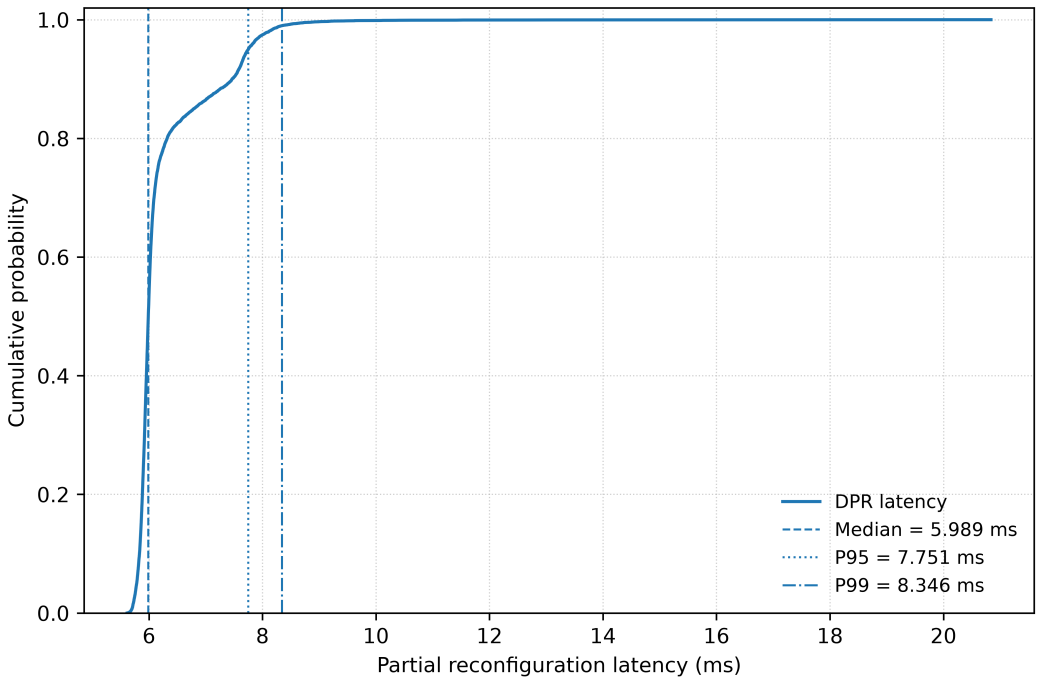
Bảng 29: Thống kê thời gian nạp Partial Bitstream trong phép thử DPR.

Thông số	Thời gian (ms)
Minimum	5.617
Median	5.989
Mean	6.245
Standard deviation	0.688
P95	7.751
P99	8.346
Maximum	20.838

Median đạt 5.989 ms, trong khi Mean đạt 6.245 ms. Phần lớn các mẫu vì vậy tập trung quanh khoảng 6 ms. Mean lớn hơn Median cùng với giá trị Maximum 20.838 ms cho thấy phân bố có một số mẫu kéo dài về phía thời gian lớn.

Trong toàn bộ 10 000 mẫu chỉ có 15 lần có thời gian vượt 10 ms và tất cả các lần này vẫn hoàn thành tái cấu hình và kiểm tra AES-256 thành công. Các giá trị lớn xuất hiện không làm thay đổi tính đúng đắn của hệ thống, nhưng làm tăng độ phân tán của thời gian tái cấu hình quan sát từ phía Linux.

Hình 45 thể hiện phân bố tích lũy của các mẫu thời gian thu được.



Hình 45: Phân bố tích lũy của thời gian tái cấu hình DPR trong 10 000 lần chuyển đổi.

Khoảng cách giữa Median và P95 cho thấy phần lớn quá trình nạp Partial Bitstream có thời gian khá ổn định, trong khi một số ít mẫu có độ trễ cao hơn. Các phép đo được thực hiện thông qua Linux và FPGA Manager nên thời gian ghi nhận bao gồm cả hoạt động của phần mềm quản lý cấu hình, không chỉ riêng quá trình truyền dữ liệu cấu hình bên trong FPGA.

Trong quá trình phát triển, nhóm cũng đo thời gian chuyển đổi khi gọi `fpgautil`. Giá trị trung bình của đường này nằm khoảng 45.5 ms, cao hơn đáng kể so với phép nạp trực tiếp thông qua FPGA Manager. Phân tích thời gian thực thi cho thấy phần lớn chênh lệch xuất hiện trong quá trình gọi và thực thi `fpgautil`. Vì vậy, giá trị khoảng 6 ms từ FPGA Manager được sử dụng làm số liệu chính khi phân tích quá trình nạp Partial Bitstream, còn kết quả từ `fpgautil` thể hiện chi phí của một đường điều khiển ở mức phần mềm cao hơn.

6.2.4 Đối chiếu với cơ chế lựa chọn trong Replica

Cùng phép thử 10 000 lần được thực hiện trên Replica với seed và chuỗi lựa chọn giống DPR. Toàn bộ 10 000 lần chuyển đổi và phép mã hóa đi kèm đều hoàn thành thành công. Thời gian lựa chọn Core trung bình được ghi nhận ở mức khoảng 0.725 μs, trong khi DPR cần khoảng 6.245 ms để nạp Partial Bitstream.

Sự chênh lệch này xuất phát từ cách tổ chức hai kiến trúc. Replica đã chứa đồng thời bốn Core AES-256 trong PL nên quá trình chuyển đổi chỉ thay đổi trạng thái lựa chọn Core. Với DPR, RM mới phải được truyền và cấu hình lại vào RP trước khi tiếp tục xử lý.

Kết quả này thể hiện rõ phần chi phí thời gian của cơ chế tái cấu hình. Đổi lại, DPR chỉ duy trì một RM trong RP tại mỗi thời điểm và không cần đặt đồng thời toàn bộ các biến thể trong vùng tái cấu hình. Các đánh đổi về tài nguyên và công suất của hai cách hiện thực được phân tích cùng với kết quả implementation ở các mục sau.

Phép thử chuyển đổi ngẫu nhiên cho thấy DPR duy trì đúng chức năng AES-256 qua 10 000 lần thay đổi RM, đồng thời thời gian nạp Partial Bitstream chủ yếu tập trung quanh mức 6 ms. Kết quả này xác nhận khả năng sử dụng DPR cho việc thay đổi biến thể AES-256 trong quá trình hoạt động, nhưng cũng cho thấy chi phí tái cấu hình có thể trở thành thành phần đáng kể nếu tần suất chuyển đổi cao. Ảnh hưởng của chi phí này đến một workload AES-256 hoàn chỉnh được đánh giá trong Mục 6.3. “‘latex

6.3 Đánh giá trên khối lượng công việc AES-256

Sau khi đánh giá riêng quá trình chuyển đổi RM, phép thử tiếp theo được thực hiện để xác định mức ảnh hưởng của DPR đến thời gian xử lý khi hệ thống thực hiện một workload AES-256 hoàn chỉnh. Cả DPR và Replica sử dụng cùng 10 000 Plaintext, cùng Key, cùng thứ tự biến thể và cùng seed `3152026`. Giá trị hash của dataset ở hai phía đều bằng `0x561249f9e59fa66f`.

Mỗi biến thể xử lý liên tiếp 250 khối dữ liệu trước khi chuyển sang biến thể tiếp theo theo thứ tự LUT, BRAM, Composite và Noise. Một chu kỳ gồm 1 000 phép mã hóa và được lặp lại 10 lần. Toàn bộ workload gồm 40 batch, 10 000 phép mã hóa và 39 lần chuyển đổi giữa các biến thể. Không có khoảng chờ nhân tạo được chèn giữa các phép mã hóa hoặc giữa các batch.

6.3.1 Kết quả thực nghiệm trên kiến trúc DPR

Kiến trúc DPR hoàn thành toàn bộ 10 000 phép mã hóa và 39 lần chuyển đổi RM. Ciphertext của tất cả các phép thử đều trùng với kết quả tham chiếu, đồng thời không ghi nhận lỗi trong quá trình nạp Partial Bitstream.

Tổng thời gian xử lý AES-256 được ghi nhận là 23.443279 ms, trong khi tổng thời gian dành cho 39 lần chuyển đổi RM đạt 240.096670 ms. Thời gian thực thi toàn workload là 267.601277 ms, tương ứng throughput 37 369.029 phép mã hóa mỗi giây.

Thời gian trung bình của một giao dịch AES-256 đạt 2.344 μs. Kết quả theo từng biến thể được trình bày trong Bảng 30.

Bảng 30: Thời gian xử lý AES-256 theo từng RM trong workload DPR.

RM	**Số phép mã hóa**	**Thời gian trung bình (μs)**
LUT	2 500	2.367
BRAM	2 500	2.324
Composite	2 500	2.330
Noise	2 500	2.357
Toàn bộ	**10 000**	**2.344**

Các giá trị giữa bốn RM nằm trong khoảng từ 2.324 μs đến 2.367 μs. Mức chênh lệch nhỏ cho thấy thời gian của một giao dịch AES-256 nhìn từ phía phần mềm tương đối gần nhau giữa các biến thể trong điều kiện thử nghiệm hiện tại.

Ngược lại, thời gian chuyển đổi RM lớn hơn đáng kể. Giá trị trung bình của một lần chuyển đổi DPR là 6 156.325 μs, trong đó thời gian lập trình Partial Bitstream trung bình chiếm 6 033.713 μs. Phần còn lại gồm thao tác HOLD, kiểm tra trạng thái FPGA Manager, RELEASE và đọc lại thanh ghi điều khiển.

Tổng thời gian chuyển đổi chiếm 89.7218% thời gian thực thi workload. Kết quả này cho thấy phần lớn thời gian của hệ thống không nằm ở phép mã hóa AES-256 mà nằm ở quá trình thay đổi RM.

6.3.2 Đối chiếu với kiến trúc Replica

Cùng workload được thực hiện trên Replica với đúng dataset và thứ tự biến thể đã sử dụng cho DPR. Replica hoàn thành 10 000 phép mã hóa và 39 lần chuyển đổi Core, không ghi nhận lỗi Ciphertext hoặc lỗi lựa chọn biến thể.

Tổng thời gian xử lý AES-256 của Replica đạt 23.019693 ms, gần với 23.443279 ms của DPR. Thời gian trung bình của một giao dịch AES-256 là 2.302 μs, trong khi các Core LUT, BRAM, Composite và Noise lần lượt đạt 2.300 μs, 2.301 μs, 2.308 μs và 2.299 μs.

Bảng 31 tổng hợp các thông số chính của hai kiến trúc.

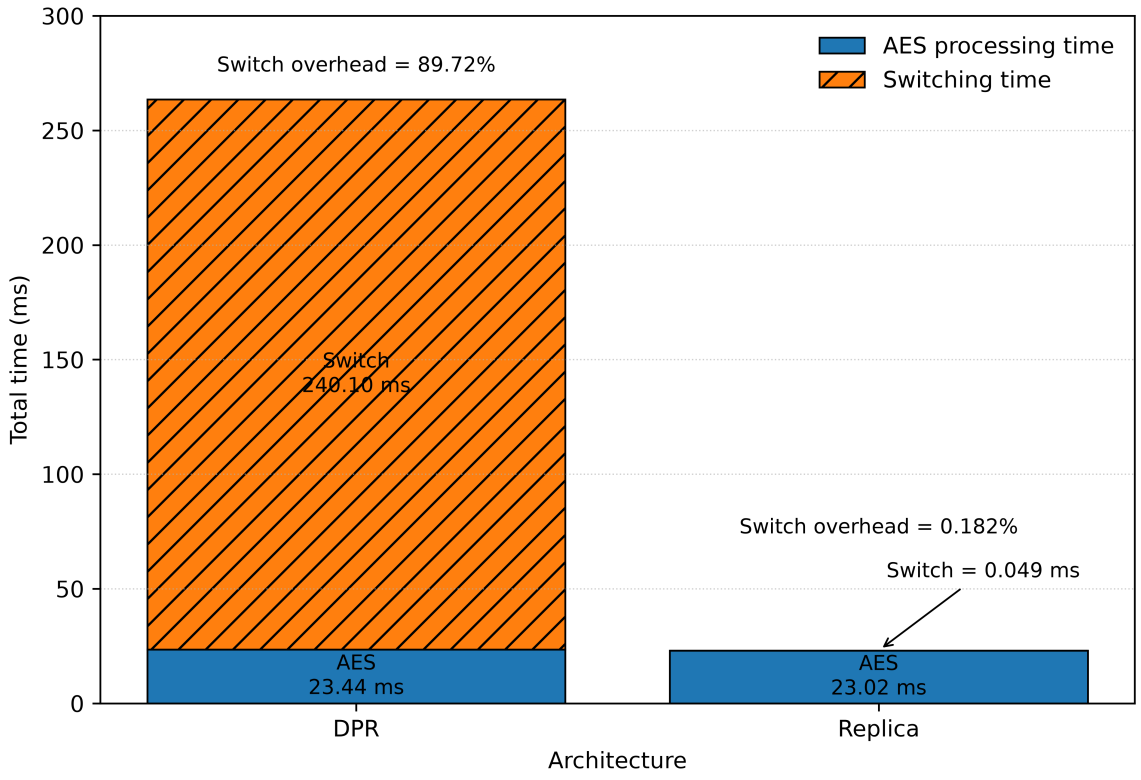
Bảng 31: Kết quả thực nghiệm trên cùng workload AES-256.

Thông số	DPR	Replica
Số phép mã hóa	10 000	10 000
AES-256 PASS	10 000	10 000
Số lần chuyển đổi	39	39
Thời gian AES trung bình (μs)	2.344	2.302
Thời gian chuyển đổi trung bình (μs)	6 156.325	1.269
Tổng thời gian AES (ms)	23.443279	23.019693
Tổng thời gian chuyển đổi (ms)	240.096670	0.049480
Thời gian toàn workload (ms)	267.601277	27.122111
Throughput (phép mã hóa/s)	37 369.029	368 702.864
Tỷ lệ thời gian chuyển đổi	89.7218%	0.1824%

Thời gian xử lý AES-256 trung bình của hai kiến trúc chỉ chênh lệch nhỏ, trong khi thời gian chuyển đổi khác nhau rõ rệt. Replica chỉ thay đổi giá trị lựa chọn Core nên thời gian chuyển đổi trung bình ở mức 1.269 μs. DPR phải thực hiện quá trình nạp Partial Bitstream trước khi RM mới có thể hoạt động, dẫn đến thời gian chuyển đổi trung bình khoảng 6.156 ms.

Sự khác biệt này làm throughput của Replica đạt 368 702.864 phép mã hóa mỗi giây, trong khi DPR đạt 37 369.029 phép mã hóa mỗi giây trong workload đã sử dụng. Replica vì vậy đạt throughput cao hơn khoảng 9.87 lần trong điều kiện 250 phép mã hóa cho mỗi batch.

Hình 46 trình bày sự khác biệt về thời gian xử lý và chi phí chuyển đổi giữa hai kiến trúc.



Hình 46: So sánh thời gian xử lý AES-256 và thời gian chuyển đổi của DPR và Replica trên cùng workload.

Kết quả cho thấy chênh lệch hiệu năng của hai kiến trúc trong phép thử này chủ yếu xuất phát từ cơ chế chuyển đổi biến thể. Tổng thời gian thực hiện AES-256 của DPR và Replica gần tương đương, trong khi tổng thời gian chuyển đổi của DPR cao hơn nhiều lần so với thời gian mã hóa.

6.3.3 Ảnh hưởng của tần suất tái cấu hình đến hiệu năng

Trong workload đã xây dựng, mỗi lần tái cấu hình được thực hiện sau 250 phép mã hóa. Với thời gian trung bình khoảng 6.156 ms cho một lần chuyển đổi và khoảng 2.344 μs cho một giao dịch AES-256, chi phí tái cấu hình vẫn lớn hơn đáng kể so với thời gian xử lý của một batch.

Kết quả này cho thấy hiệu năng của DPR phụ thuộc trực tiếp vào số lượng phép mã hóa được thực hiện giữa hai lần tái cấu hình. Khi số khối dữ liệu xử lý trên mỗi RM tăng, chi phí của một lần nạp Partial Bitstream được phân bố trên nhiều phép mã hóa hơn và tỷ lệ thời gian chuyển đổi trong toàn workload giảm. Ngược lại, việc thay đổi RM với tần suất cao làm thời gian tái cấu hình chiếm phần lớn thời gian xử lý.

Đặc điểm này tạo ra sự đánh đổi chính của thiết kế DPR. Cơ chế tái cấu hình cho phép nhiều biến thể AES-256 sử dụng chung một Reconfigurable Partition thay vì tồn tại đồng thời trong PL, nhưng việc thay đổi RM không diễn ra tức thời. Trong các ứng dụng có tần suất chuyển đổi thấp hoặc mỗi RM xử lý một lượng dữ liệu đủ lớn trước khi được thay thế, chi phí tái cấu hình có thể được phân bố trên toàn bộ workload. Với các trường hợp yêu cầu thay đổi biến thể liên tục, thời gian nạp Partial Bitstream trở thành thành phần chính làm giảm throughput.

Replica thể hiện trường hợp đối chiếu ngược lại. Việc giữ đồng thời các Core giúp quá trình lựa chọn diễn ra rất nhanh, nhưng yêu cầu nhiều tài nguyên phần cứng hơn. Do đó, kết quả của phép thử không chỉ phản ánh sự chênh lệch về tốc độ mà còn cho thấy hai hướng hiện thực sử dụng hai cách đánh đổi khác nhau giữa thời gian chuyển đổi và chi phí phần cứng.

Các số liệu về công suất và thông số implementation của hai kiến trúc được trình bày trong Mục 6.4 nhằm bổ sung góc nhìn về phần chi phí phần cứng tương ứng với sự khác biệt hiệu năng đã quan sát trong phép thử này. “‘ “‘latex

6.4 Đánh giá công suất của các cấu hình

Bên cạnh thời gian tái cấu hình và hiệu năng xử lý, công suất của các phương án hiện thực được ước lượng từ báo cáo sau implementation trong Vivado. Các báo cáo được tạo riêng cho bốn cấu hình DPR gồm LUT, BRAM, Composite và Noise, đồng thời thực hiện tương tự cho kiến trúc Replica.

Các báo cáo đều sử dụng cùng phương pháp ước lượng của Vivado, không sử dụng file switching activity từ mô phỏng và có Confidence Level ở mức Medium. Vì vậy, các giá trị trong mục này được sử dụng để so sánh xu hướng công suất giữa các thiết kế trong cùng điều kiện, thay vì xem như công suất đo trực tiếp trên bo mạch KV260.

6.4.1 Kết quả ước lượng công suất

Bảng 32 tổng hợp Total On-Chip Power, Dynamic Power và Device Static Power của năm cấu hình được đánh giá.

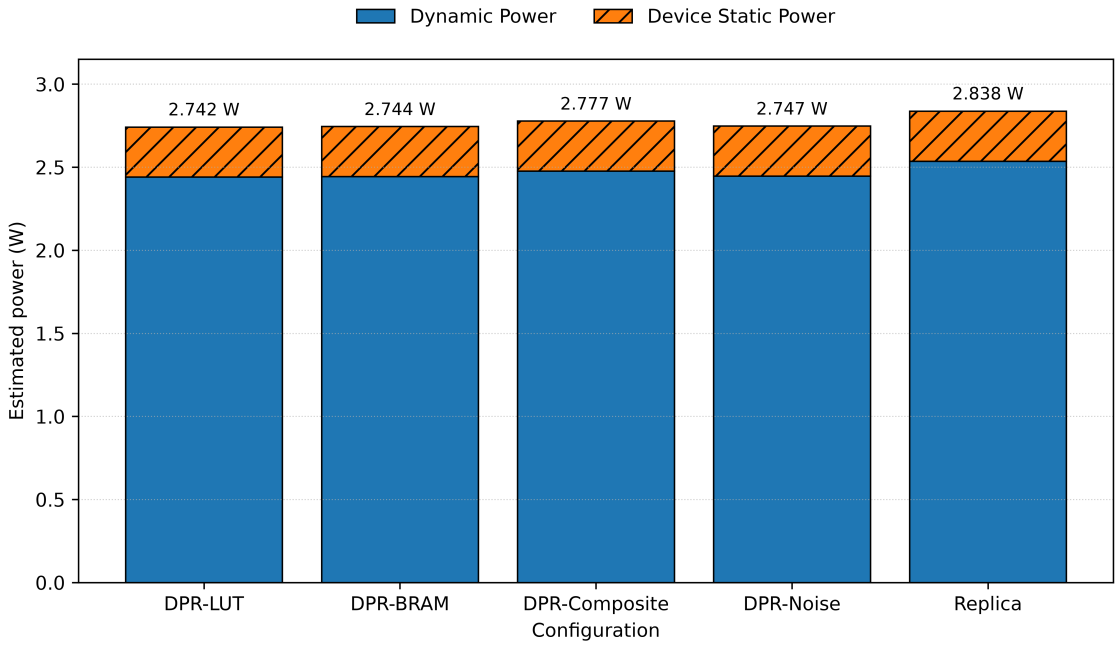
Bảng 32: Công suất ước lượng sau implementation của các cấu hình AES-256.

Cấu hình	Total (W)	Dynamic (W)	Static (W)
DPR–LUT	2.742	2.441	0.301
DPR–BRAM	2.744	2.444	0.301
DPR–Composite	2.777	2.477	0.301
DPR–Noise	2.747	2.447	0.301
Replica	2.838	2.536	0.301

Trong bốn cấu hình DPR, Total On-Chip Power nằm trong khoảng từ 2.742 W đến 2.777 W. LUT có giá trị thấp nhất, trong khi Composite có giá trị cao nhất. Chênh lệch giữa bốn cấu hình tương đối nhỏ so với tổng công suất của toàn hệ thống.

Device Static Power của cả năm thiết kế đều bằng 0.301 W. Phần khác biệt chủ yếu xuất hiện ở Dynamic Power, với các cấu hình DPR nằm trong khoảng từ 2.441 W đến 2.477 W, còn Replica đạt 2.536 W.

Hình 47 thể hiện thành phần Dynamic Power và Device Static Power của từng cấu hình.



Hình 47: So sánh công suất ước lượng sau implementation của bốn cấu hình DPR và kiến trúc Replica.

6.4.2 Phân tích sự khác biệt giữa DPR và Replica

Trung bình số học của Total On-Chip Power trên bốn cấu hình DPR là 2.7525 W. Replica đạt 2.838 W, cao hơn khoảng 3.1% so với giá trị trung bình này. Nếu chỉ xét Dynamic Power, giá trị trung bình của bốn cấu hình DPR là khoảng 2.452 W, trong khi Replica đạt 2.536 W, tương ứng mức chênh lệch khoảng 3.4%.

Kết quả này phù hợp với cách tổ chức của hai kiến trúc. Trong DPR, tại một thời điểm Reconfigurable Partition chỉ chứa một RM, trong khi Replica tích hợp đồng thời cả bốn Core

AES-256 trong Programmable Logic. Việc duy trì đồng thời nhiều khối logic làm Dynamic Power của Replica cao hơn so với từng cấu hình DPR.

Tuy nhiên, mức chênh lệch Total On-Chip Power chỉ nằm ở vài phần trăm, nhỏ hơn nhiều so với mức chênh lệch về thời gian chuyển đổi đã quan sát trong Mục 6.3. Điều này cho thấy trong thiết kế hiện tại, lợi thế của DPR thể hiện rõ hơn ở khả năng không phải duy trì đồng thời toàn bộ các biến thể phần cứng, trong khi chi phí lớn nhất của DPR lại nằm ở thời gian nạp Partial Bitstream.

Kết quả công suất cũng bổ sung cho phép so sánh hiệu năng ở Mục 6.3. Replica giảm gần như toàn bộ chi phí chuyển đổi bằng cách giữ sẵn các Core trong PL, đổi lại mức Dynamic Power cao hơn. DPR sử dụng một RM tại mỗi thời điểm nhưng phải chấp nhận độ trễ khi thay đổi cấu hình. Hai kết quả này thể hiện sự đánh đổi giữa chi phí phần cứng và chi phí tái cấu hình của hai cách hiện thực.

Các giá trị công suất trong mục này được lấy từ Vivado post-implementation power report và không sử dụng dữ liệu hoạt động SAIF hoặc VCD. Vì vậy, kết quả phản ánh mức ước lượng trong cùng điều kiện thiết kế và được sử dụng chủ yếu cho mục đích đối chiếu giữa các cấu hình. “‘ “‘latex

6.5 Thảo luận kết quả và giới hạn của thiết kế

Các kết quả thực nghiệm cho thấy cơ chế DPR đã thực hiện ổn định việc thay đổi giữa bốn biến thể AES-256 trên Kria KV260. Trong 10 000 lần chuyển đổi ngẫu nhiên, toàn bộ quá trình nạp Partial Bitstream và phép mã hóa thực hiện sau đó đều hoàn thành thành công. Kết quả này bổ sung cho phần kiểm chứng chức năng ở Chương 5, đồng thời cho thấy thiết kế có thể duy trì hoạt động AES-256 qua nhiều lần thay đổi RM liên tiếp.

Bên cạnh tính ổn định, các phép đo cũng cho thấy thời gian tái cấu hình là yếu tố ảnh hưởng lớn nhất đến hiệu năng của kiến trúc DPR. Thời gian nạp Partial Bitstream trong phép thử chuyển đổi ngẫu nhiên có giá trị trung bình 6.245 ms và Median 5.989 ms. Trong workload ở Mục 6.3, thời gian chuyển đổi hoàn chỉnh trung bình đạt 6.156 ms, trong khi một giao dịch AES-256 chỉ mất trung bình 2.344 μs. Sự khác biệt về độ lớn giữa hai thành phần này làm cho chi phí tái cấu hình chiếm phần lớn thời gian xử lý khi RM được thay đổi thường xuyên.

6.5.1 Ưu điểm của kiến trúc DPR

Ưu điểm chính của DPR nằm ở khả năng thay đổi phần cứng trong thời gian hoạt động mà không phải duy trì đồng thời toàn bộ các biến thể AES-256 trong Programmable Logic. Bốn RM LUT, BRAM, Composite và Noise sử dụng chung một Reconfigurable Partition và duy trì cùng giao diện với phần static của hệ thống. Việc thay đổi biến thể vì vậy không làm thay đổi cấu trúc giao tiếp AXI4-Lite giữa Processing System và phần cứng AES-256.

Cách tổ chức này cho phép bổ sung các RM khác trong tương lai mà không yêu cầu tất cả các Core phải cùng tồn tại trong một thiết kế. Đặc điểm này có ý nghĩa khi số lượng biến thể tăng hoặc khi từng biến thể có kích thước lớn. Trong khi Replica phải dành tài nguyên cho tất cả các Core được tích hợp, DPR chỉ sử dụng tài nguyên của RM đang được nạp tại một thời điểm.

Kết quả về công suất cũng thể hiện xu hướng tương tự. Total On-Chip Power trung bình của bốn cấu hình DPR đạt 2.7525 W, thấp hơn mức 2.838 W của Replica. Mức chênh lệch không lớn, tuy nhiên Dynamic Power của Replica vẫn cao hơn các cấu hình DPR do nhiều Core cùng tồn tại trong PL.

Một điểm khác của thiết kế là quá trình chuyển đổi RM được thực hiện trong khi phần static của hệ thống được giữ nguyên. Phần điều khiển AXI4-Lite, vùng thanh ghi Key, Plaintext, Ciphertext và các tín hiệu điều khiển không cần được xây dựng lại sau mỗi lần thay đổi biến thể. Đây là đặc trưng phù hợp với mục tiêu của Dynamic Partial Reconfiguration, trong đó chỉ một vùng xác định của FPGA được thay đổi trong thời gian hoạt động [10].

6.5.2 Hạn chế của cơ chế tái cấu hình

Chi phí chính của DPR trong thiết kế hiện tại là thời gian nạp Partial Bitstream. Với workload gồm 250 phép mã hóa trên mỗi biến thể trước khi chuyển RM, tổng thời gian chuyển đổi chiếm 89.7218% thời gian thực thi. Trong cùng điều kiện thử nghiệm, Replica chỉ dành 0.1824% thời gian cho quá trình lựa chọn Core. Throughput của DPR đạt 37 369.029 phép mã hóa mỗi giây, trong khi Replica đạt 368 702.864 phép mã hóa mỗi giây. Mức chênh lệch khoảng 9.87 lần trong phép thử này chủ yếu xuất phát từ quá trình tái cấu hình, vì thời gian trung bình của một giao dịch AES-256 giữa hai kiến trúc chỉ chênh lệch nhỏ.

Kết quả trên phụ thuộc vào số lượng phép mã hóa được thực hiện giữa hai lần thay đổi RM. Khi một RM xử lý nhiều dữ liệu hơn trước khi được thay thế, chi phí của một lần tái cấu hình được phân bố trên số lượng phép mã hóa lớn hơn. Ngược lại, với các workload thay đổi RM thường xuyên, thời gian nạp Partial Bitstream trở thành thành phần chi phối tổng thời gian xử lý. Vì vậy, tỷ lệ throughput giữa DPR và Replica trong phép thử hiện tại phản ánh cấu hình workload đã lựa chọn và không phải một hệ số cố định cho mọi trường hợp sử dụng.

Các phép đo thời gian DPR được thực hiện từ chương trình chạy trên Linux thông qua FPGA Manager. Do đó, số liệu bao gồm cả hoạt động của hệ điều hành và phần mềm quản lý cấu hình. Một số mẫu có độ trễ lớn hơn phần còn lại đã xuất hiện trong 10 000 lần thử, với giá trị lớn nhất 20.838 ms, trong khi 99% số mẫu không vượt quá 8.346 ms. Đặc điểm này cho thấy thời gian quan sát được không chỉ phụ thuộc vào quá trình cấu hình bên trong FPGA mà còn chịu ảnh hưởng từ môi trường thực thi phía Processing System.

6.5.3 Đối chiếu với các nghiên cứu trước

Việc sử dụng Partial Reconfiguration để thay đổi các hiện thực phần cứng khác nhau của thuật toán mật mã đã được nghiên cứu trong một số công trình trước đây. Hettwer và cộng sự sử dụng nhiều hiện thực AES có bố trí phần cứng khác nhau và thay đổi chúng trong quá trình hoạt động trên nền tảng Zynq UltraScale+ nhằm khai thác implementation diversity [11]. Một hướng nghiên cứu khác là SPREAD của Bow và cộng sự, trong đó DPR được kết hợp với nhiều hiện thực S-box khác nhau để thay đổi đặc tính phần cứng của AES theo thời gian [12].

Các công trình trên cho thấy DPR có thể được sử dụng như một cơ chế để thay đổi cấu trúc hiện thực của phần cứng mật mã trong thời gian chạy. Thiết kế trong báo cáo này sử dụng cùng

nguyên lý tổng quát nhưng tập trung vào việc xây dựng và đánh giá cơ chế chuyển đổi giữa bốn biến thể AES-256 trên Kria KV260. Các RM được thay đổi ở mức Core AES-256 hoàn chỉnh, trong khi phần đánh giá tập trung vào độ ổn định của quá trình tái cấu hình, thời gian chuyển đổi, ảnh hưởng đến workload và công suất ước lượng.

Do kích thước vùng tái cấu hình, nền tảng FPGA, đường nạp cấu hình và phạm vi đo của từng nghiên cứu khác nhau, thời gian tái cấu hình của đề tài không được đối chiếu trực tiếp bằng giá trị tuyệt đối với các công trình trên. Theo đặc điểm của DFX, thời gian cấu hình phụ thuộc vào lượng dữ liệu cấu hình cần truyền và băng thông của đường cấu hình [10]. Vì vậy, kết quả khoảng 6 ms của thiết kế hiện tại được sử dụng để đánh giá chính hệ thống trên KV260 và để so sánh với kiến trúc Replica được xây dựng trong cùng điều kiện.

Điểm khác biệt của phép đánh giá trong báo cáo là DPR và Replica được triển khai trên cùng nền tảng, sử dụng cùng bốn biến thể AES-256, cùng giao diện AXI4-Lite và cùng tập dữ liệu thử nghiệm. Cách tổ chức này cho phép tách rõ ảnh hưởng của cơ chế thay đổi biến thể khỏi thời gian thực hiện AES-256. Kết quả ở Mục 6.3 cho thấy thời gian mã hóa của hai kiến trúc gần tương đương, trong khi phần chênh lệch của workload chủ yếu xuất hiện ở thời gian tái cấu hình.

6.5.4 Phạm vi đánh giá và hướng phát triển

Các thực nghiệm trong báo cáo tập trung vào khả năng hoạt động của DPR, tính đúng đắn của AES-256 và những đánh đổi về thời gian chuyển đổi, hiệu năng và công suất. Chuỗi lựa chọn RM sử dụng seed cố định nhằm tạo lại cùng workload giữa các lần thử và giữa DPR với Replica. Phân bố thu được tương đối đồng đều giữa bốn RM, nhưng kết quả này chỉ phản ánh đặc điểm của chuỗi lựa chọn trong thực nghiệm, chưa phải đánh giá về độ an toàn của bộ sinh số ngẫu nhiên.

Tương tự, việc sử dụng nhiều biến thể phần cứng và thay đổi chúng bằng DPR tạo ra khả năng thay đổi cấu trúc hiện thực của AES-256 theo thời gian. Tuy nhiên, báo cáo hiện tại chưa thực hiện các phép đo power trace, Correlation Power Analysis (CPA), Differential Power Analysis (DPA), Test Vector Leakage Assessment (TVLA) hoặc Fault Injection. Vì vậy, kết quả của Chương 6 được giới hạn ở đánh giá kiến trúc và hiệu năng, không được sử dụng để kết luận mức tăng khả năng chống lại các tấn công side-channel hoặc fault attack.

Phần công suất cũng được lấy từ Vivado post-implementation power report với Confidence Level ở mức Medium và không sử dụng switching activity từ SAIF hoặc VCD. Các giá trị này phù hợp cho việc quan sát xu hướng giữa các cấu hình trong cùng môi trường thiết kế, nhưng chưa thay thế cho phép đo công suất trực tiếp trên phần cứng.

Từ các kết quả đã thu được, DPR phù hợp với các trường hợp cần thay đổi giữa nhiều biến thể phần cứng nhưng không yêu cầu tái cấu hình sau một số lượng rất nhỏ phép mã hóa. Kiến trúc đạt được khả năng thay đổi RM trong thời gian hoạt động và duy trì đúng chức năng AES-256 qua các phép thử liên tục, đổi lại thời gian nạp Partial Bitstream tạo ra chi phí đáng kể khi tần suất chuyển đổi tăng. Đây là sự đánh đổi chính được ghi nhận từ quá trình hiện thực và thực nghiệm trên Kria KV260. “‘

Tài liệu

[1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication 197 (FIPS 197), updated May 2023. Available at: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf.

[2] N. Quân, "[AES] Bài 1 – Lý thuyết về mã hóa AES-128," *VLSI Technology*, September 2019. Available at: https://nguyenquanicd.blogspot.com/2019/09/aes-bai-1-ly-thuyet-ve-ma-hoa-aes-128.html.

[3] D. Canright, "A Very Compact S-Box for AES," in *Cryptographic Hardware and Embedded Systems – CHES 2005*, Lecture Notes in Computer Science, vol. 3659, pp. 441–455, 2005. doi: 10.1007/11545262_32.

[4] AMD, *Vivado Design Suite User Guide: Synthesis*, UG901, Version 2026.1, July 2026. Available at: https://docs.amd.com/r/en-US/ug901-vivado-synthesis.

[5] AMD, *Vivado Design Suite User Guide: Power Analysis and Optimization*, UG907, Version 2026.1, June 2026. Available at: https://docs.amd.com/r/en-US/ug907-vivado-power-analysis-optimization.

[6] A. Sajadi, N. Zidaric, T. Stefanov, and N. Mentens, "A Systematic Comparison of Side-channel Countermeasures for RISC-V-based SoCs," in *2024 IEEE Nordic Circuits and Systems Conference (NorCAS)*, pp. 1–7, 2024. doi: 10.1109/NorCAS64408.2024.10752477.

[7] AMD, *Zynq UltraScale+ Device Technical Reference Manual*, UG1085, Rev. 2.5, March 2025. Available at: https://docs.amd.com/v/u/en-US/ug1085-zynq-ultrascale-trm.

[8] AMD, *Vivado Design Suite User Guide: Design Analysis and Closure Techniques*, UG906, Version 2026.1, June 2026. Available at: https://docs.amd.com/r/en-US/ug906-vivado-design-analysis.

[9] Arm Limited, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H, 2020. Available at: https://support.arm.com/documentation/ihi0022/h.

[10] AMD, *Vivado Design Suite User Guide: Dynamic Function eXchange*, UG909, Version 2026.1, July 2026. Available at: https://docs.amd.com/r/en-US/ug909-vivado-partial-reconfiguration.

[11] B. Hettwer, J. Petersen, S. Gehrer, H. Neumann, and T. Güneysu, "Securing Cryptographic Circuits by Exploiting Implementation Diversity and Partial Reconfiguration on FPGAs," in *Proceedings of the Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 260–263, 2019. doi: 10.23919/DATE.2019.8714801.

[12] I. Bow, N. Bete, F. Saqib, W. Che, C. Patel, R. Robucci, C. Chan, and J. Plusquellic, "Side-Channel Power Resistance for Encryption Algorithms Using Implementation Diversity," *Cryptography*, vol. 4, no. 2, Art. no. 13, 2020. doi: 10.3390/cryptography4020013.